

Large Vision Models

Dr. Jingjing Deng
Senior Lecturer in AI, SEMT
University of Bristol, UK



- 1980s - Rediscover Back-propagation
- 1990s - Winter for Neural Networks
 - LeNet Handwritten Digits Recognition
- 2000s - Era of Internet & Gaming
 - Massive Data & GPGPUs
- 2010s - Early Success & Boom of ANN
 - AlexNet (CNN) on ImageNet
 - LSTM & Transformers (Attention)
- 2024 - Nobel Prize Winners
 - Physics - Artificial Neural Networks
 - Chemistry - AlphaFold Protein Design

The Nobel Prize in Physics 2024

John J. Hopfield

“for foundational discoveries and inventions that enable machine learning with artificial neural networks”



John Hopfield. Ill. Niklas Elmehed © Nobel Prize Outreach

Geoffrey Hinton

“for foundational discoveries and inventions that enable machine learning with artificial neural networks”



Geoffrey Hinton. Ill. Niklas Elmehed © Nobel Prize Outreach

The Nobel Prize in Chemistry 2024

David Baker

“for computational protein design”



David Baker. Ill. Niklas Elmehed © Nobel Prize Outreach

Demis Hassabis

“for protein structure prediction”



Demis Hassabis. Ill. Niklas Elmehed © Nobel Prize Outreach

John Jumper

“for protein structure prediction”

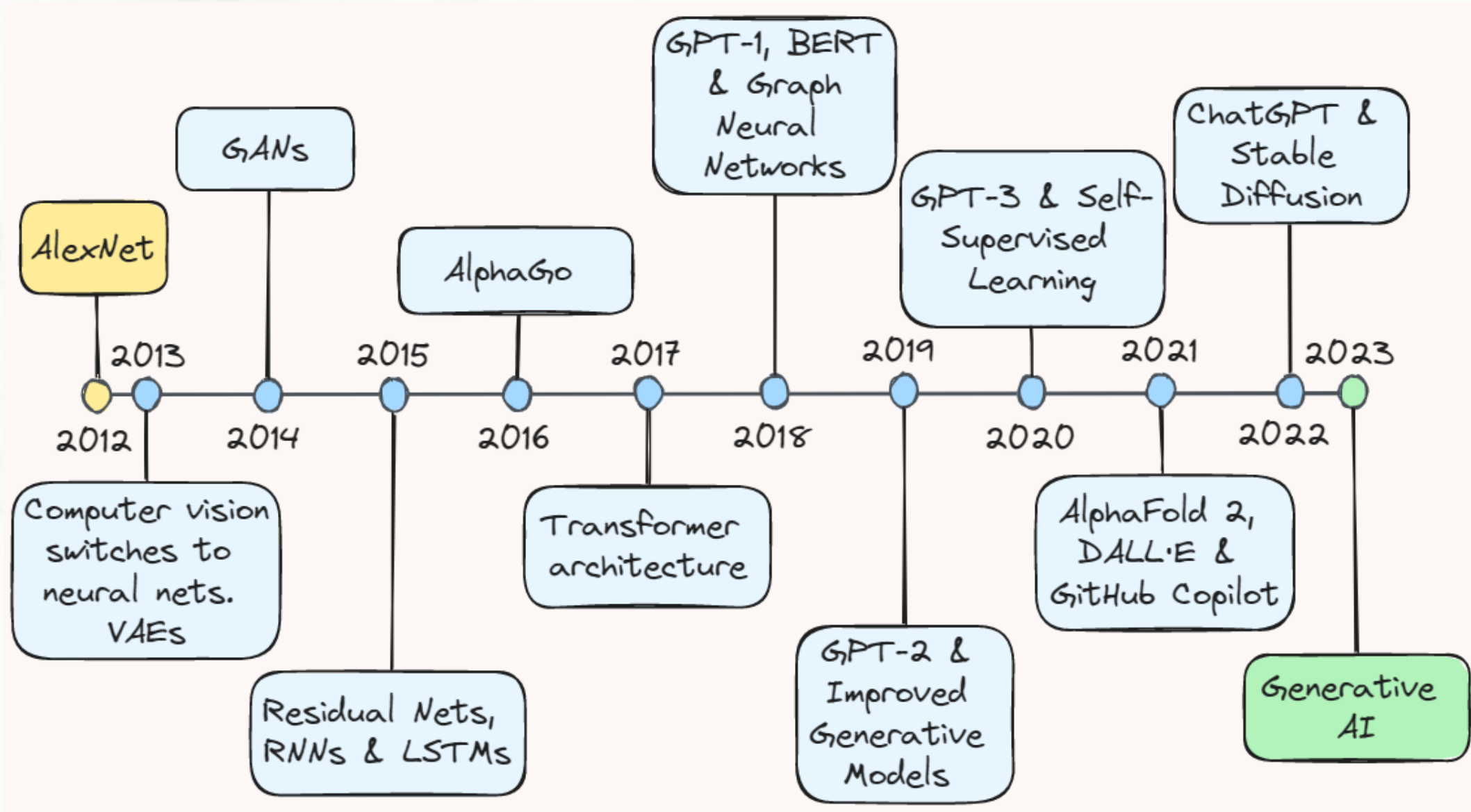


John Jumper. Ill. Niklas Elmehed © Nobel Prize Outreach

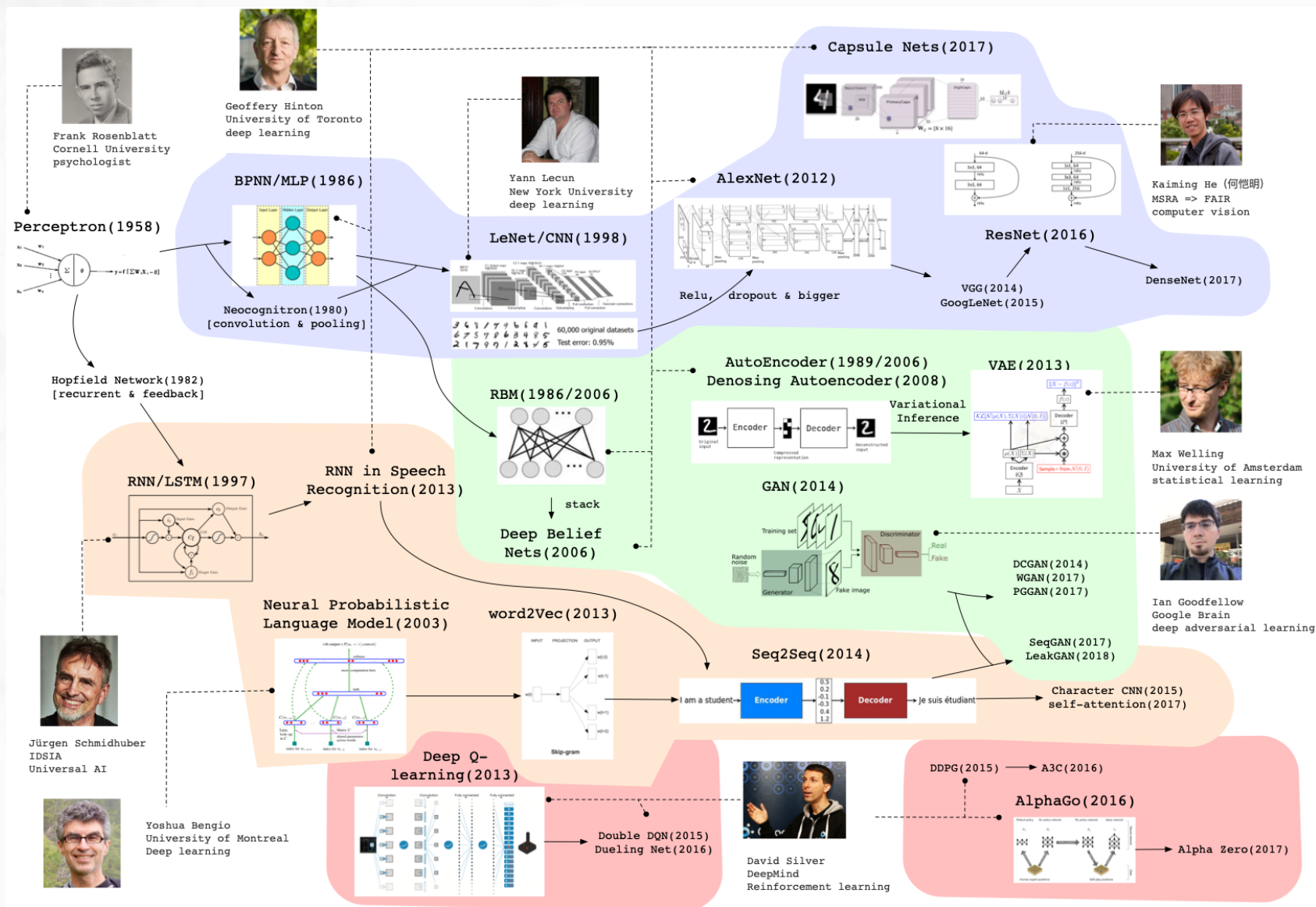
1. Rumelhart, D.E., Hinton, G.E. and Williams, R.J., 1986. Learning representations by back-propagating errors. nature, 323(6088), pp.533-536.

2. <https://www.nobelprize.org/all-nobel-prizes-2024/>

“Deep” Learning Driven AI Era



An Incomplete Map of LVMs



1 <http://keg.cs.tsinghua.edu.cn/lietang/>

2 <https://blog.sylarhai.com/introduction-to-large-language-models>

Go Deeper – Scalable Networks

Scale CNNs & Transformer Backbone for Hierarchical Representation Learning

The Era of “Bigger Data, Bigger Challenge”

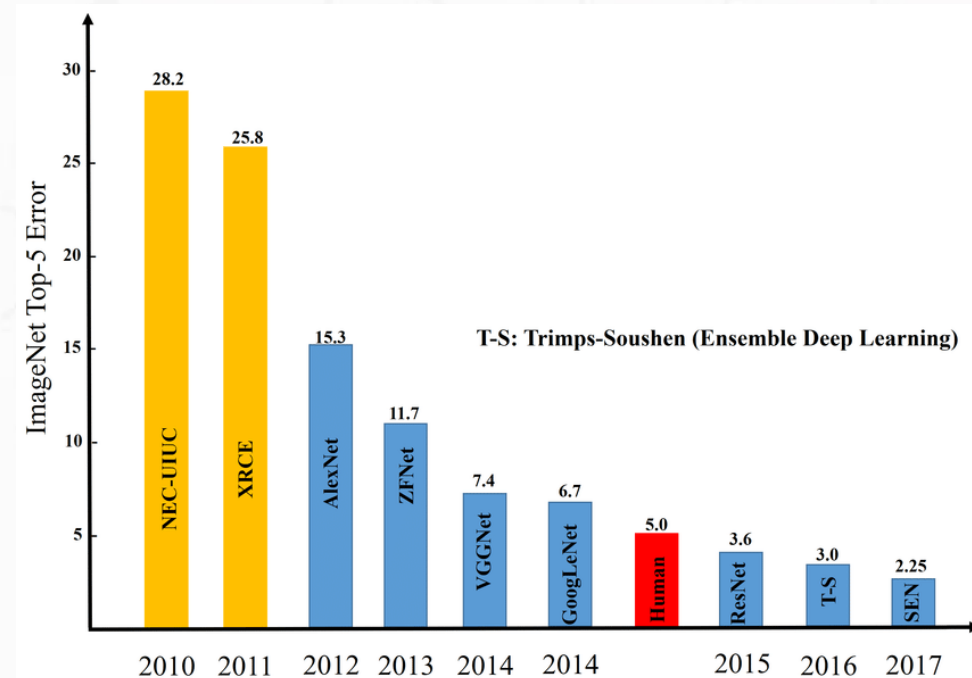


Mammal → Placental → Carnivor → Canine → Dog → Working Dog → Siberian Husky



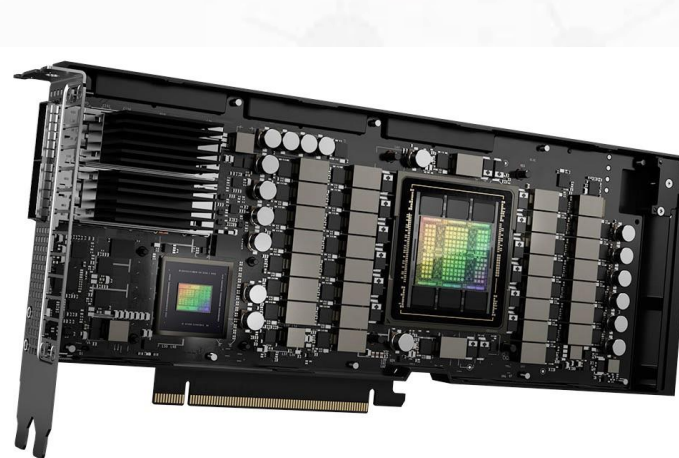
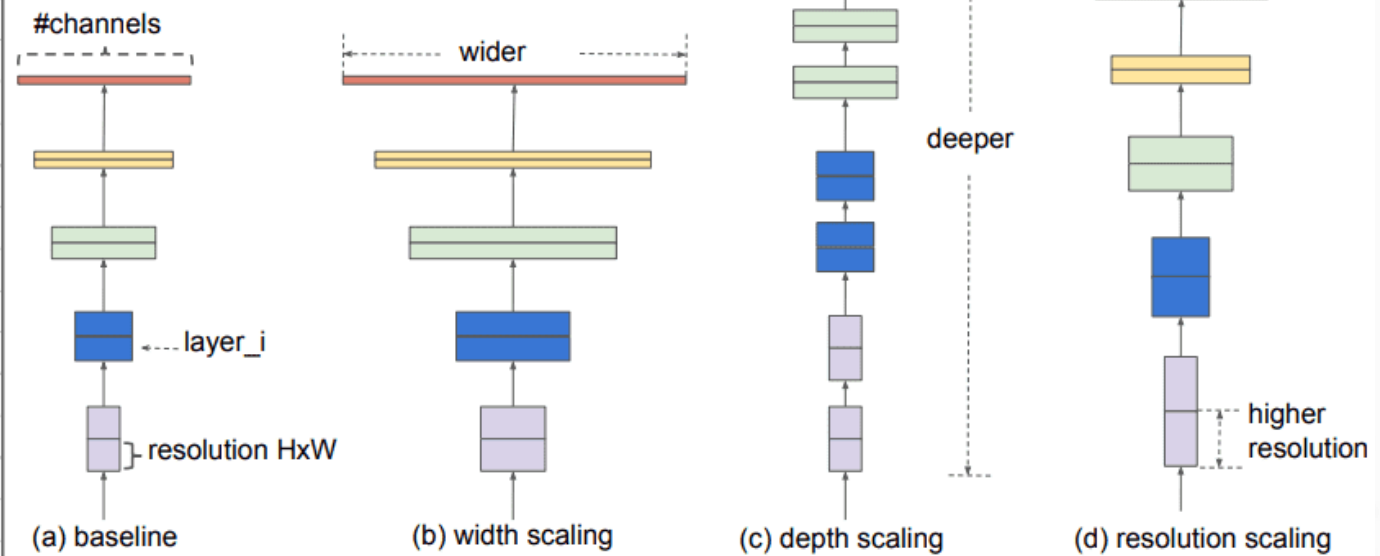
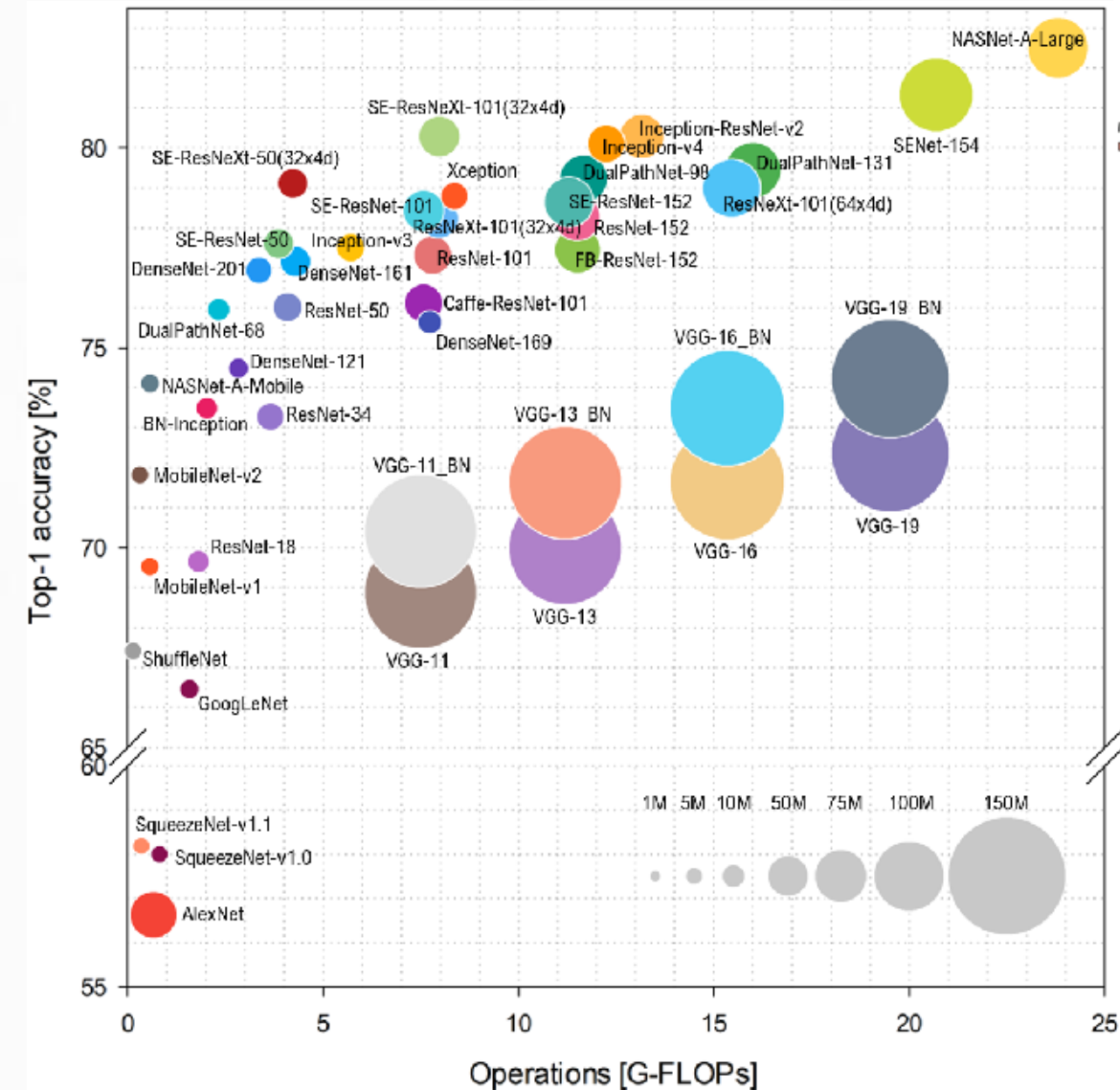
Vehicle → Craft → Watercraft → Sailing Vessel → Sailboat → Trimaran

Dataset split	Extended ImageNet		ILSVRC CLS 2012	
	Train	Val.	Train	Val.
# of images	2,456,727	273,140	1,281,167	50,000
Avg im # per cls	251-1300	34-50	732-1300	50
Avg anno per im	1.53	1.17	1.41	1.02
Avg obj scale	25.37 %	25.50%	25.39%	25.61 %
Small obj %	4.81	4.27	2.35	2.47
Twisted obj %	42.77	44.55	40.72	42.94
Inner-cls distance	0.434	0.396	0.462	0.411
Inter-cls distance	1.12	1.46	1.52	1.55



- <https://image-net.org/index.php>
- <https://www.pinecone.io/learn/series/image-search/imagenet/>
- https://www.researchgate.net/figure/Statistics-comparison-between-Extended-ImageNet-Classification-FIC-dataset-and-ILSVRC_tbl1_317101547

Benchmark CNN Architectures



1. Bianco, Simone, et al. "Benchmark analysis of representative deep neural network architectures." IEEE access 6 (2018): 64270-64277.

2. <https://theaisummer.com/cnn-architectures/>

3. <https://www.nvidia.com/en-gb/>

AlexNet (2012) – The First Breakthrough of DL

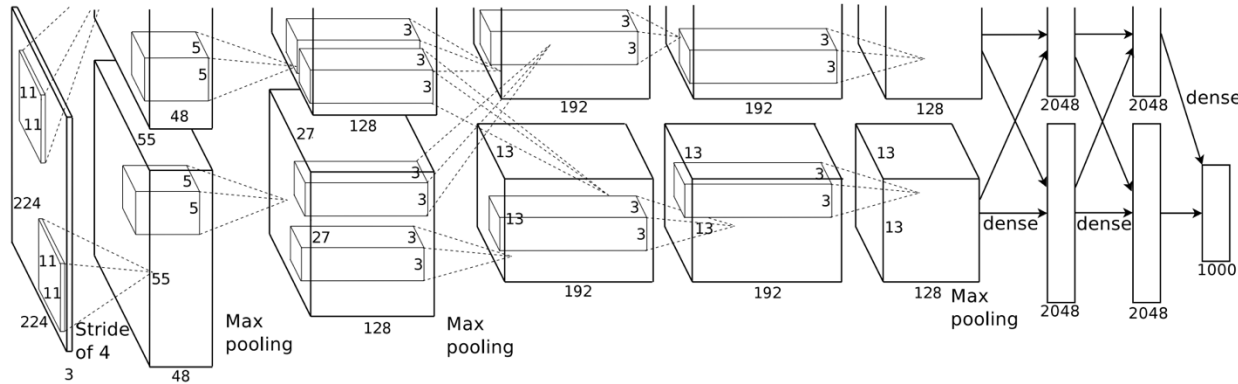


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

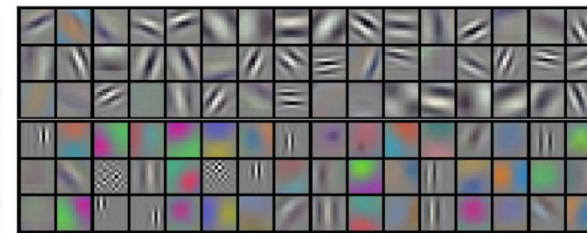
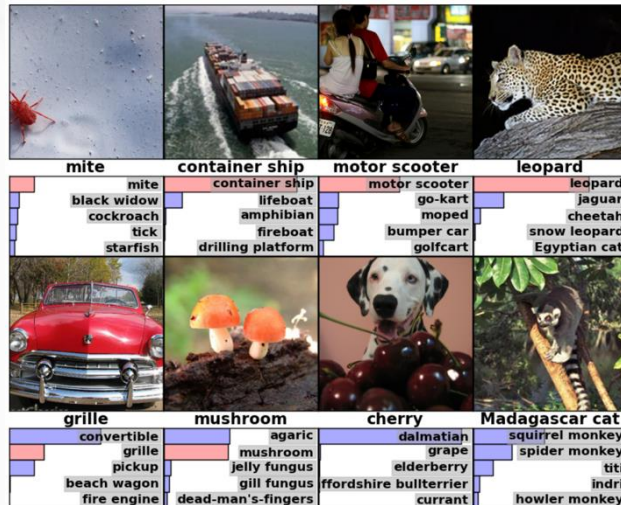
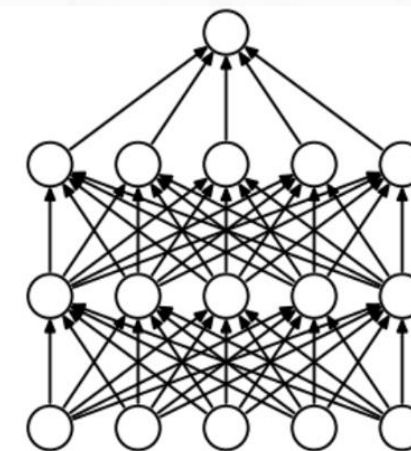
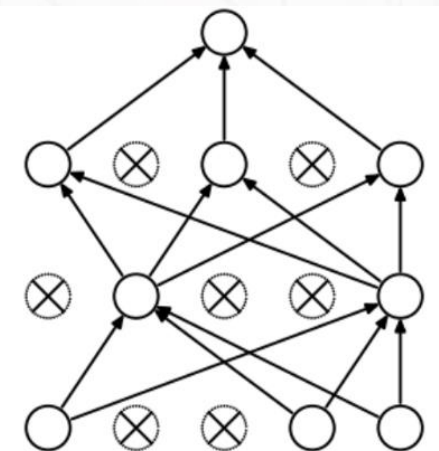


Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

- Rectified Linear Unit (ReLU) as the activation function.
- Local Response Normalization (LRN) as “Lateral Inhibition”.
- Group Convolution.
- Overlapping Pooling.
- Data Augmentation.
- Dropout at Fully Connected Layer.
- Dual GPUs Training (Nvidia GTX 580 3GiB GPU).
- Batched SGD, Weight Decay, Momentum.
- “Confidence” in DL.
 - Top5 Error Rate on Test Set
 - 28.2% (2010), 25.8% (2011) to 16.4% (2012).

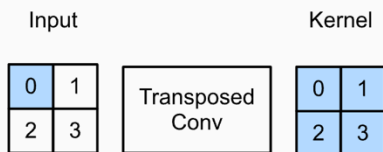
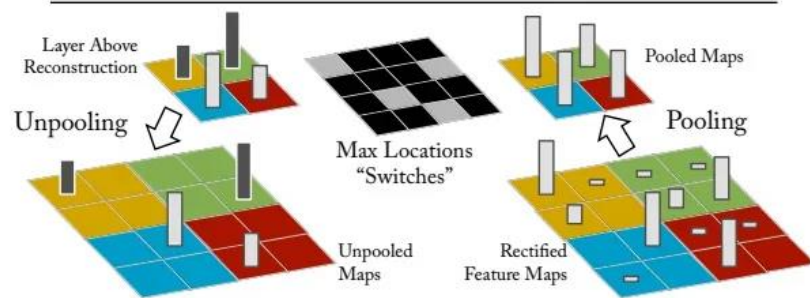
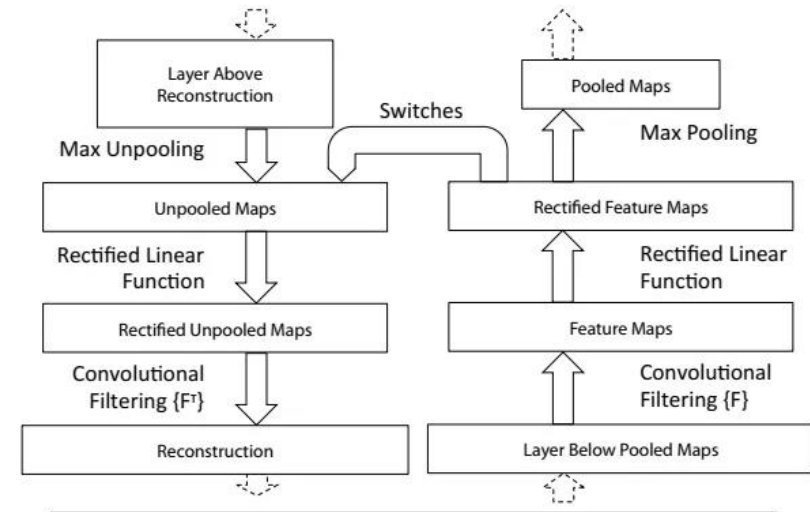


(a) Standard Neural Net



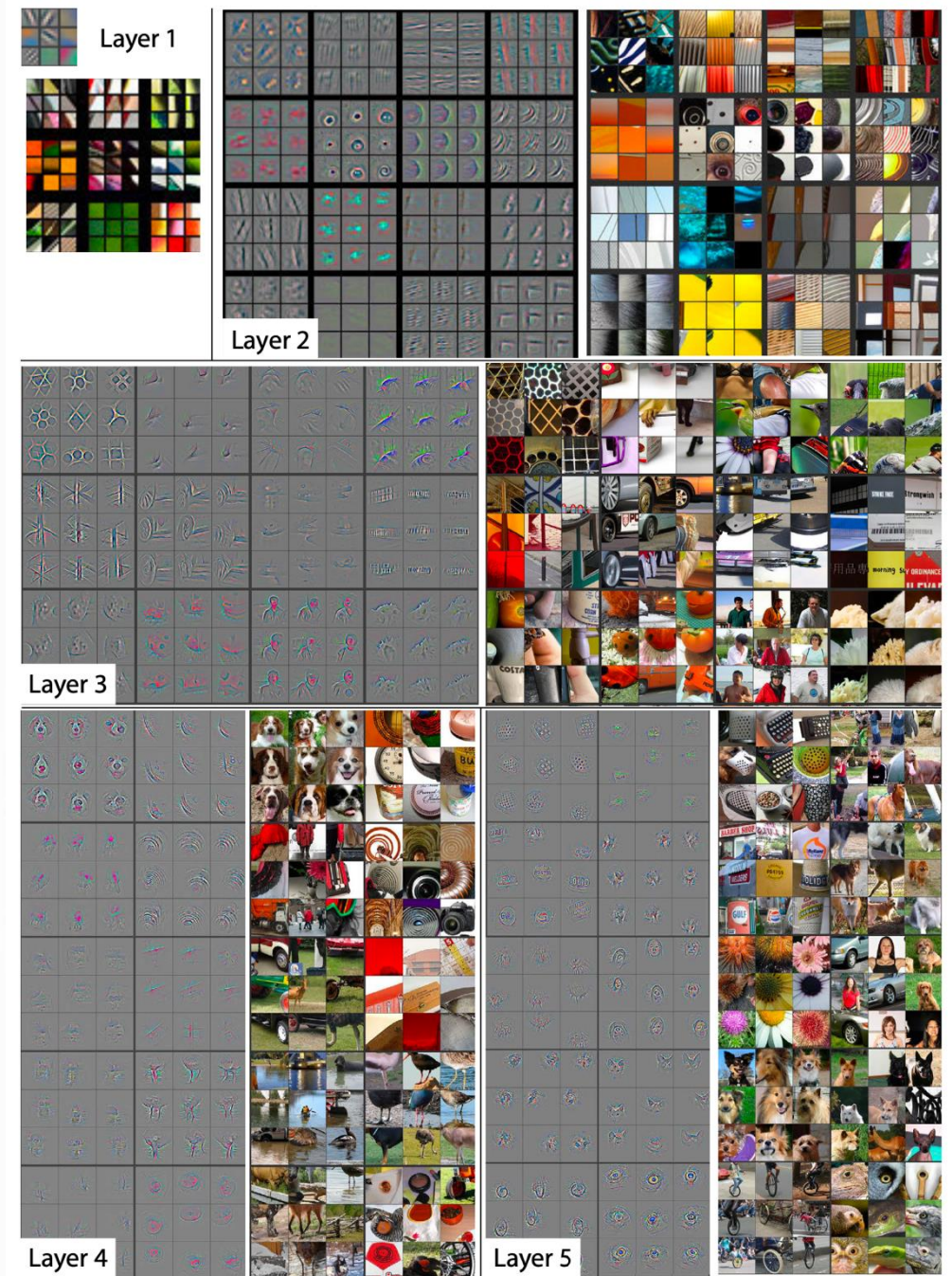
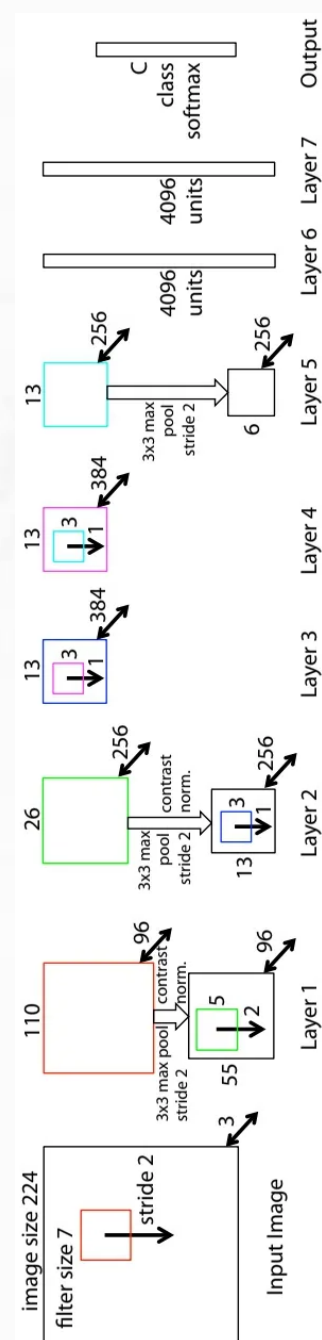
(b) After applying dropout.

ZFNet (2013)



Output

$$= \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} + \begin{bmatrix} 0 & 2 \\ 4 & 6 \end{bmatrix} + \begin{bmatrix} 0 & 3 \\ 6 & 9 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 4 & 6 \\ 4 & 12 & 9 \end{bmatrix}$$



VGG16



AlexNet

Input

Pool

3x4 conv, 256

3x4 conv, 192

Pool

FC 4096

FC 4096

FC 1000

Softmax

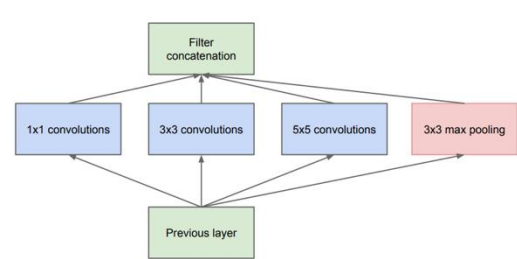
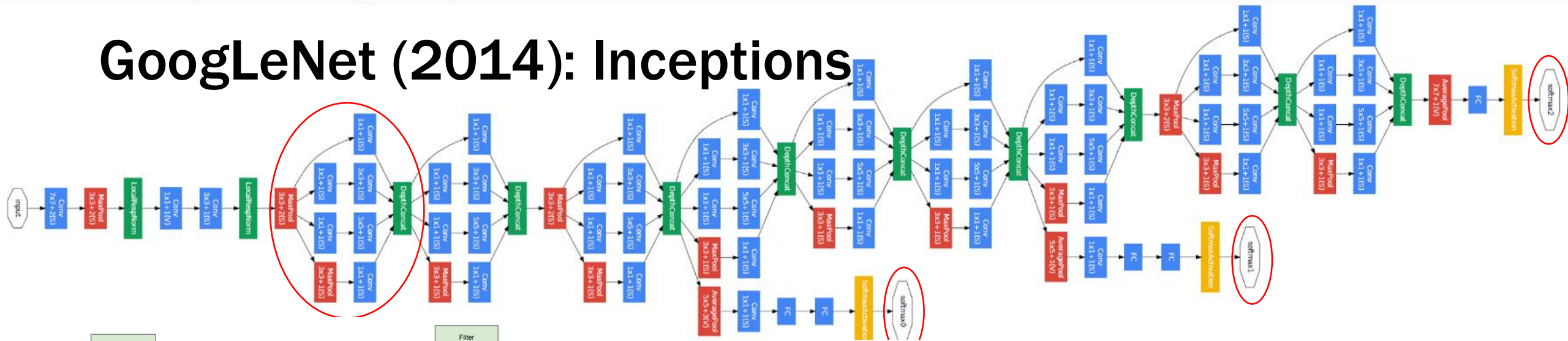
Legend:

- CONV
- ACT
- POOL
- FC
- CLASS

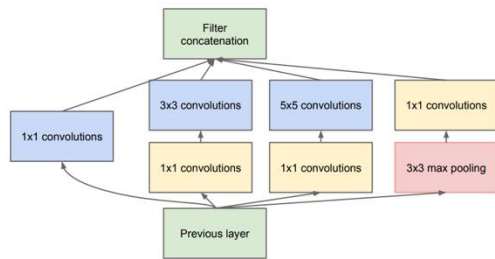
- Deeper Stem Structure.
- Denser Fully Connected Layer.
- Stacking Small (3x3) CNN Kernels.
- Max-Pooling (2x2).
- Building Block.
- Pretrained Model.
- Typical Training Strategies.



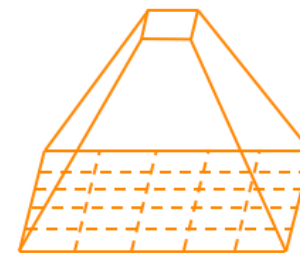
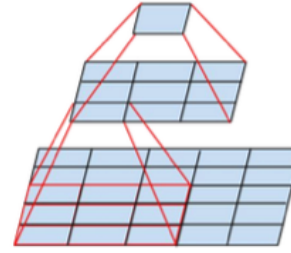
GoogLeNet (2014): Inceptions



(a) Inception module, naïve version

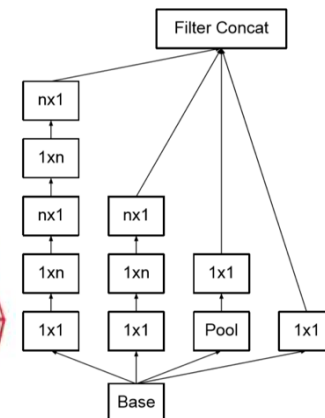
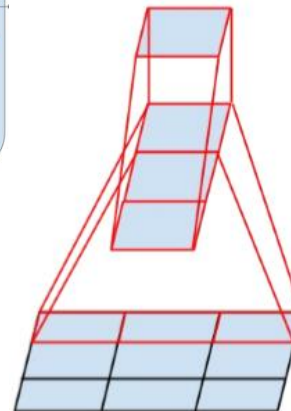


(b) Inception module with dimension reductions



two successive 3x3 convolutions

5x5 convolution

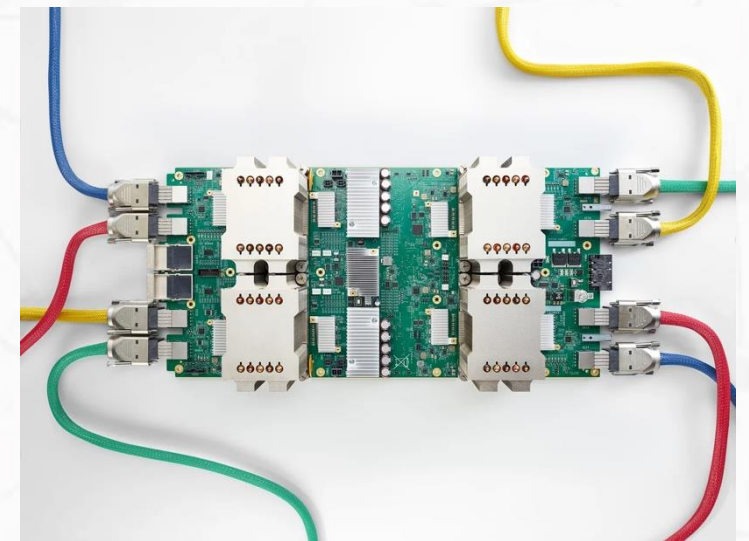


Inception V1: GoogLeNet, Scale the “Width”.

Inception V2: Batch Normalisation.

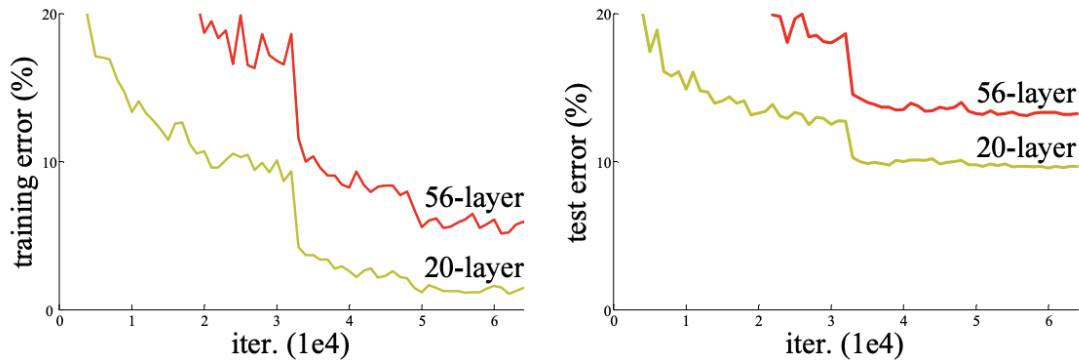
Inception V3: Factorizing Convolutions.

Inception V4: Combine Residual Block.



1. Szegedy, Christian, et al. "Going deeper with convolutions." Proceedings of the IEEE conference on computer vision and pattern recognition. 2015.
2. Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." International conference on machine learning. pmlr, 2015.
3. Szegedy, Christian, et al. "Rethinking the inception architecture for computer vision." Proceedings of the IEEE conference on computer vision and pattern recognition. 2016.
4. Szegedy, Christian, et al. "Inception-v4, inception-resnet and the impact of residual connections on learning." Proceedings of the AAAI conference on artificial intelligence. Vol. 31. No. 1. 2017.
5. <https://towardsdatascience.com/batch-normal-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739>
6. <https://www.jeremyjordan.me/convnet-architectures/>

ResNet (2015): Learnable & Adaptive Depth



- Network Degradation Problem.
- Training Difficulty due to #Parameters, #Layer etc.
- Novel Residual Connection.
- Learnable & Adaptive Network Depth.
- Simple but Effective Solution, Widely Adopted.

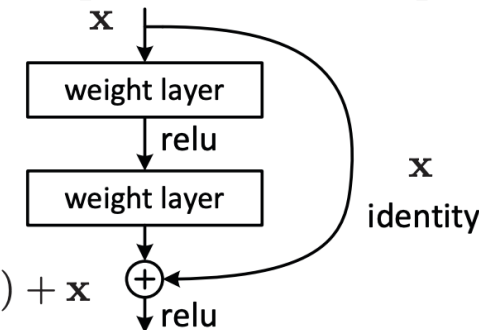
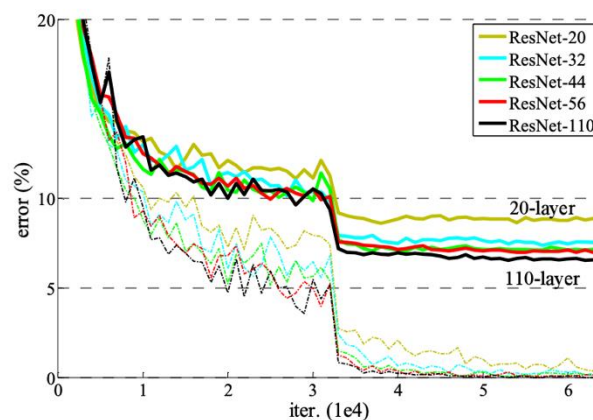
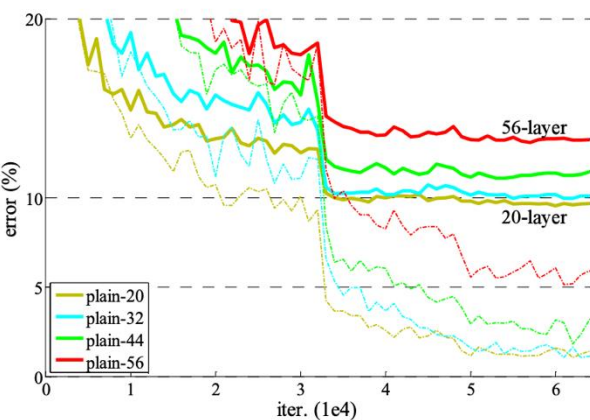


Figure 2. Residual learning: a building block.

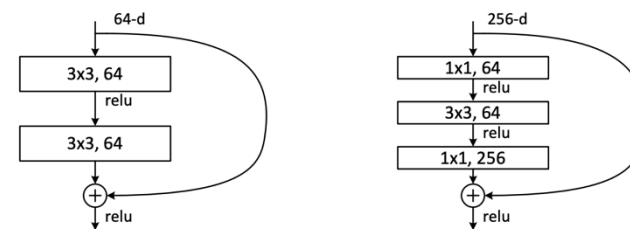
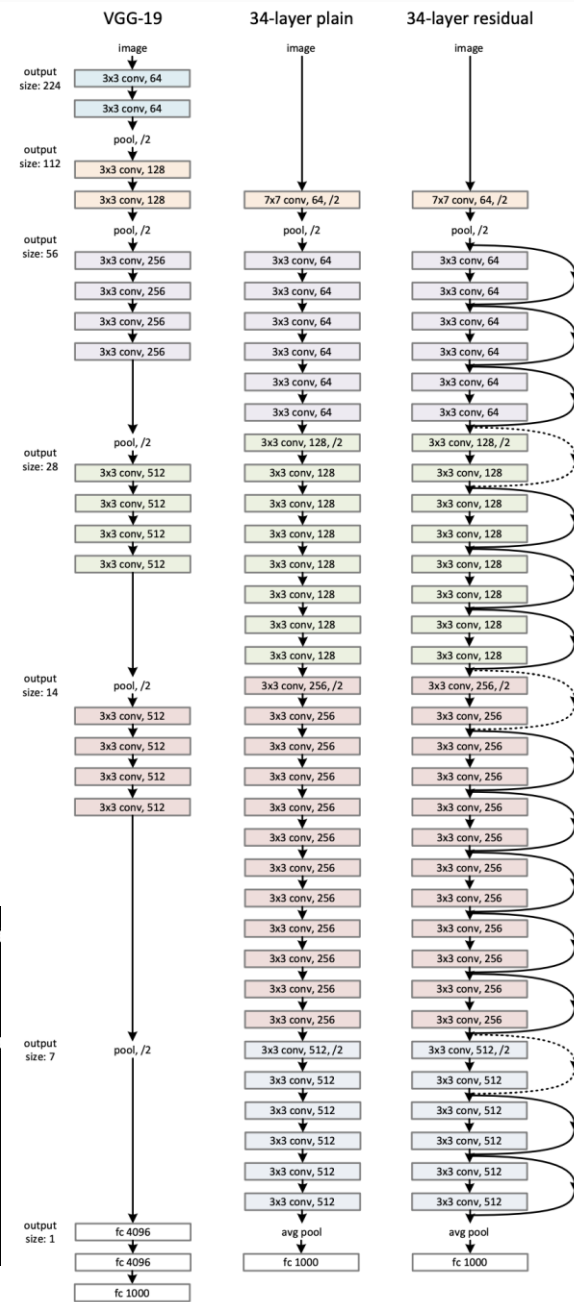


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.



1. He, Kaiming, et al. "Deep residual learning for image recognition." Proceedings of the IEEE conference on computer vision and pattern recognition. 2016.

2. <https://www.deepmind.com/research/highlighted-research/alphago>

DenseNet (2017): Skipping Anything

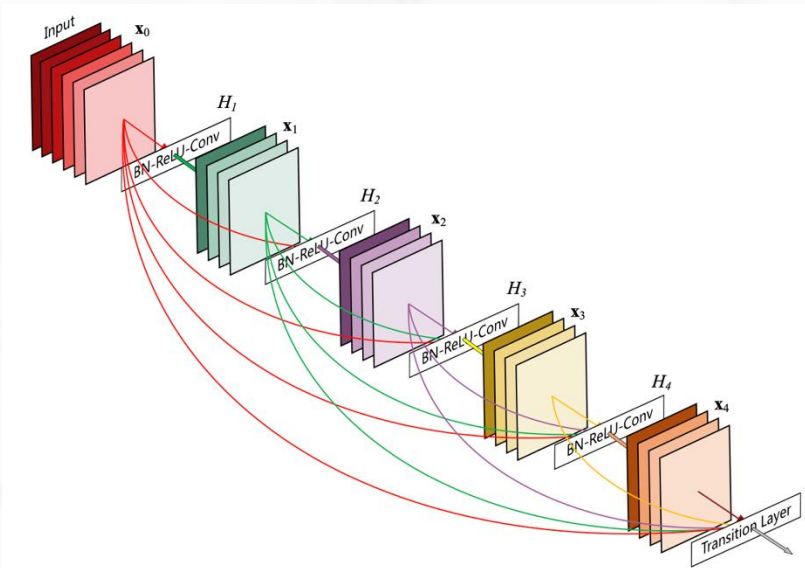


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

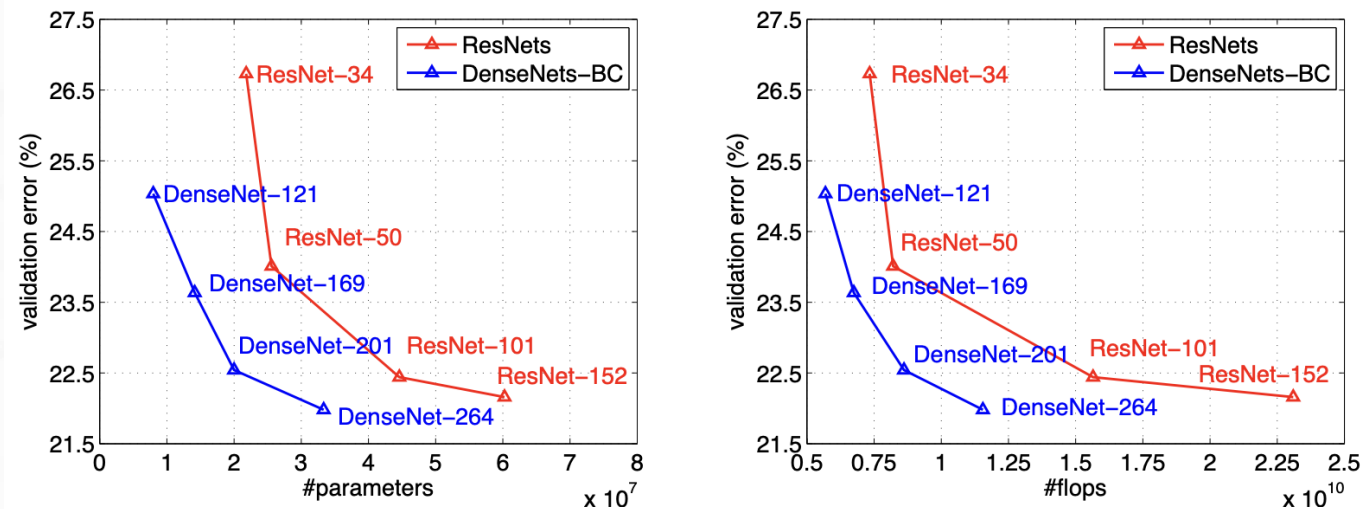


Figure 3: Comparison of the DenseNets and ResNets top-1 error rates (single-crop testing) on the ImageNet validation dataset as a function of learned parameters (*left*) and FLOPs during test-time (*right*).

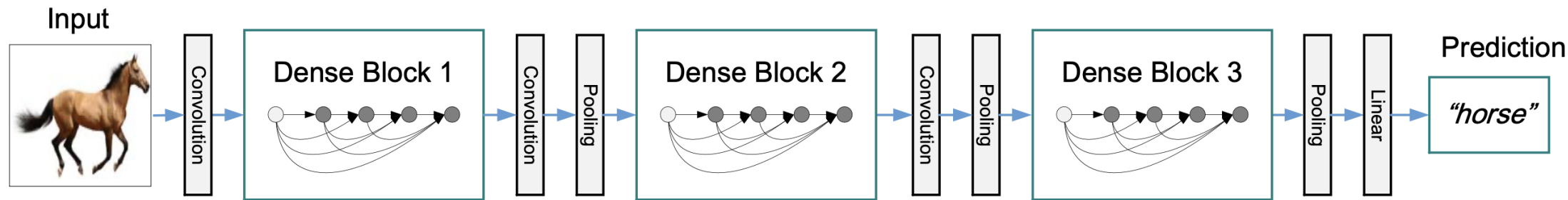


Figure 2: A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

Trimps-Soushen (2016): Ensemble of Deep Models

Error Categories	Numbers	Percentages(%)
Label May Wrong	221	15.16
Multiple Objects (>5)	118	8.09
Non-Obvious Main Object	355	24.35
Confusing Label	206	14.13
Fine-grained Label	258	17.70
Obvious Wrong	234	16.05
Partial Object	66	4.53

Predict:
1 pencil box
2 diaper
3 bib
4 purse
5 running shoe

Ground Truth:
sleeping bag



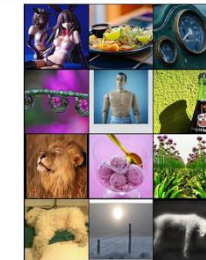
Predict:
1 dock
2 submarine
3 boathouse
4 breakwater
5 lifeboat

Ground Truth:
paper towel



Predict:
1 lion
2 web site
3 frying pan
4 teddy
5 pop bottle

Ground Truth:
ice cream



Predict:
1 apron
2 plastic bag
3 sleeping bag
4 umbrella
5 bulletproof vest

Ground Truth:
poncho



Predict:
1 carton
2 packet
3 toilet tissue
4 vending machine
5 crate

Ground Truth:
sunscreen

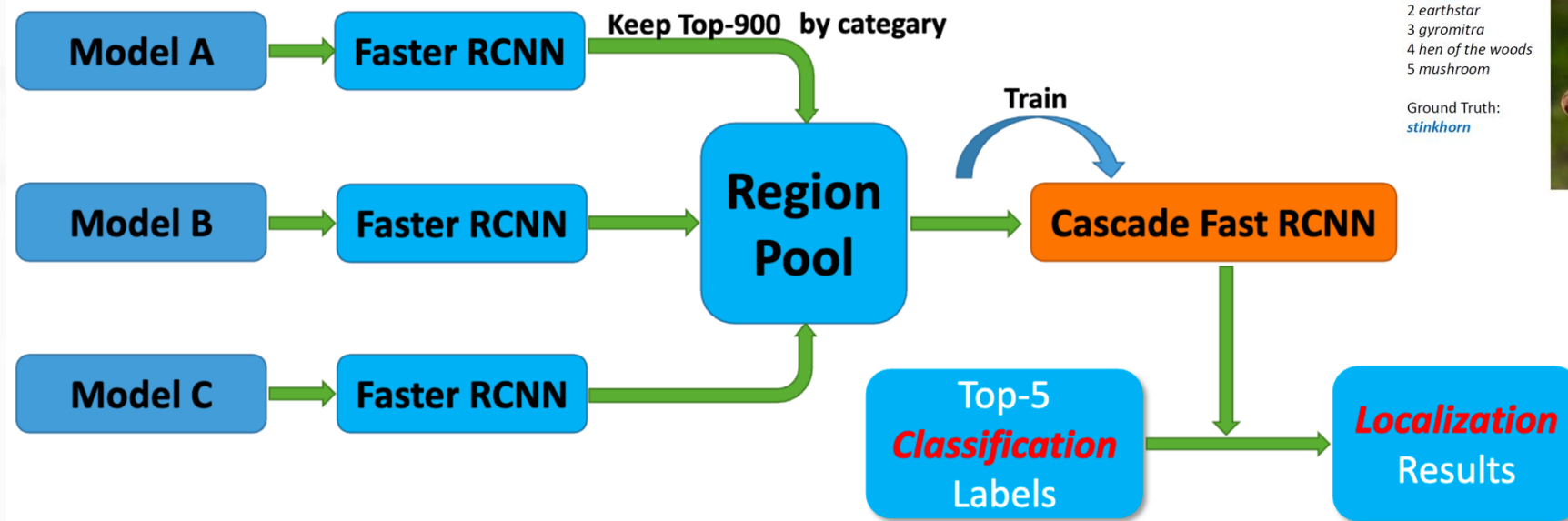


Predict:
1 plate
2 carbonara
3 chocolate sauce
4 bakery
5 ice cream

Ground Truth:
restaurant



- **Object Localization**
 - 1st place, 7.71% error
- **Object Classification**
 - 1st place, 2.99% error
- **Object Detection**
 - 3rd place, 0.618 mAP
- **Scene Classification**
 - 3rd place, 10.30% error
- **Object Detection from video**
 - 3rd place, 0.71 mAP

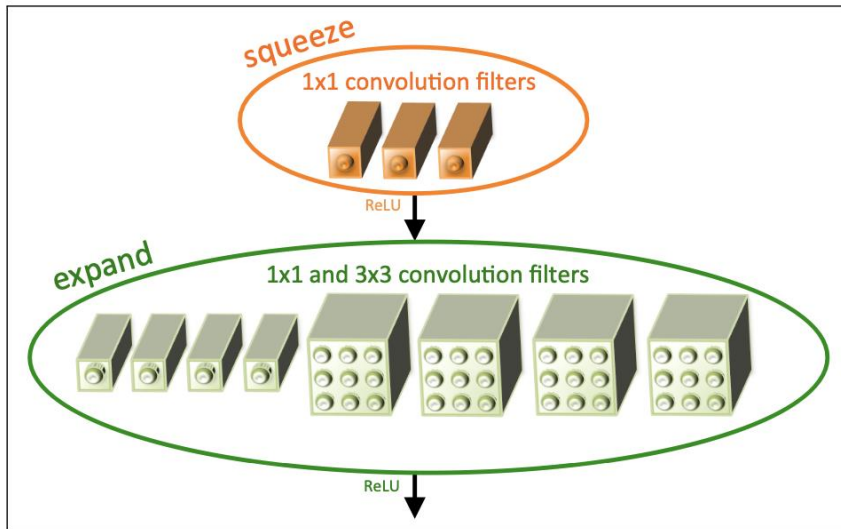


Predict:
1 bolete
2 earthstar
3 gyromitra
4 hen of the woods
5 mushroom

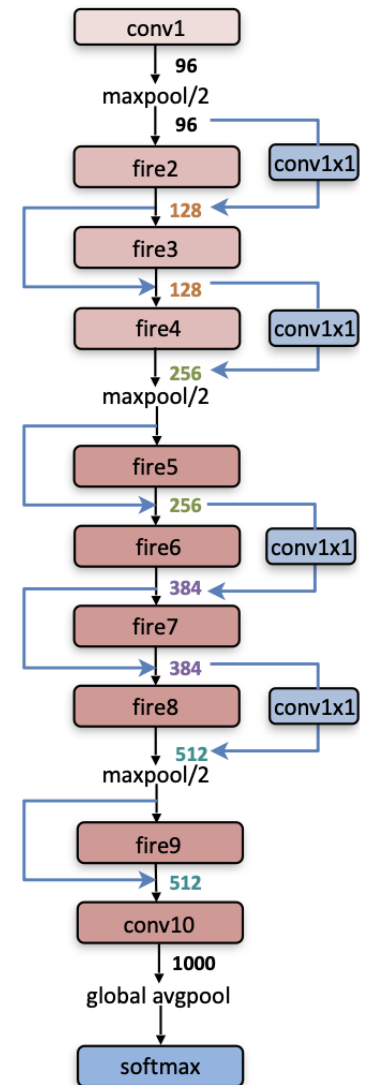
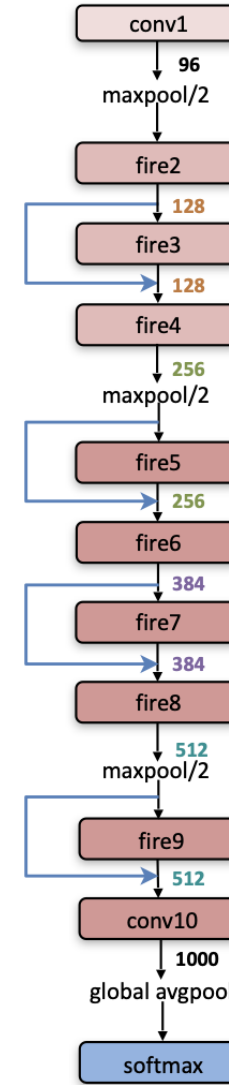
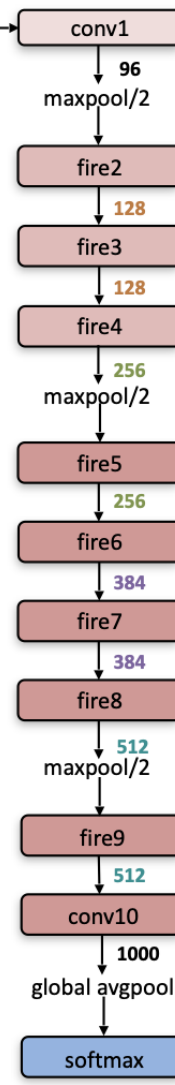
Ground Truth:
stinkhorn



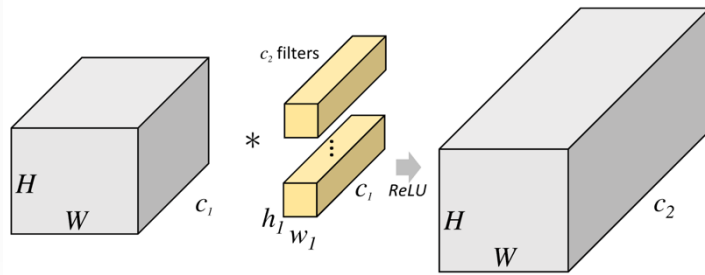
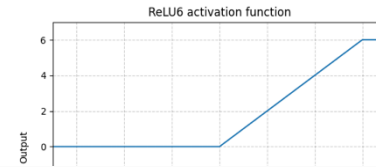
SqueezeNet (2017): Model Compression Perspective



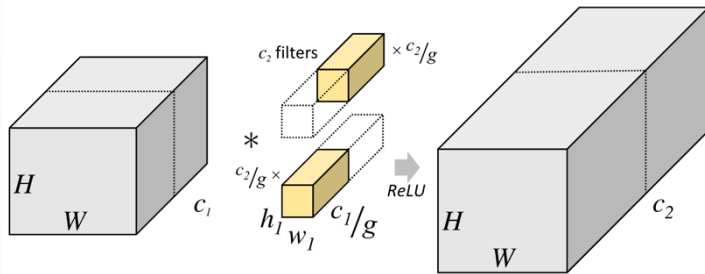
- Objective 1: Model Compression for Embedded System.
 - Reduce #Parameter & #Operations
- Objective 2: Minimum Compromise on Accuracy.
 - Top-1 & Top-5 Error Rate
- Strategy 1: Replace 3x3 filters with 1x1 filters.
 - 1x1 filter has 9X fewer parameters than a 3x3 filter.
- Strategy 2. Decrease the number of input channels to 3x3 filters.
 - $s_{1x1} < e_{1x1} + e_{3x3}$
- Strategy 3. Downsample late in the network so that convolution layers have large activation maps.
 - Large activation maps can lead to higher classification accuracy



MobileNet (2017): Separable Convolution

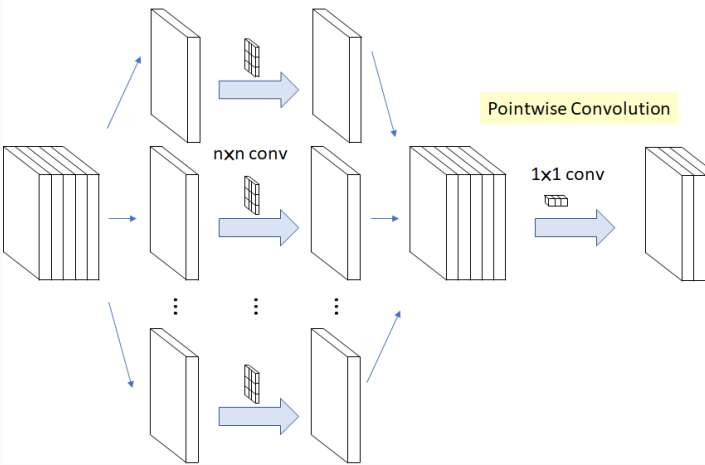


A normal convolutional layer. Yellow blocks represent learned parameters, gray blocks represent feature maps/input images (working memory).

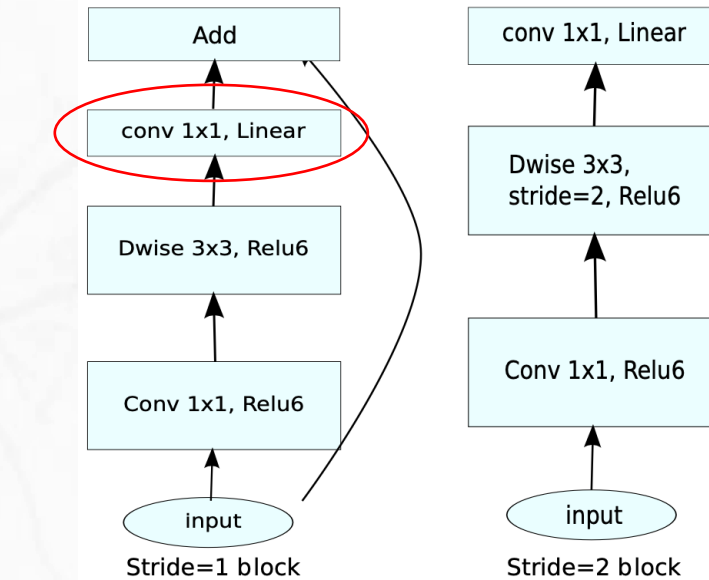


A convolutional layer with 2 filter groups. Note that each of the filters in the grouped convolutional layer is now exactly half the depth, i.e. half the parameters and half the compute as the original filter.

Depthwise Convolution

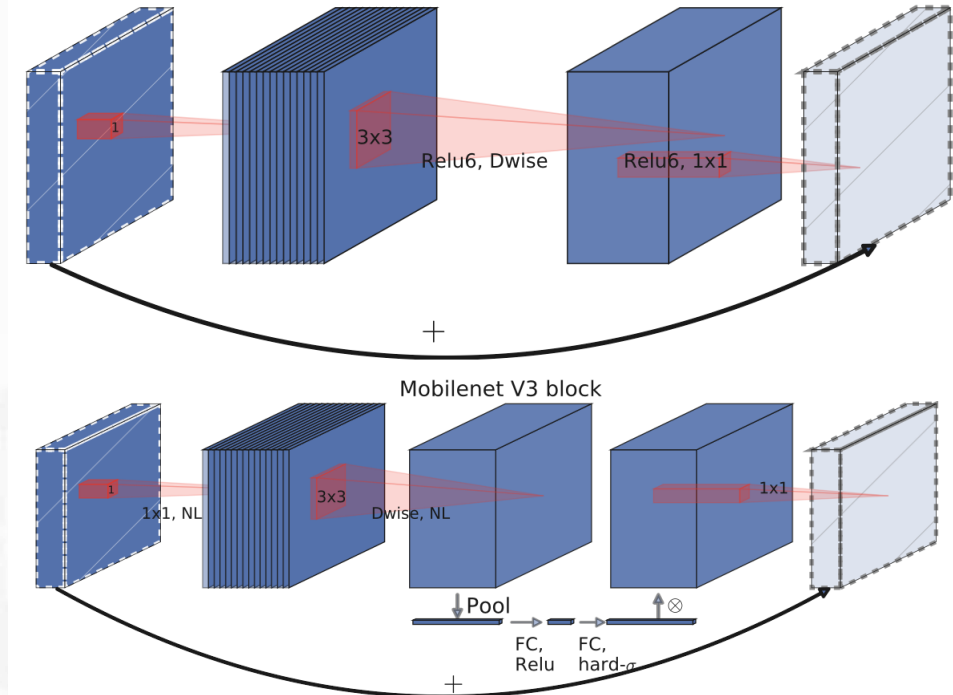


Pointwise Convolution

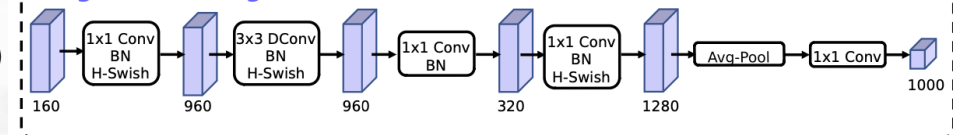


- For Mobile Applications & Embedded Systems
- MobileNet V1
 - Depth-Wise (Channel Independent) Convolution
 - Point-Wise (Spatial Independent) Convolution
- MobileNet V2
 - Inverted Residual (Compress Dim. -> Expand Dim.)
 - Linear Bottleneck Mitigates ReLU Suppression
- MobileNet V3
 - Squeeze-and-Excite Operation, H-Swish Activation
 - Network Architecture Search
 - New Design of Last Stage (Fully Connected Layer)

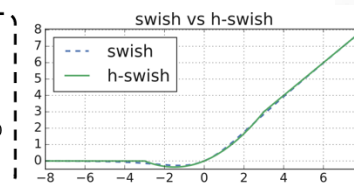
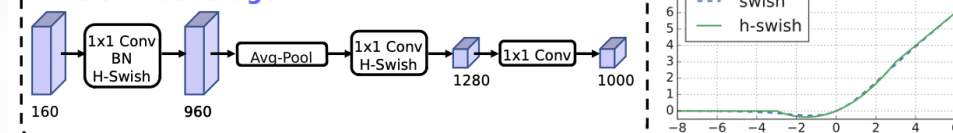
MobileNet V2: bottleneck with residual



Original Last Stage



Efficient Last Stage

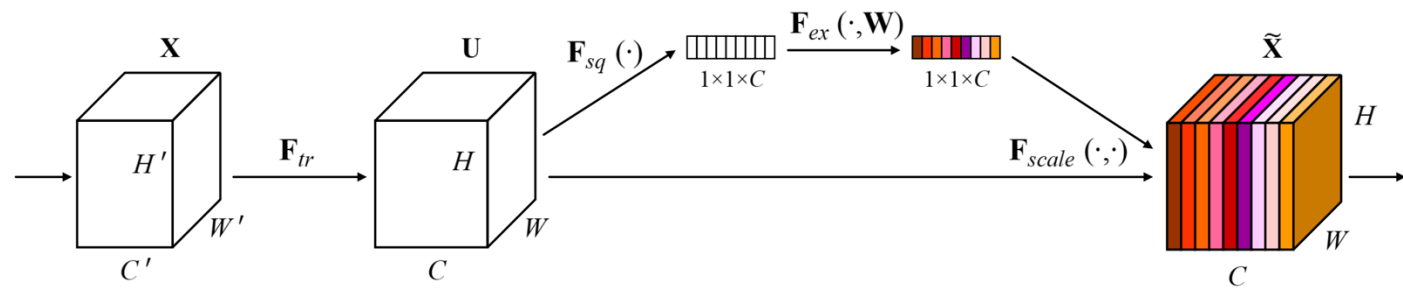
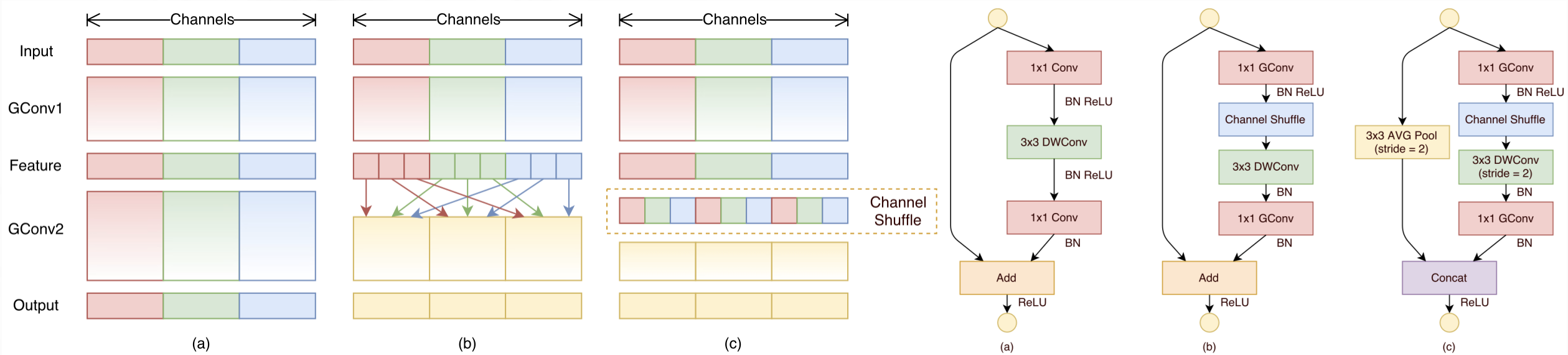


1. Howard, Andrew G., et al. "Mobilenets: Efficient convolutional neural networks for mobile vision applications." arXiv preprint arXiv:1704.04861 (2017).

2. <https://blog.yani.ai/filter-group-tutorial/>

3. <https://towardsdatascience.com/review-xception-with-depth-wise-separable-convolution-better-than-inception-v3-image-dc967dd42568>

ShuffleNet & SENet (2017): Are Channels Learnt to be Equal?



- **Equally Weighted: Channel Shuffling**
 - Point-wise Group Convolution + Depth-wise Convolution
- **Unequally Weighted: Channel Squeeze & Excitement**
 - Learnable Channel-wise Feature Map Scaling
- General Building Block Used with Others

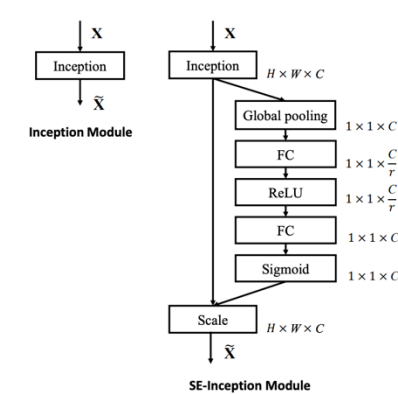


Fig. 2. The schema of the original Inception module (left) and the SE-Inception module (right).

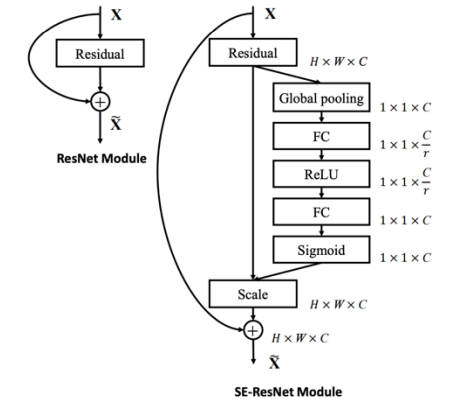
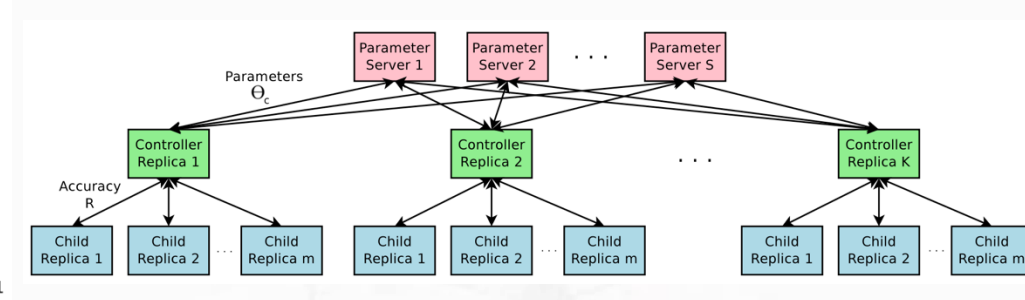
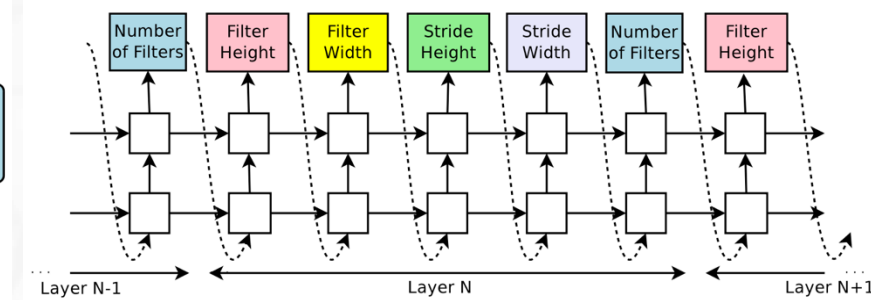
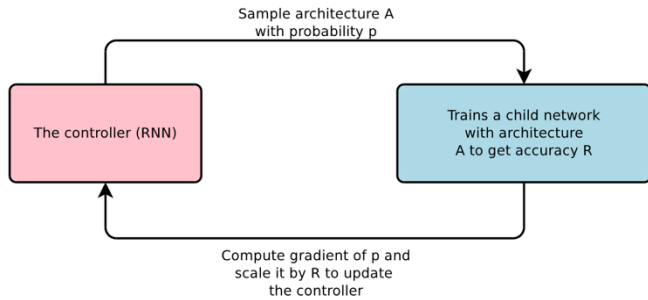


Fig. 3. The schema of the original ResNet module (left) and the SE-ResNet module (right).

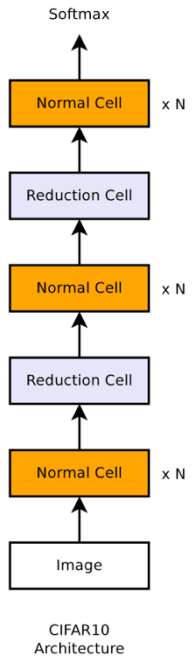
1. Hu, Jie, Li Shen, and Gang Sun. "Squeeze-and-excitation networks." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.

2. Zhang, Xiangyu, et al. "Shufflenet: An extremely efficient convolutional neural network for mobile devices." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.

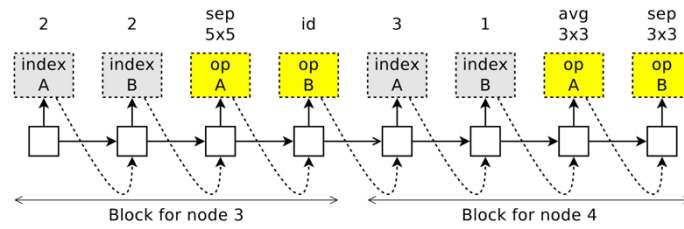
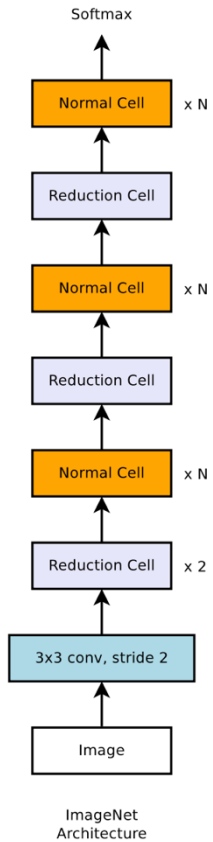
Network Architecture Searching: NN Automation?



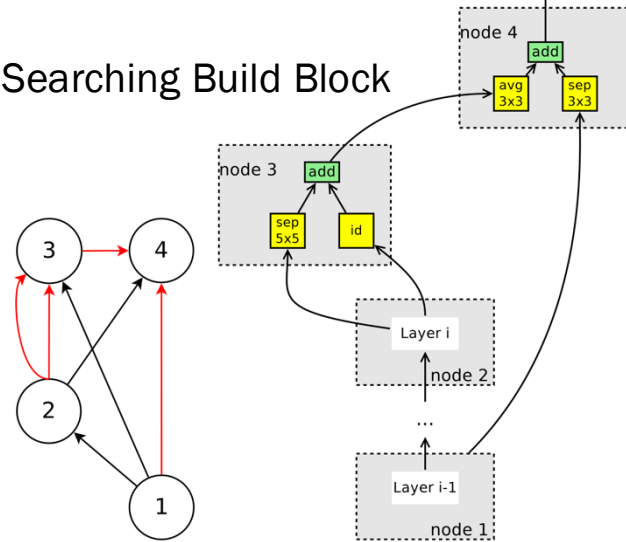
Predefined NASNet Searching Space



Transfer



Searching Build Block



Searching Space

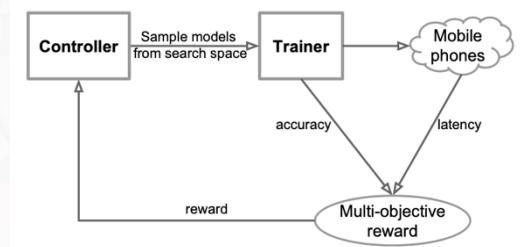
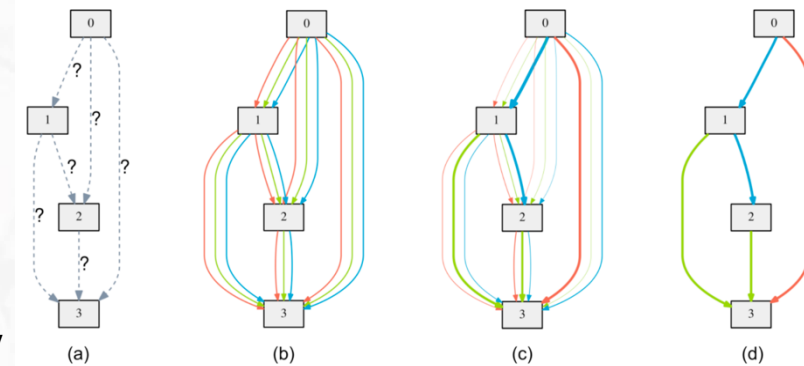
- Hyper-Parameter
- Chain Structure
- Cell Structure
- Randomly Wired

Searching Strategy

- Reinforcement Learning – Policy Gradient
- Evolutionary Algorithm
- Differentiable Architecture Searching

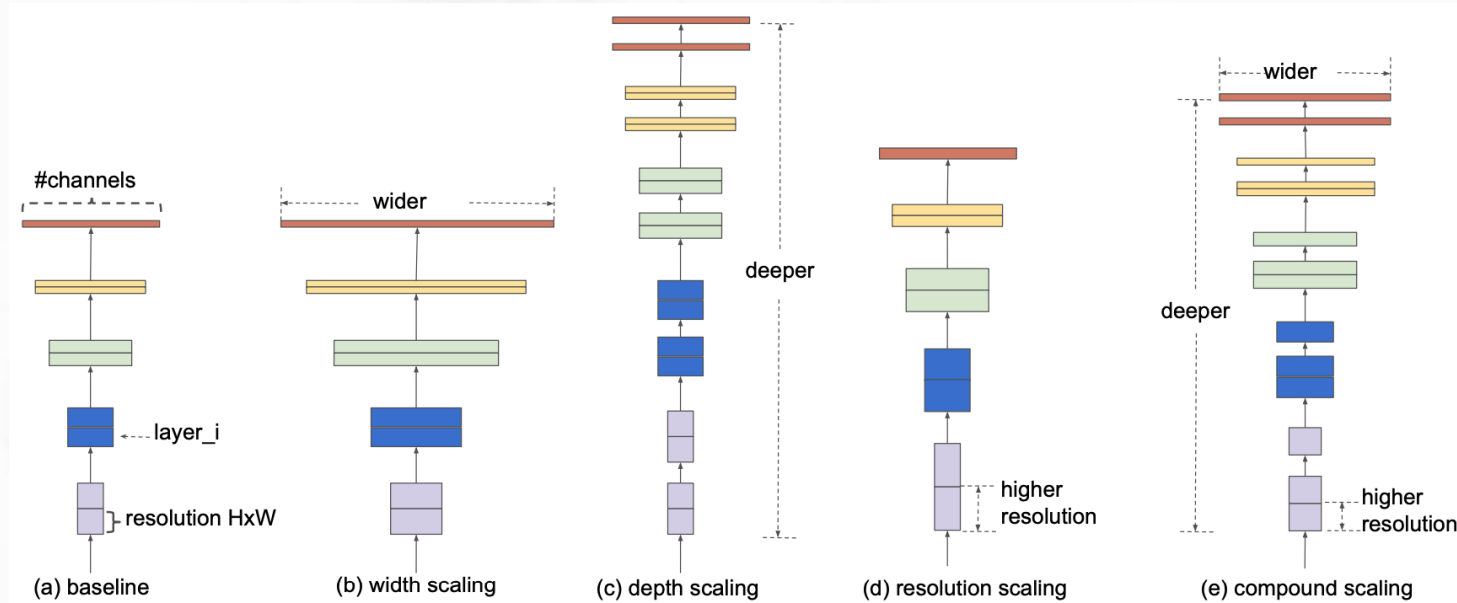
Searching Objective

- Bottom-Up Weight Sharing
- Transferring from a Small Dataset to a Large Dataset
- Top-Down One-Shot
- Platform-Aware (MnasNet)



1. Zoph, Barret, and Quoc V. Le. "Neural architecture search with reinforcement learning." arXiv preprint arXiv:1611.01578 (2016).
2. Zoph, Barret, et al. "Learning transferable architectures for scalable image recognition." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.
3. Pham, Hieu, et al. "Efficient neural architecture search via parameters sharing." International conference on machine learning. PMLR, 2018.
4. Tan, Mingxing, et al. "Mnasnet: Platform-aware neural architecture search for mobile." Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019.
5. Liu, Hanxiao, Karen Simonyan, and Yiming Yang. "Darts: Differentiable architecture search." arXiv preprint arXiv:1806.09055 (2018).

EfficientNet: Compound Scaling



$$\max_{d,w,r} \text{Accuracy}(\mathcal{N}(d,w,r))$$

$$s.t. \quad \mathcal{N}(d,w,r) = \bigodot_{i=1 \dots s} \hat{\mathcal{F}}_i^{d \cdot \hat{L}_i}(X_{(r \cdot \hat{H}_i, r \cdot \hat{W}_i, w \cdot \hat{C}_i)})$$

$$\text{Memory}(\mathcal{N}) \leq \text{target_memory}$$

$$\text{FLOPS}(\mathcal{N}) \leq \text{target_flops}$$

(2)

$$\text{depth: } d = \alpha^\phi$$

$$\text{width: } w = \beta^\phi$$

$$\text{resolution: } r = \gamma^\phi$$

$$s.t. \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

where w, d, r are coefficients for scaling network width, depth, and resolution; $\hat{\mathcal{F}}_i, \hat{L}_i, \hat{H}_i, \hat{W}_i, \hat{C}_i$ are predefined parameters in baseline network (see Table 1 as an example).

- Observation 1 – Scaling up any dimension of network width, depth, or resolution improves accuracy, but the accuracy gain diminishes for bigger models.
- Observation 2 – In order to pursue better accuracy and efficiency, it is critical to balance all dimensions of network width, depth, and resolution during ConvNet scaling.
- Therefore, a new compound scaling method is proposed, which uses a compound coefficient ϕ to uniformly scale network width, depth, and resolution in a principled way.

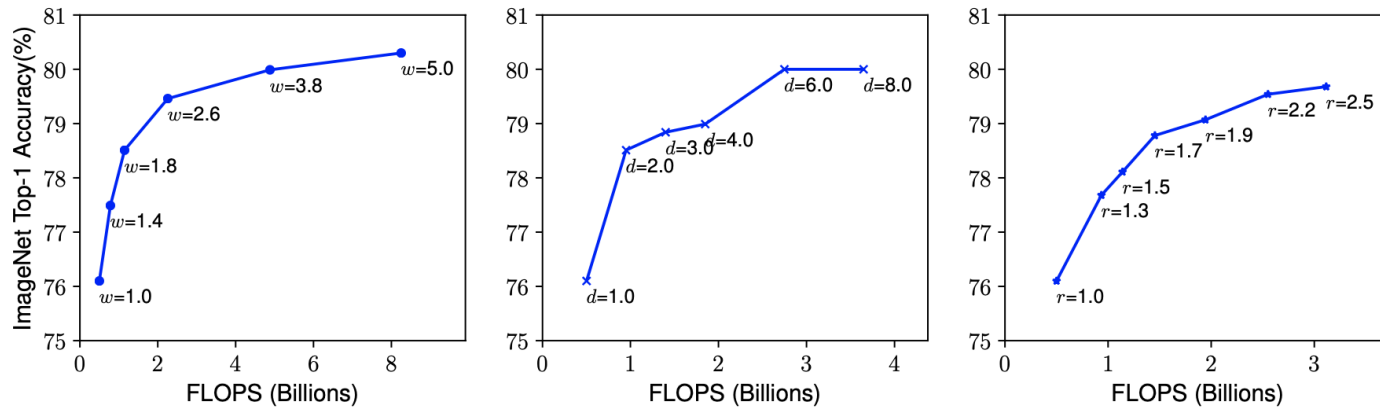


Figure 3. Scaling Up a Baseline Model with Different Network Width (w), Depth (d), and Resolution (r) Coefficients. Bigger networks with larger width, depth, or resolution tend to achieve higher accuracy, but the accuracy gain quickly saturate after reaching 80%, demonstrating the limitation of single dimension scaling. Baseline network is described in Table 1.

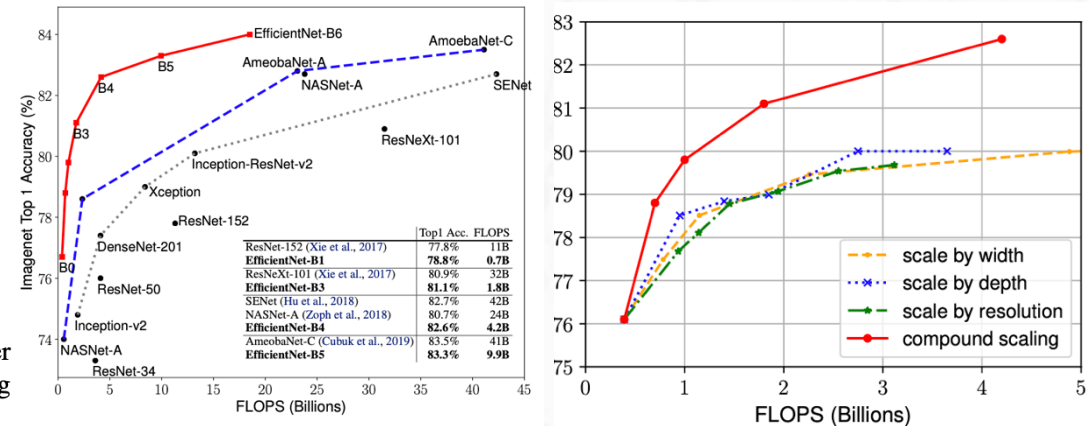
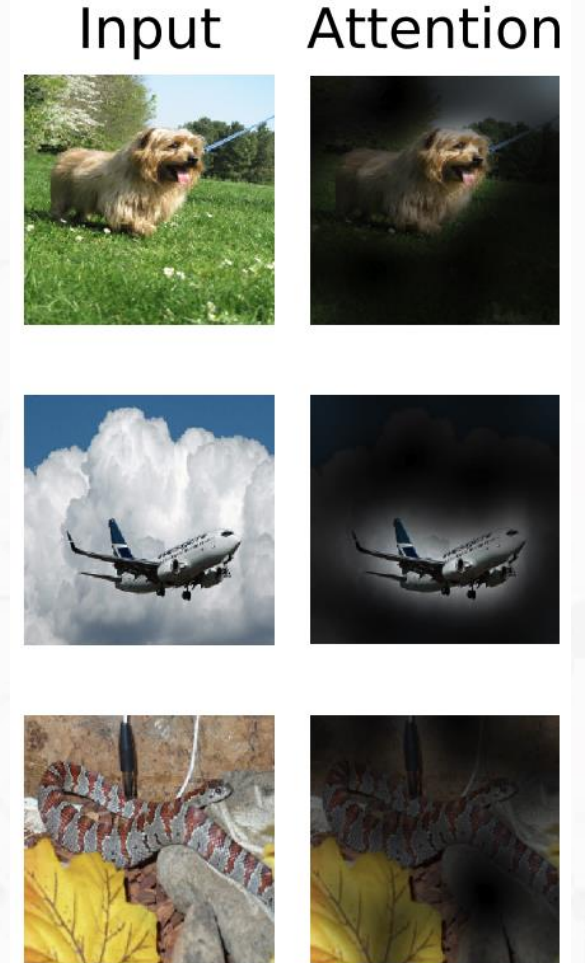
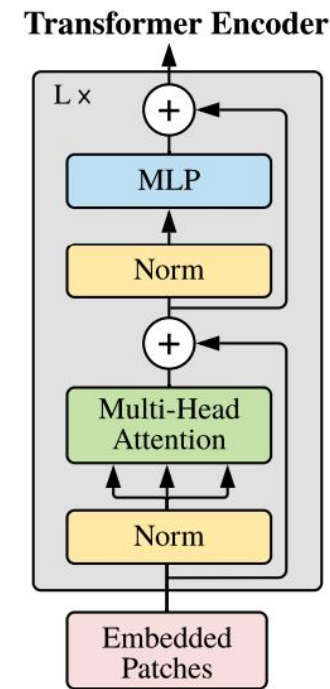
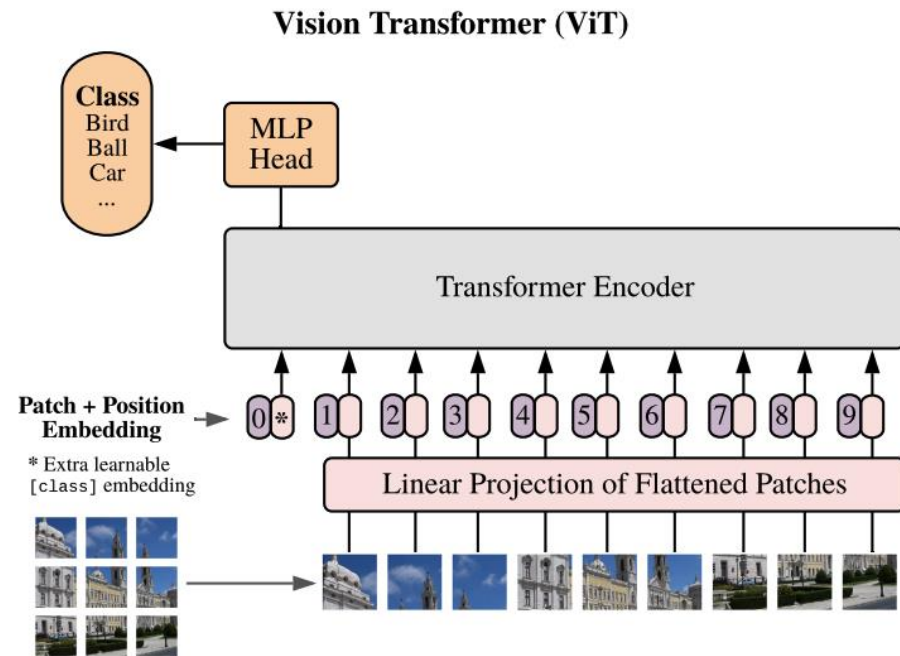
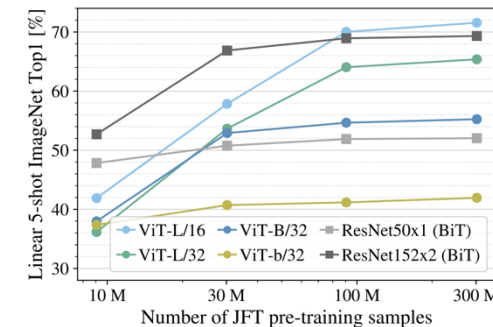
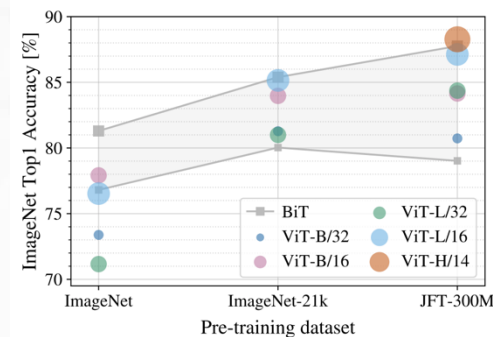


Figure 5. FLOPS vs. ImageNet Accuracy.

ViT: A Picture is Worth Thousands 16 x 16 Words

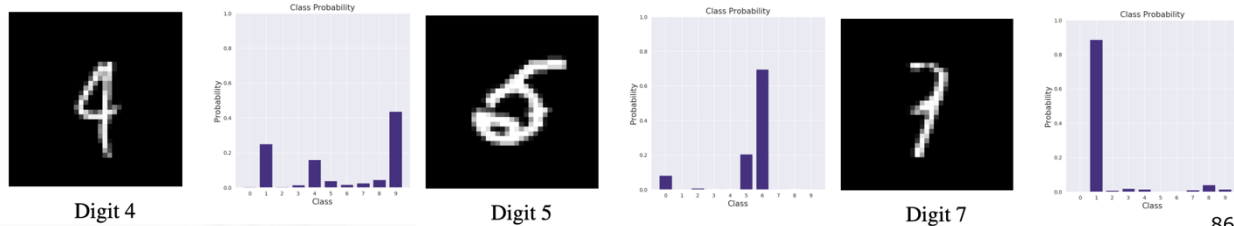
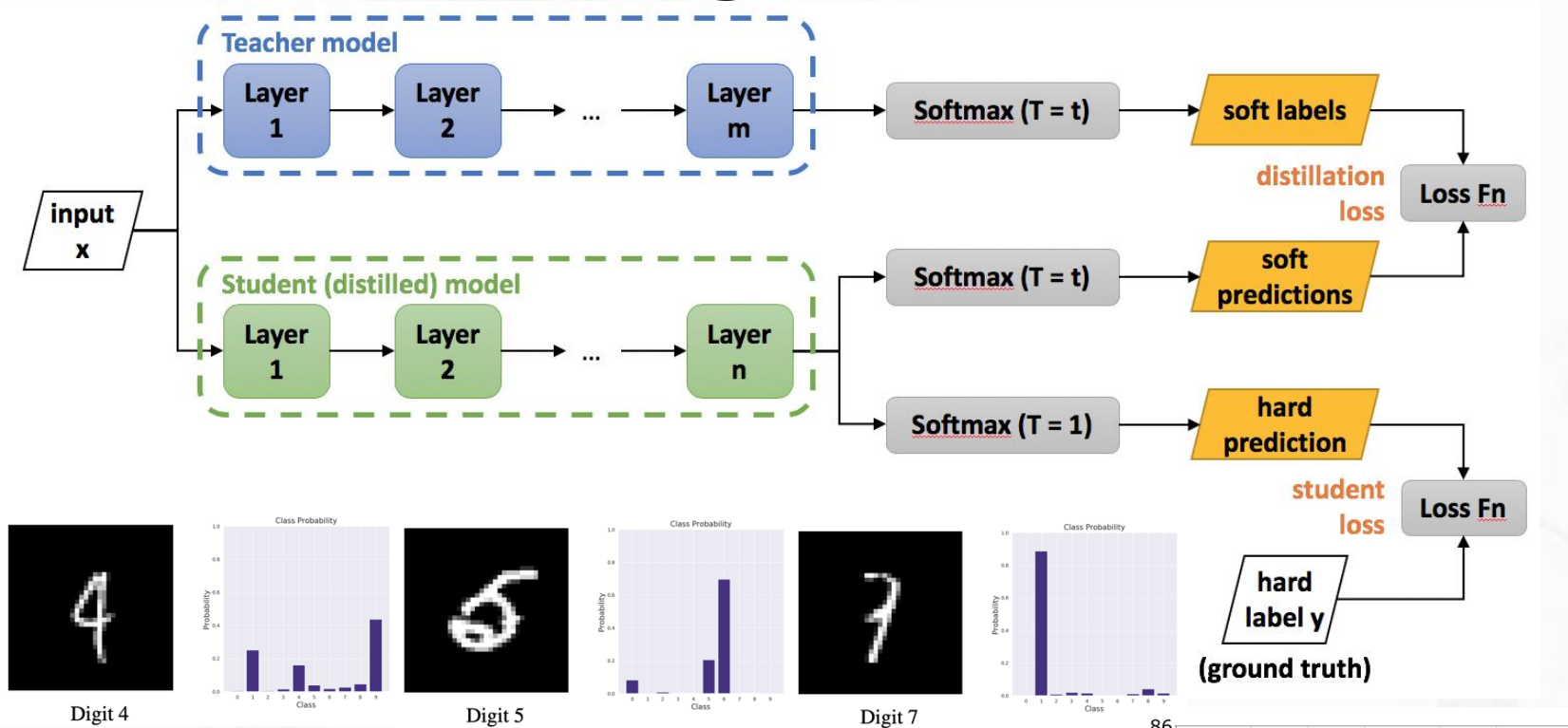


- Split an image into fixed-size patches, i.e., 16 x 16.
- Linearly embed each of them.
- Add position embeddings and an extra “class” token.
- Feed the resulting sequence of vectors to a standard Transformer encoder.
- The output of the “class” token is the image classification prediction.

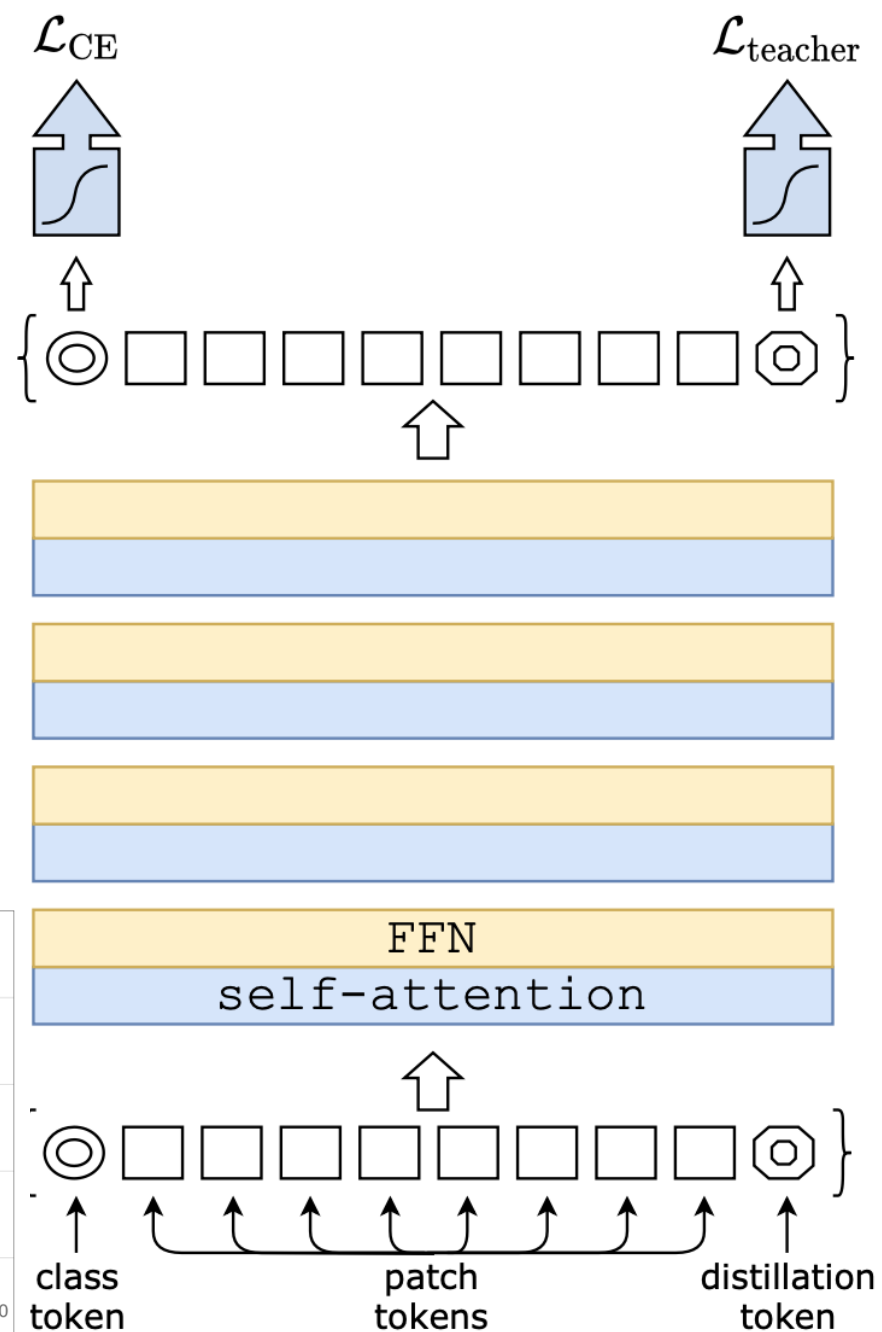
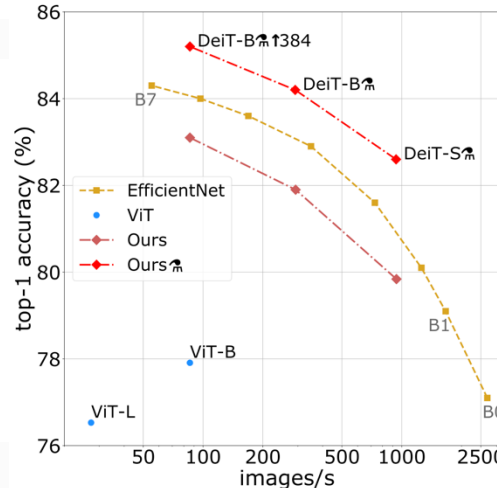


- JFT-300M Dataset is an **internal** Google dataset which is not available for public access.

DeiT: Knowledge Distillation



method ↓	Supervision		ImageNet top-1 (%)			
	label	teacher	Ti 224	S 224	B 224	B [†] 384
DeiT– no distillation	✓	✗	72.2	79.8	81.8	83.1
DeiT– usual distillation	✗	soft	72.2	79.8	81.8	83.2
DeiT– hard distillation	✗	hard	74.3	80.9	83.0	84.0
DeiT [‡] : class embedding	✓	hard	73.9	80.9	83.0	84.2
DeiT [‡] : distil. embedding	✓	hard	74.6	81.1	83.1	84.4
DeiT [‡] : class+distillation	✓	hard	74.5	81.2	83.4	84.5



1. https://intellabs.github.io/distiller/knowledge_distillation.html

2. Park, Chanwoo, et al. "Sampling-based bayesian inference with gradient uncertainty." arXiv preprint arXiv:1812.03285 (2018).

3. Touvron, Hugo, et al. "Training data-efficient image transformers & distillation through attention." International conference on machine learning. PMLR, 2021.

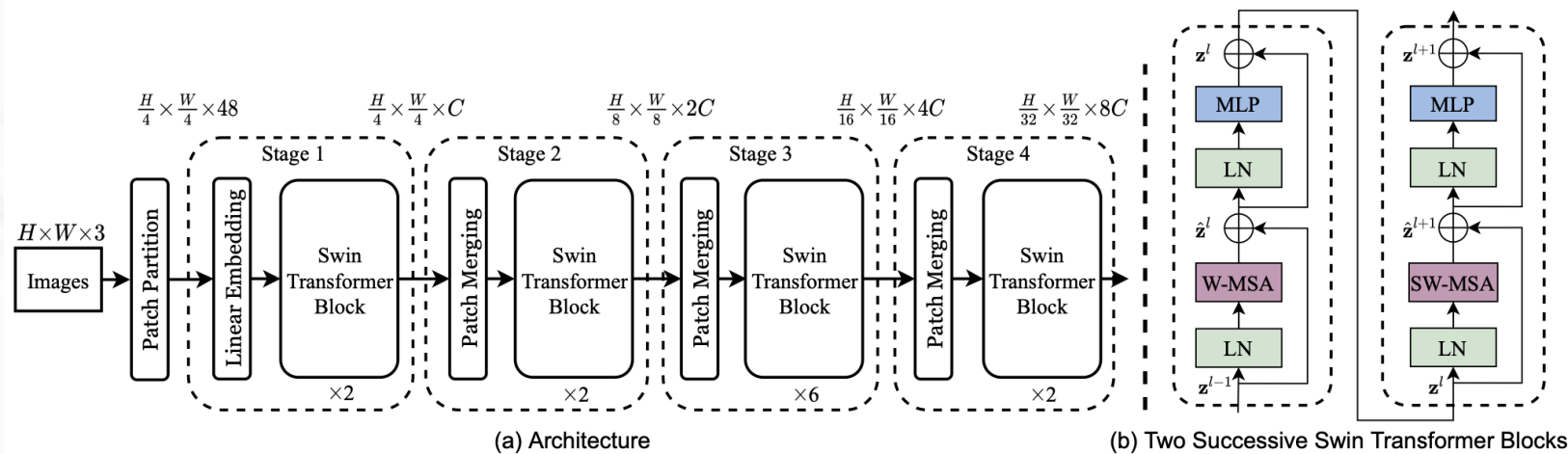
Layer l

Layer l+1

A local window to perform self-attention

A patch

-
- The diagram illustrates the proposed window partitioning method in three stages:
- window partition**: A 4x4 grid is partitioned into four 2x2 windows.
 - cyclic shift**: The windows are shifted cyclically. For example, the top-left window (A) moves to the top-right position (C), the top-right (C) moves to the bottom-right (B), the bottom-right (B) moves to the bottom-left (C), and the bottom-left (C) moves to the top-left (A).
 - reverse cyclic shift**: The shifted windows are shifted back to their original positions, restoring the original grid structure.



22

ConViT : Convolution + Transformer

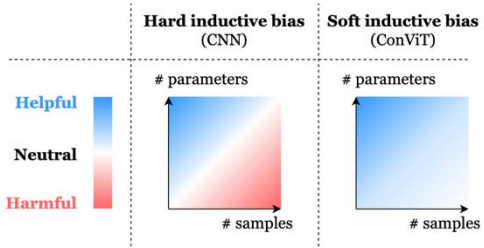
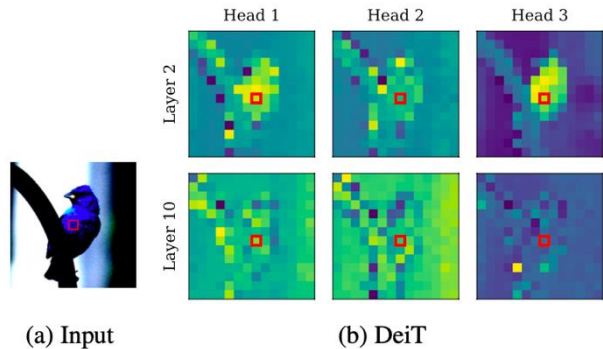
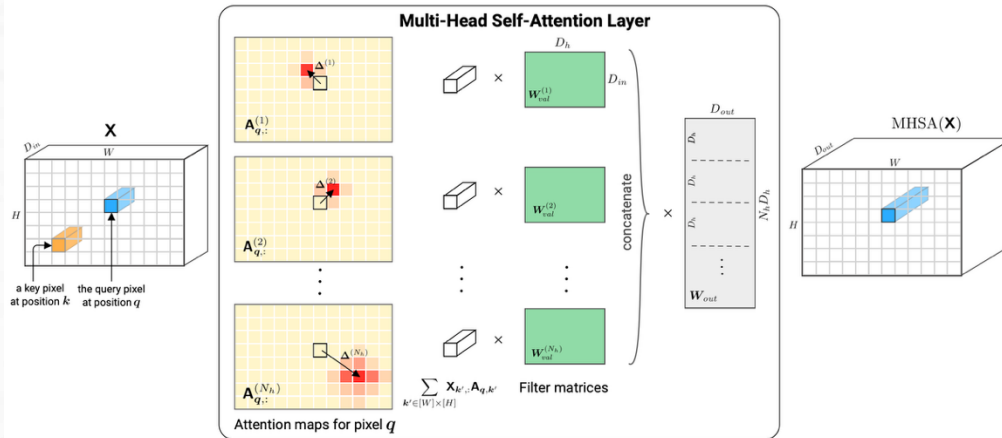
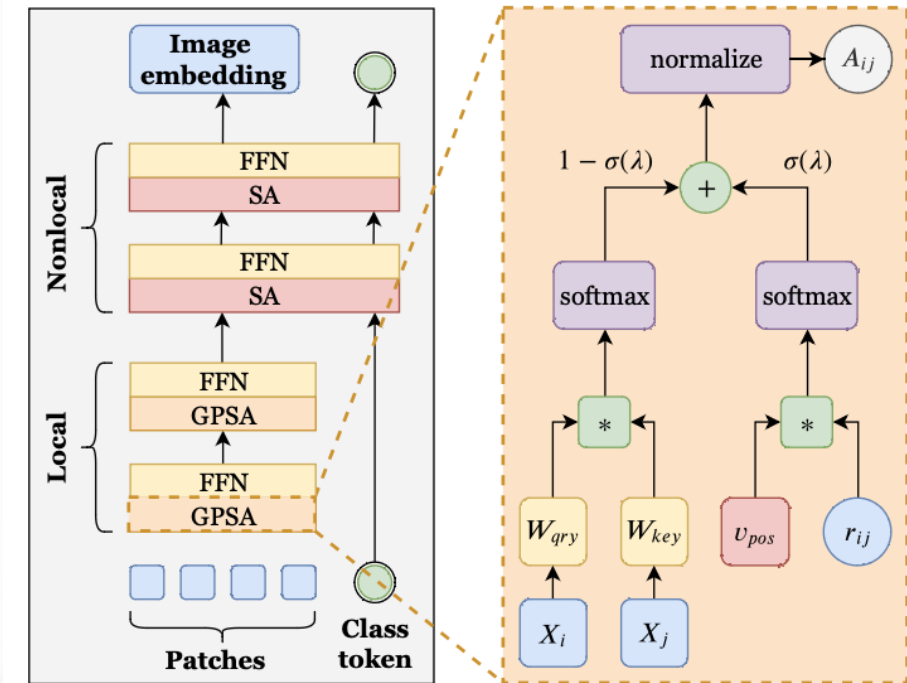
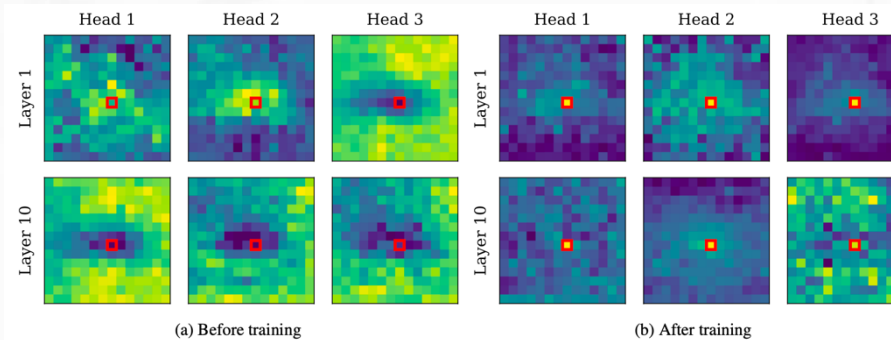
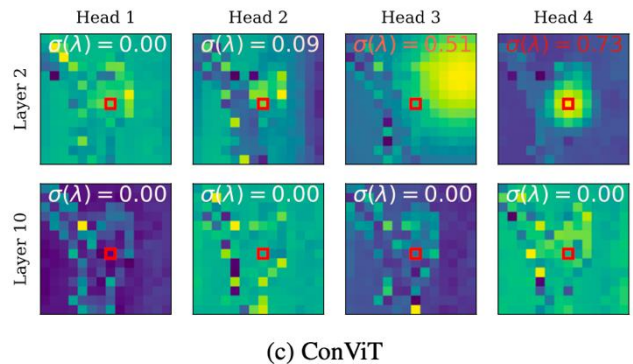


Figure 1. Soft inductive biases can help models learn without being restrictive. Hard inductive biases, such as the architectural constraints of CNNs, can greatly improve the sample-efficiency of learning, but can become constraining when the size of the dataset is not an issue. The soft inductive biases introduced by the ConViT avoid this limitation by vanishing away when not required.



- Instead of interleaving convolutional layers with SA layers as done in hybrid models, let the layers decide whether to be convolutional or not by adjusting a set of gating parameters.
- Combine the biases of varied architectures and let the model choose which ones are best for a given task could become a promising direction
- Reduce the need for greedy architectural search while offering higher interpretability.



ConViT

Gated Positional Self-Attention

Figure 4. **Architecture of the ConViT.** The ConViT (left) is a version of the ViT in which some of the self-attention (SA) layers are replaced with gated positional self-attention layers (GPSA; right). Because GPSA layers involve positional information, the class token is concatenated with hidden representation after the last GPSA layer. In this paper, we typically take 10 GPSA layers followed by 2 vanilla SA layers. FFN: feedforward network (2 linear layers separated by a GeLU activation); W_{qry} : query weights; W_{key} : key weights; v_{pos} : attention center and span embeddings (learned); r_{qk} : relative position encodings (fixed); λ : gating parameter (learned); σ : sigmoid function.

ConvNeXts : Modernise Convolution

(pre-)training config	ConvNeXt-T/S/B/L ImageNet-1K 224 ²	ConvNeXt-T/S/B/L/XL ImageNet-22K 224 ²
weight init	trunc. normal (0.2)	trunc. normal (0.2)
optimizer	AdamW	AdamW
base learning rate	4e-3	4e-3
weight decay	0.05	0.05
optimizer momentum	$\beta_1, \beta_2=0.9, 0.999$	$\beta_1, \beta_2=0.9, 0.999$
batch size	4096	4096
training epochs	300	90
learning rate schedule	cosine decay	cosine decay
warmup epochs	20	5
warmup schedule	linear	linear
layer-wise lr decay [6, 12]	None	None
randaugment [14]	(9, 0.5)	(9, 0.5)
mixup [90]	0.8	0.8
cutmix [89]	1.0	1.0
random erasing [91]	0.25	0.25
label smoothing [69]	0.1	0.1
stochastic depth [37]	0.1/0.4/0.5/0.5	0.0/0.0/0.1/0.1/0.2
layer scale [74]	1e-6	1e-6
head init scale [74]	None	None
gradient clip	None	None
exp. mov. avg. (EMA) [51]	0.9999	None

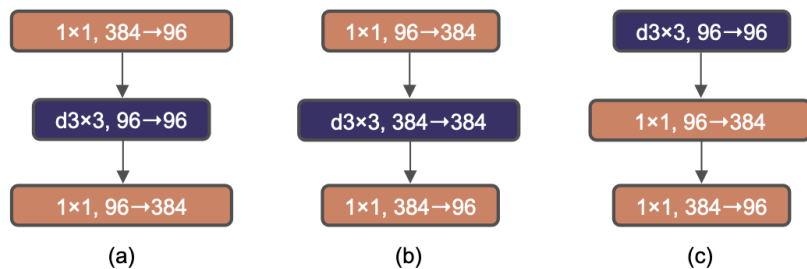
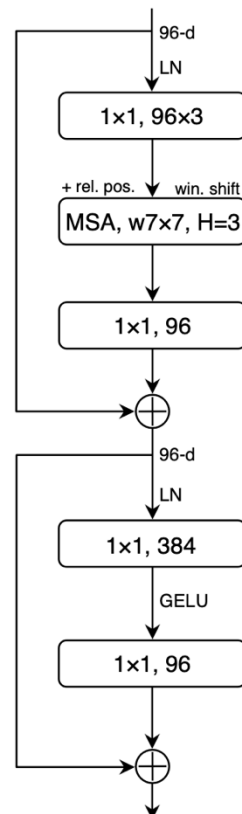
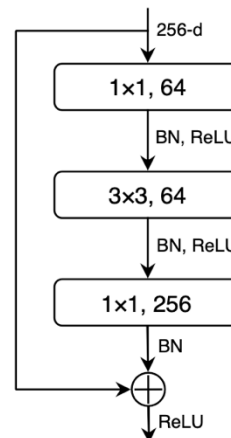


Figure 3. **Block modifications and resulted specifications.** (a) is a ResNeXt block; in (b) we create an inverted bottleneck block and in (c) the position of the spatial depthwise conv layer is moved up.

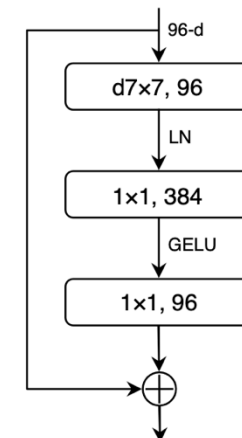
Swin Transformer Block



ResNet Block



ConvNeXt Block



- Comprehensive Training Parameters
- Tune the Tradition Principles
 - Stage Ratio, Patchify Stem, ResNeXtify (Separable Convolution), GELU, Fewer Activation & Normalisation.
- Challenge the Tradition Principles
 - Small Kernel (VGG) v.s. Large Kernel
 - Bottleneck (ResNet) v.s. Inverted Bottleneck
 - Batch Normalisation v.s. Layer Normalisation

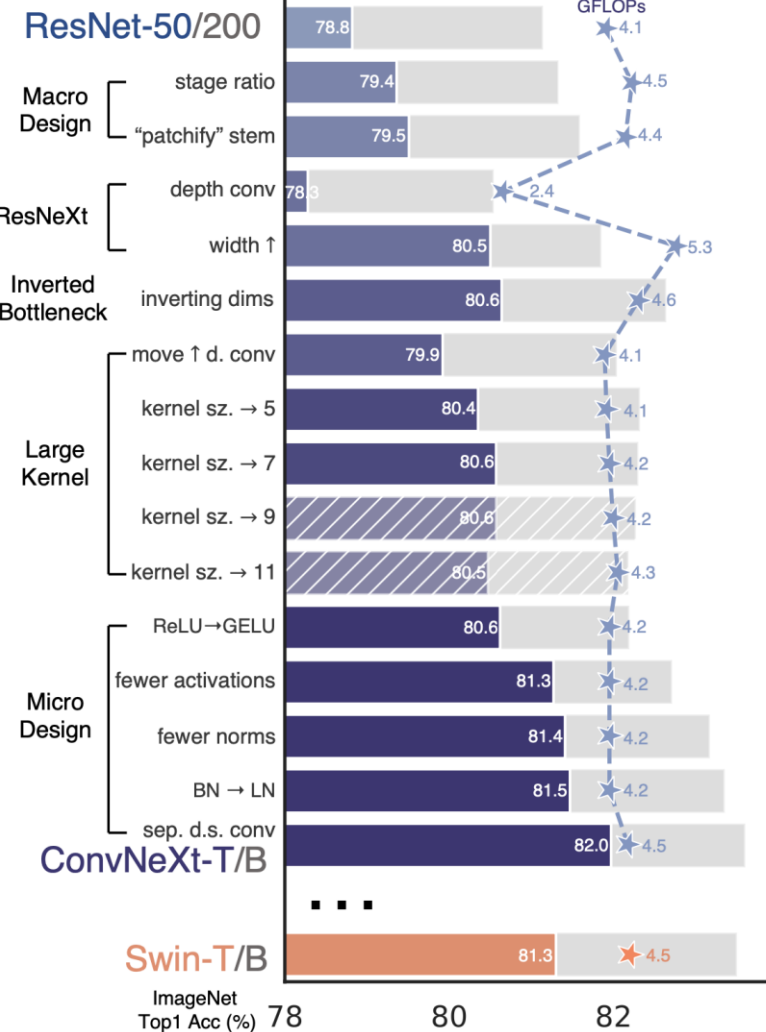
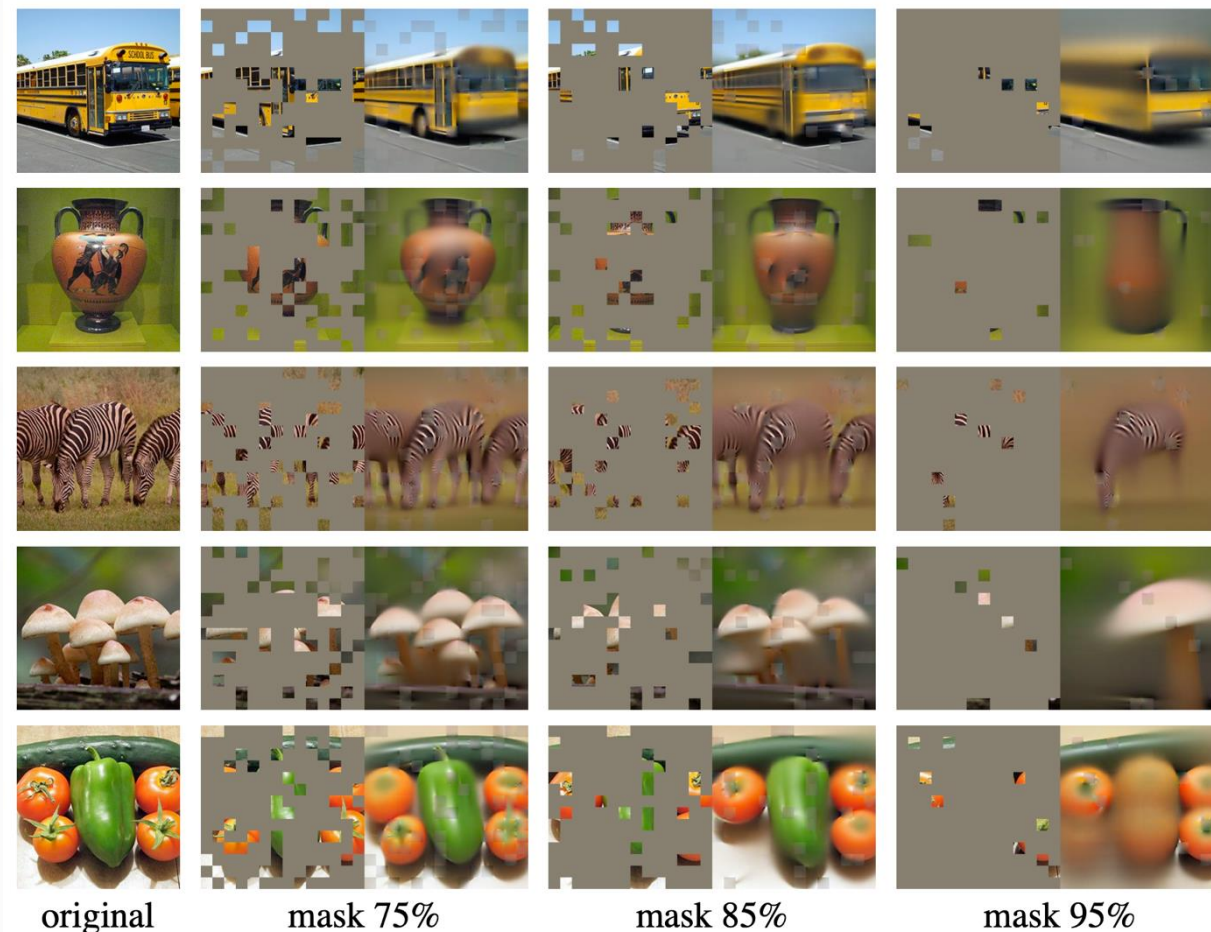
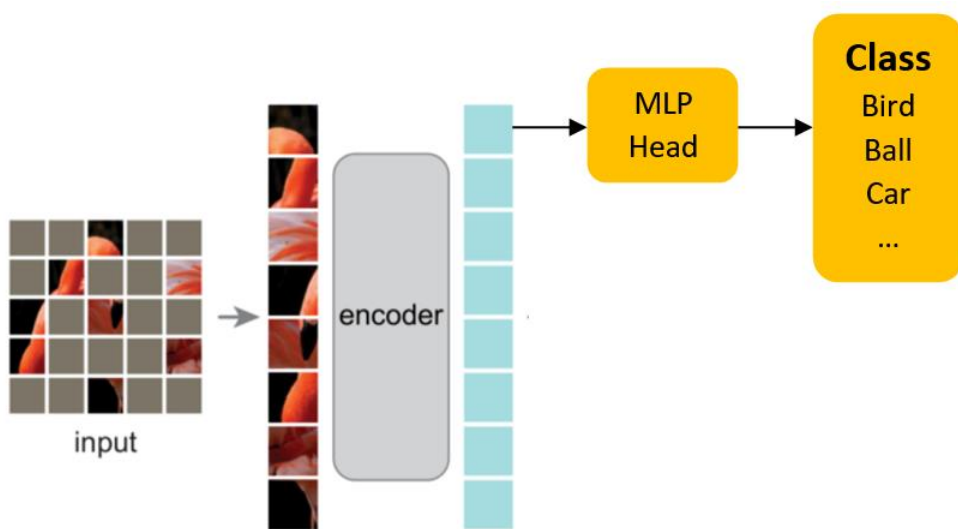
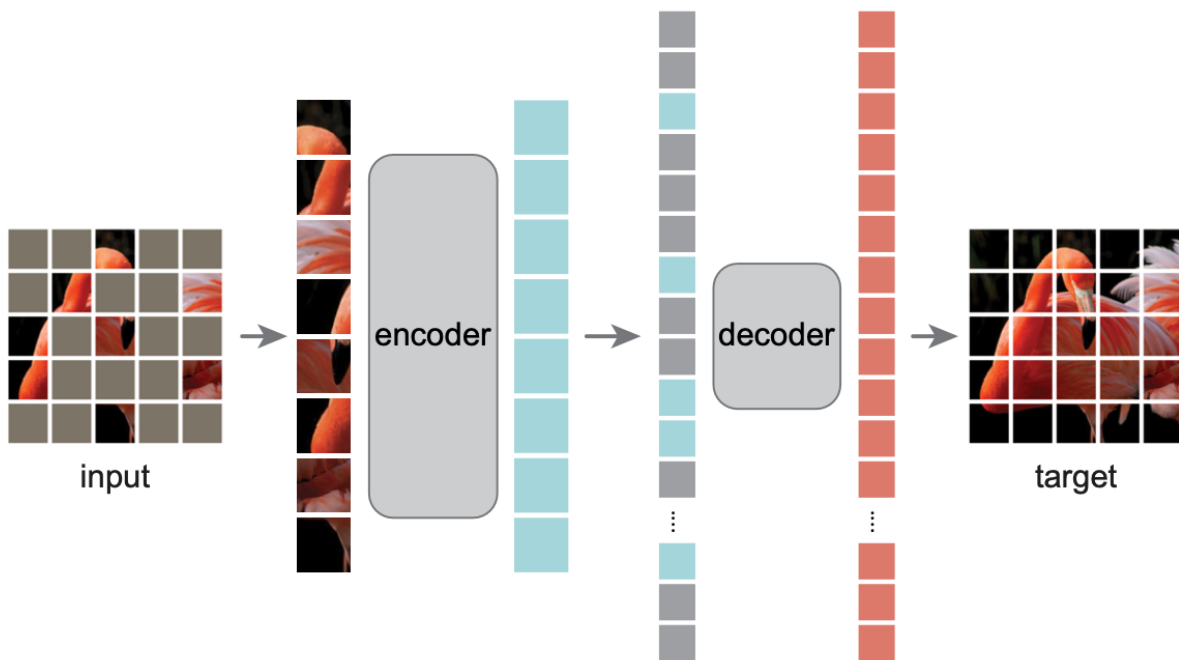
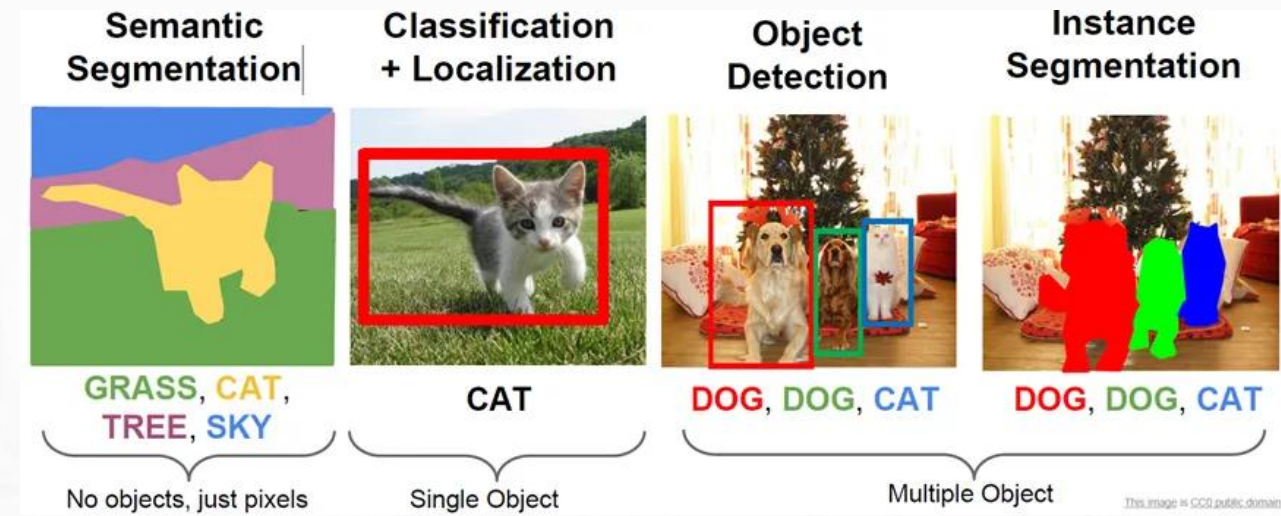


Figure 2. We modernize a standard ConvNet (ResNet) towards the design of a hierarchical vision Transformer (Swin), without introducing any attention-based modules. The foreground bars are model accuracies in the ResNet-50/Swin-T FLOP regime; results for the ResNet-200/Swin-B regime are shown with the gray bars. A hatched bar means the modification is not adopted. Detailed results for both regimes are in the appendix. Many Transformer architectural choices can be incorporated in a ConvNet, and they lead to increasingly better performance. In the end, our pure ConvNet model, named ConvNeXt, can outperform the Swin Transformer.

MAE: Towards Vision Foundation Model



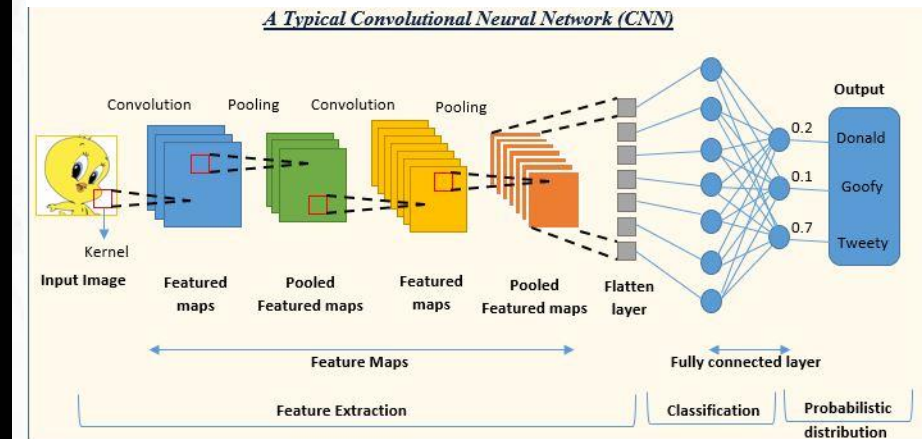
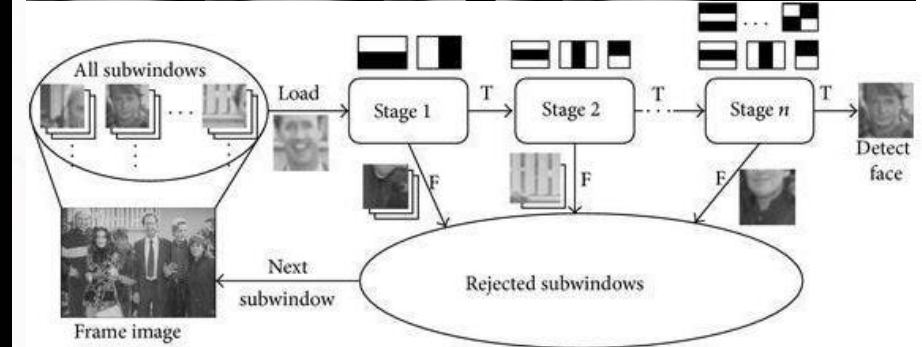
- What makes masked autoencoding different between vision and language?
 - Architecture Difference, Information Density, Semantic Level
- Scalable self-supervised learners for computer vision.
- Mask random patches of the input image and reconstruct the missing pixels.



Discriminative Models

Object Detection & Segmentation

Face Detection: Classic Approach v.s. Deep Learning



1. https://www.youtube.com/watch?v=Zedlhu_QQIE
2. <https://www.analyticsvidhya.com/blog/2022/01/convolutional-neural-network-an-overview/>
3. <https://medium.com/@baselanaya/faces-detection-using-haar-cascade-3e175aef84f5>

“Shallow” Object Detection – HoG + SVM

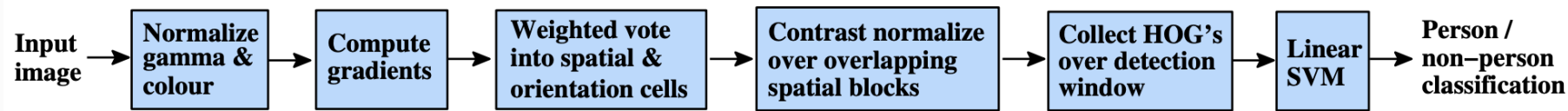


Figure 1. An overview of our feature extraction and object detection chain. The detector window is tiled with a grid of overlapping blocks in which Histogram of Oriented Gradient feature vectors are extracted. The combined vectors are fed to a linear SVM for object/non-object classification. The detection window is scanned across the image at all positions and scales, and conventional non-maximum suppression is run on the output pyramid to detect object instances, but this paper concentrates on the feature extraction process.

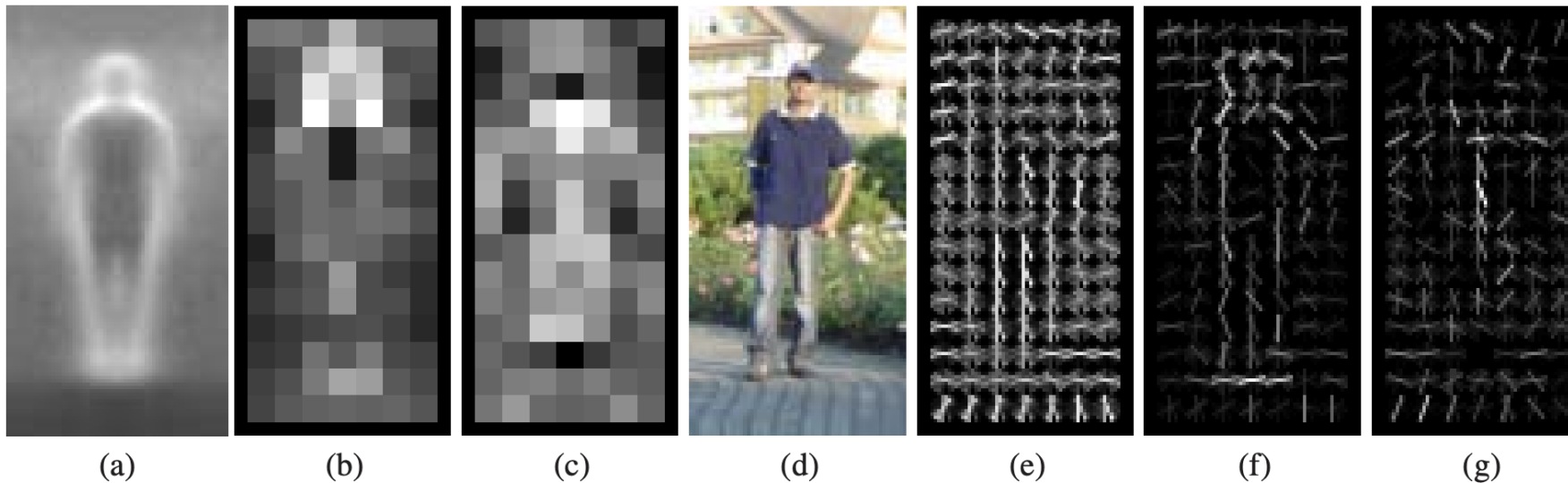
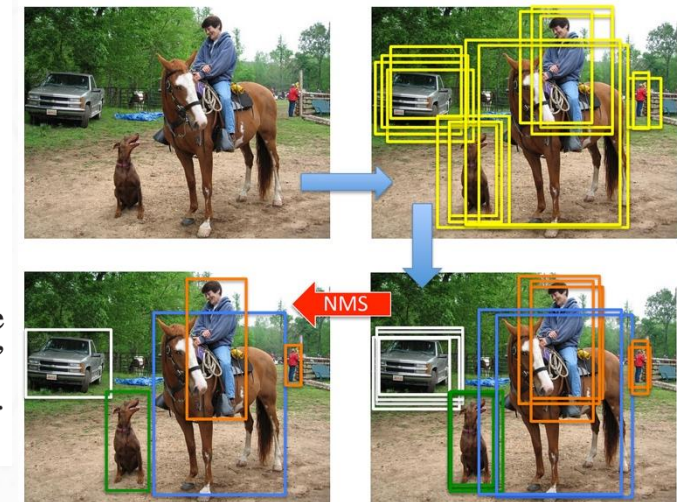


Figure 6. Our HOG detectors cue mainly on silhouette contours (especially the head, shoulders and feet). The most active blocks are centred on the image background just *outside* the contour. (a) The average gradient image over the training examples. (b) Each “pixel” shows the maximum positive SVM weight in the block centred on the pixel. (c) Likewise for the negative SVM weights. (d) A test image. (e) It’s computed R-HOG descriptor. (f,g) The R-HOG descriptor weighted by respectively the positive and the negative SVM weights.

- Manually Designed Feature
- Sliding Window or Salient Region Proposal
- Training with Positive & Negative Samples
- Test “ALL” Location & Scale
- “Shallow” Classifier or Cascade “Shallow” Classifiers
- Non-Maximum Suppression Removes Overlapping Boxes



Fast R-CNN: Patching Up R-CNN & SPPNet

$$L(p, u, t^u, v) = L_{\text{cls}}(p, u) + \lambda[u \geq 1]L_{\text{loc}}(t^u, v)$$

$$L_{\text{cls}}(p, u) = -\log p_u$$

$$L_{\text{loc}}(t^u, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^u - v_i)$$

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

	S				M				L			
multi-task training?	✓			✓	✓			✓	✓			✓
stage-wise training?		✓		✓		✓		✓		✓		✓
test-time bbox reg?			✓	✓			✓	✓			✓	✓
VOC07 mAP	52.2	53.3	54.6	57.1	54.7	55.5	56.6	59.2	62.6	63.4	64.0	66.9

Table 6. Multi-task training (forth column per group) improves mAP over piecewise training (third column per group).

method	train set	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	persn	plant	sheep	sofa	train	tv	mAP
SPPnet BB [11] [†]	07 \ diff	73.9	72.3	62.5	51.5	44.4	74.4	73.0	74.4	42.3	73.6	57.7	70.3	74.6	74.3	54.2	34.0	56.4	56.4	67.9	73.5	63.1
R-CNN BB [10]	07	73.4	77.0	63.4	45.4	44.6	75.1	78.1	79.8	40.5	73.7	62.2	79.4	78.1	73.1	64.2	35.6	66.8	67.2	70.4	71.1	66.0
FRCN [ours]	07	74.5	78.3	69.2	53.2	36.6	77.3	78.2	82.0	40.7	72.7	67.9	79.6	79.2	73.0	69.0	30.1	65.4	70.2	75.8	65.8	66.9
FRCN [ours]	07 \ diff	74.6	79.0	68.6	57.0	39.3	79.5	78.6	81.9	48.0	74.0	67.4	80.5	80.7	74.1	69.6	31.8	67.1	68.4	75.3	65.5	68.1
FRCN [ours]	07+12	77.0	78.1	69.3	59.4	38.3	81.6	78.6	86.7	42.8	78.8	68.9	84.7	82.0	76.6	69.9	31.8	70.1	74.8	80.4	70.4	70.0

Table 1. VOC 2007 test detection average precision (%). All methods use VGG16. Training set key: **07**: VOC07 trainval, **07 \ diff**: 07 without “difficult” examples, **07+12**: union of 07 and VOC12 trainval. [†]SPPnet results were prepared by the authors of [11].

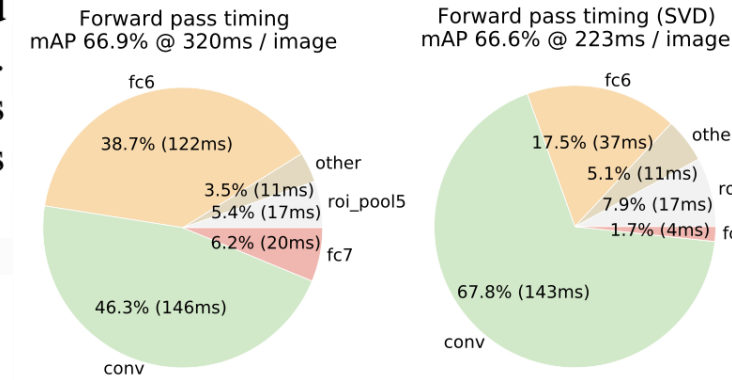


Figure 2. Timing for VGG16 before and after truncated SVD. Before SVD, fully connected layers fc6 and fc7 take 45% of the time.

method	classifier	S	M	L
R-CNN [9, 10]	SVM	58.5	60.2	66.0
FRCN [ours]	SVM	56.3	58.7	66.8
FRCN [ours]	softmax	57.1	59.2	66.9

Table 8. Fast R-CNN with softmax vs. SVM (VOC07 mAP).

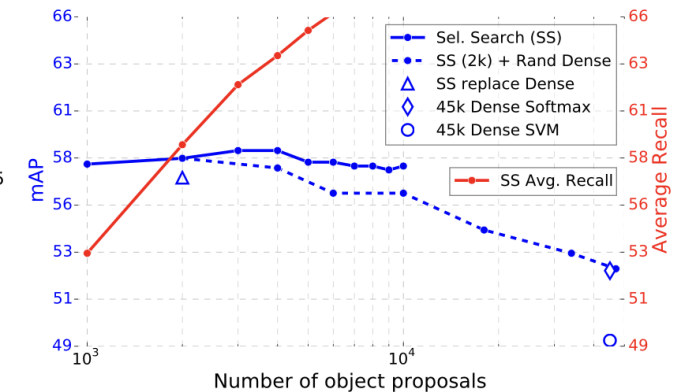


Figure 3. VOC07 test mAP and AR for various proposal schemes.

	SPPnet ZF		S		M		L
scales	1	5	1	5	1	5	1
test rate (s/im)	0.14	0.38	0.10	0.39	0.15	0.64	0.32
VOC07 mAP	58.0	59.2	57.1	58.4	59.2	60.7	66.9

Table 7. Multi-scale vs. single scale. SPPnet ZF (similar to model S) results are from [11]. Larger networks with a single-scale offer the best speed / accuracy tradeoff. (L cannot use multi-scale in our implementation due to GPU memory constraints.)

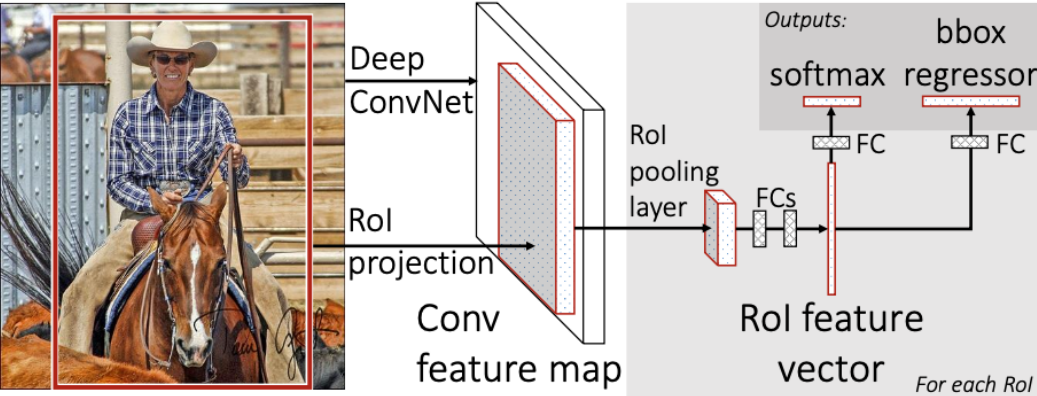
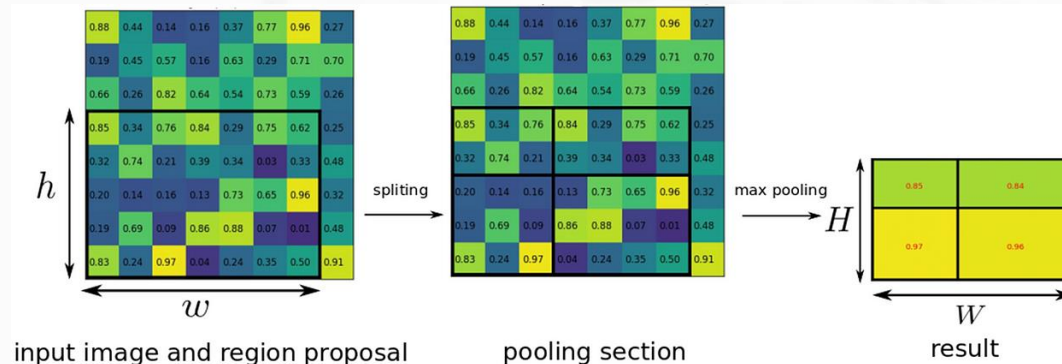


Figure 1. Fast R-CNN architecture. An input image and multiple regions of interest (RoIs) are input into a fully convolutional network. Each RoI is pooled into a fixed-size feature map and then mapped to a feature vector by fully connected layers (FCs). The network has two output vectors per RoI: softmax probabilities and per-class bounding-box regression offsets. The architecture is trained end-to-end with a multi-task loss.



Faster R-CNN : RPN

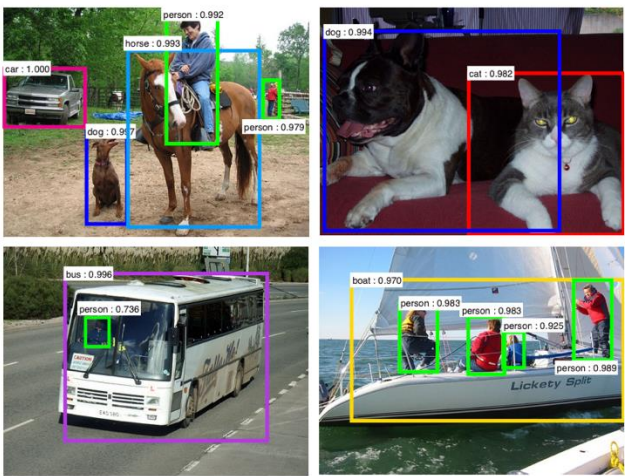
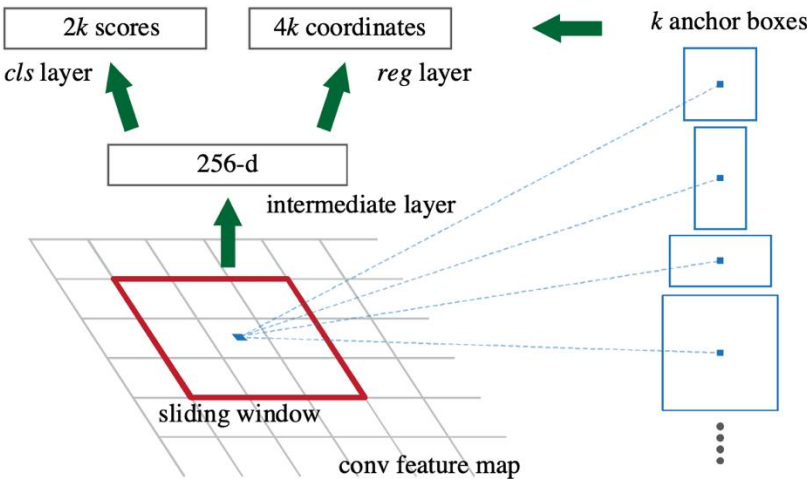
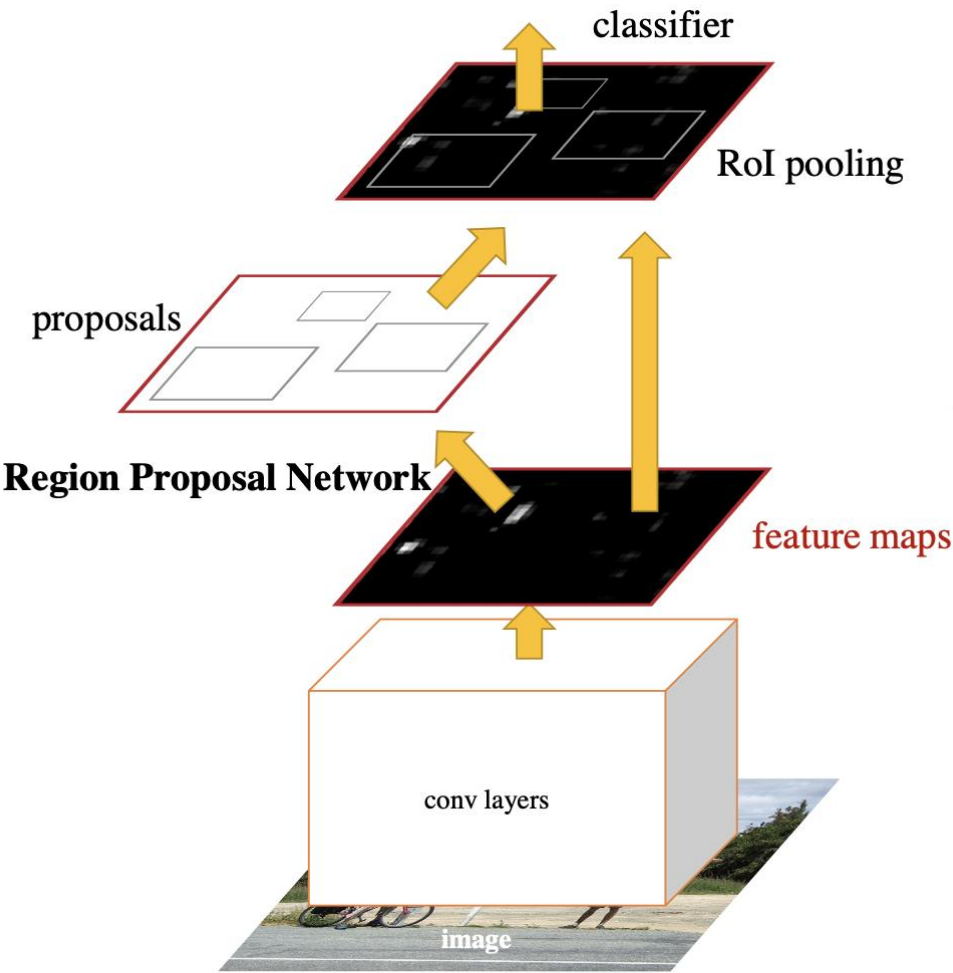


Figure 3: **Left:** Region Proposal Network (RPN). **Right:** Example detections using RPN proposals on PASCAL VOC 2007 test. Our method detects objects in a wide range of scales and aspect ratios.

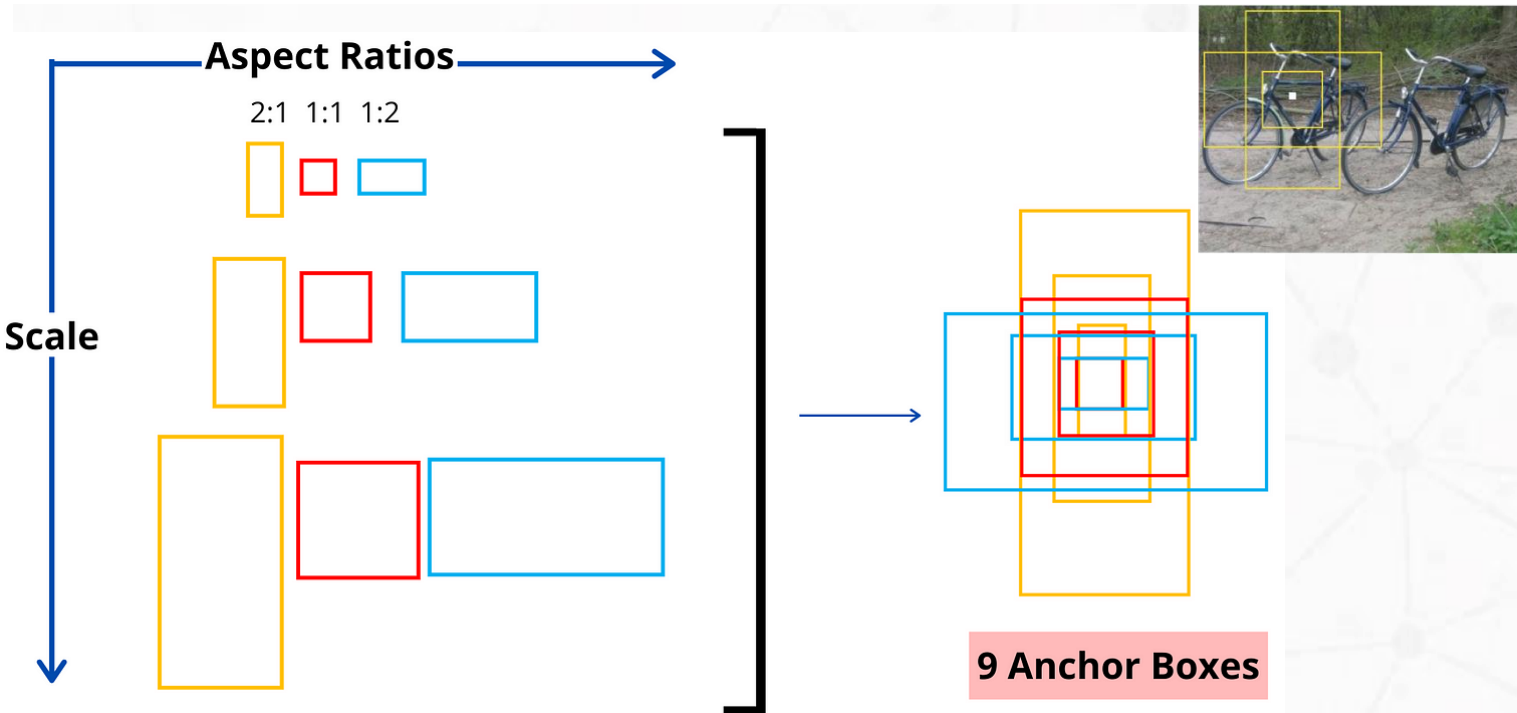


Figure 2: Faster R-CNN is a single, unified network for object detection. The RPN module serves as the 'attention' of this unified network.

Mask R-CNN : Segmentation

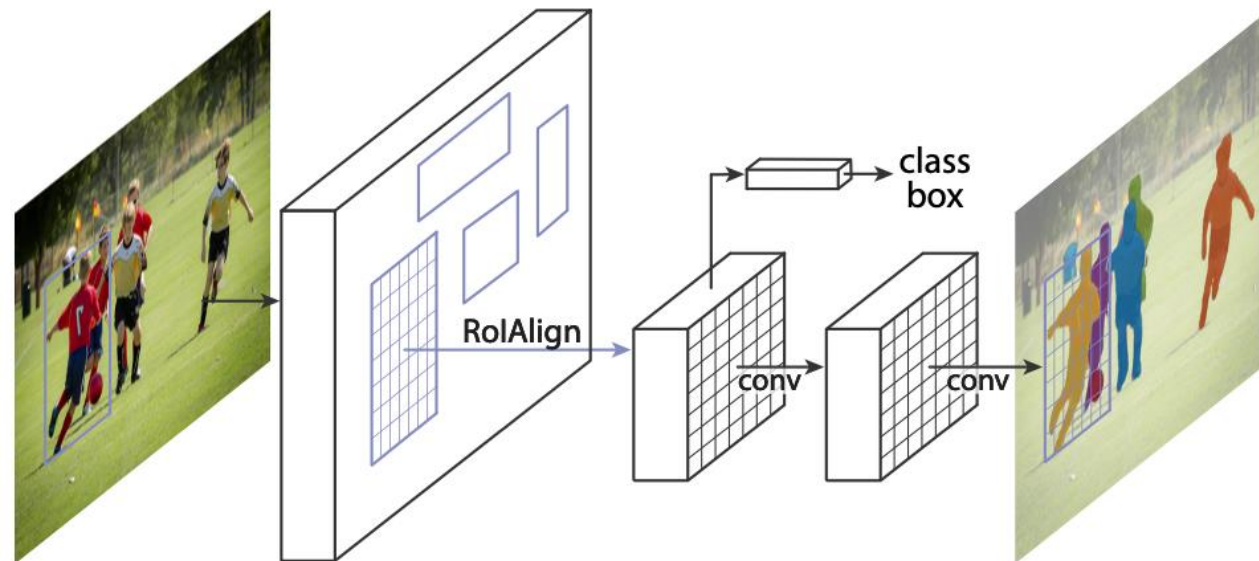


Figure 1. The **Mask R-CNN** framework for instance segmentation.

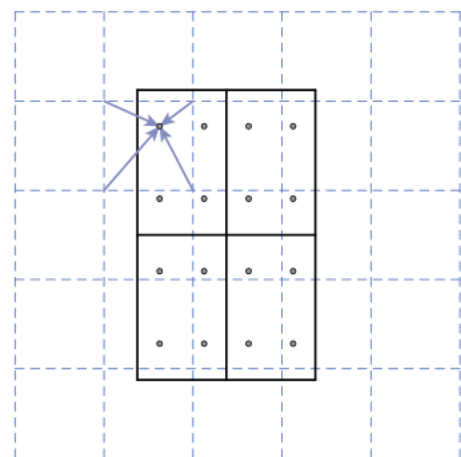
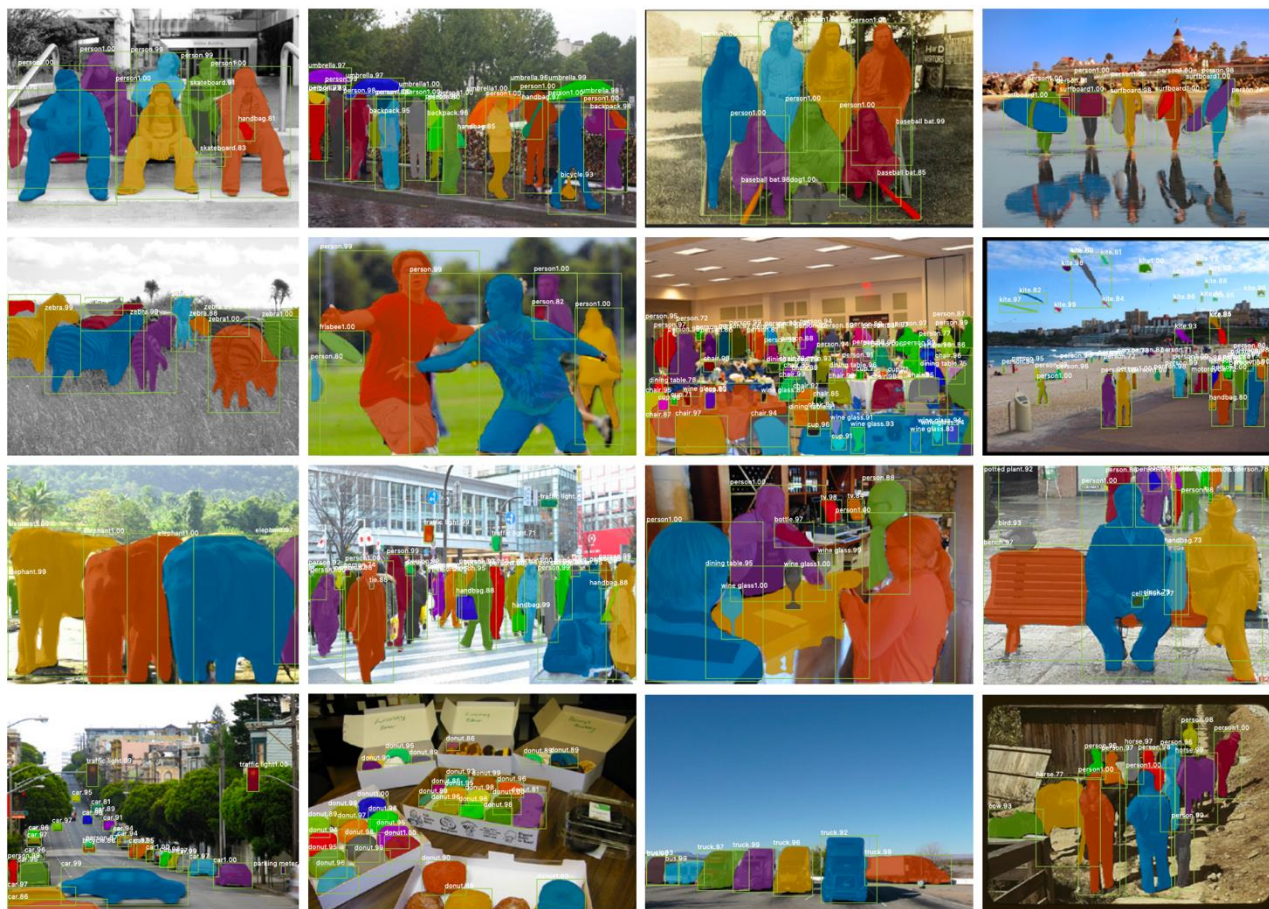
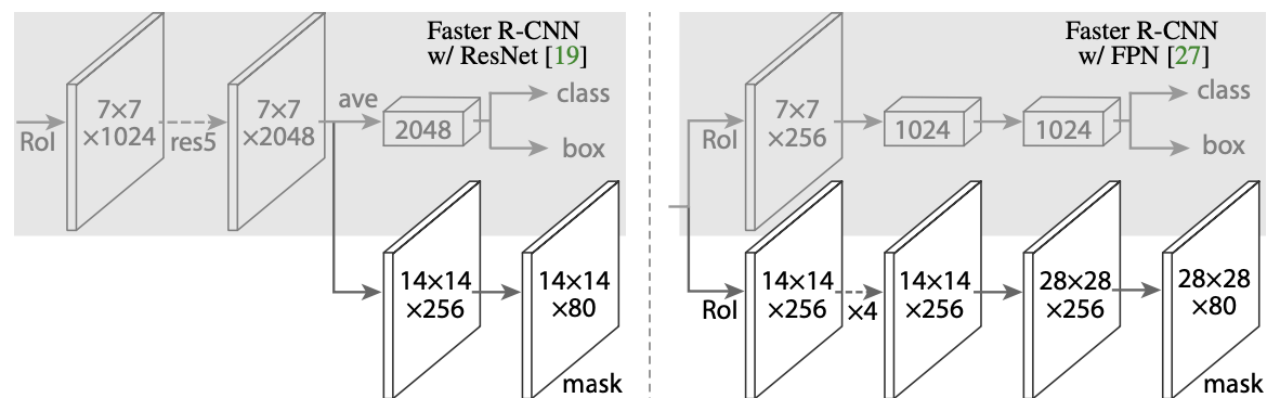


Figure 3. **RoIAlign**: The dashed grid represents a feature map, the solid lines an RoI (with 2×2 bins in this example), and the dots the 4 sampling points in each bin. RoIAlign computes the value of each sampling point by bilinear interpolation from the nearby grid points on the feature map. No quantization is performed on any coordinates involved in the RoI, its bins, or the sampling points.



R-CNN

Linear Regression for bounding box offsets

The diagram illustrates the R-CNN architecture. At the bottom, an 'Input image' of a horse is shown with several 'Regions of Interest (RoI)' outlined in red. These RoIs are extracted as 'Warped image regions'. Three parallel processing branches are shown, each consisting of a 'ConvN et' (Convolutional Neural Network) block. Each branch takes a warped image region as input and produces two outputs: a 'Bbox reg' (bounding box regression) and 'SVMs' (Support Vector Machines) for classification. The text 'Forward each region through ConvNet' is placed to the right of the branches. The text 'Classify regions with SVMs' is placed to the right of the SVM outputs. The text 'Linear Regression for bounding box offsets' is placed at the top right.

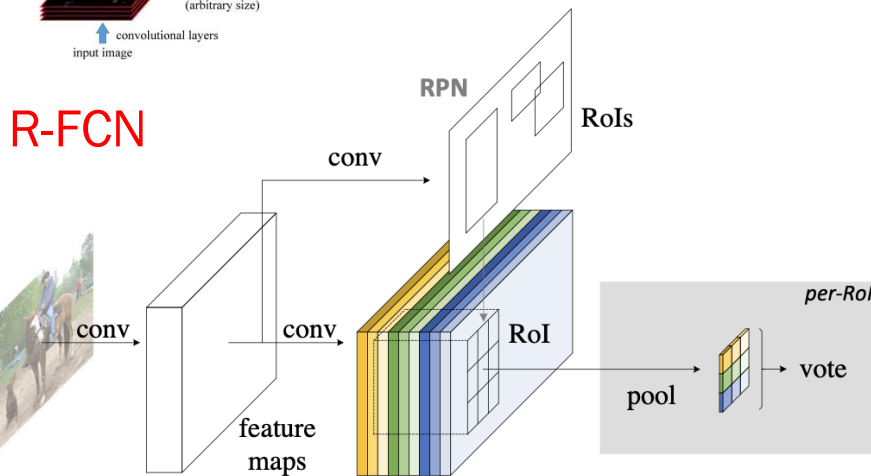
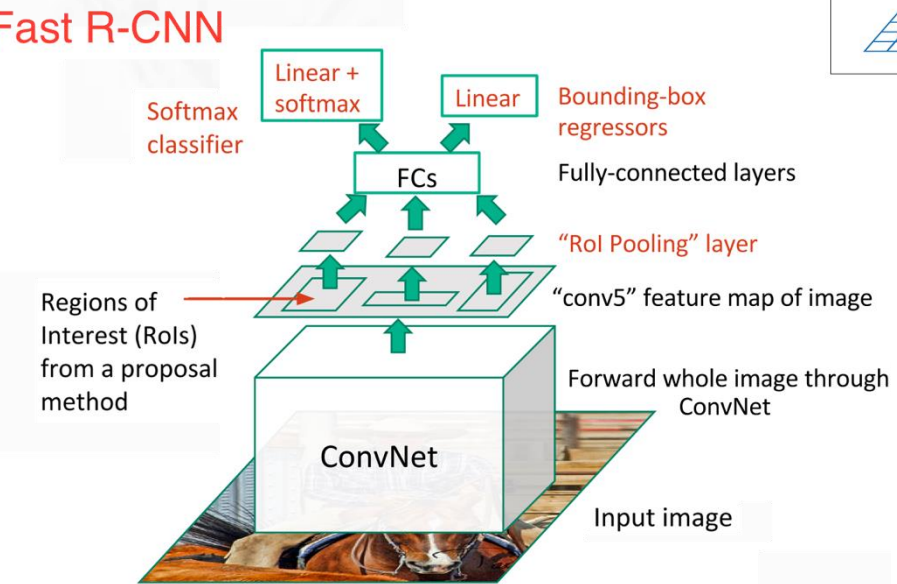
Classify regions with SVMs

Forward each region through ConvNet

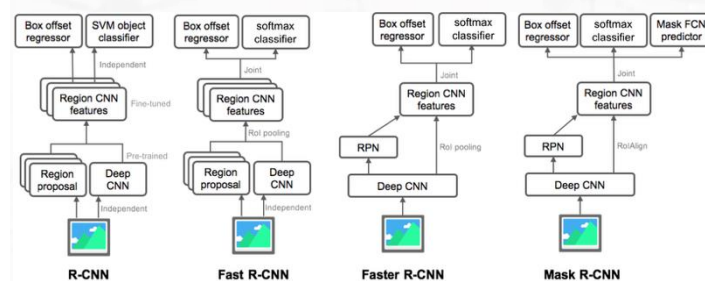
Warped image regions

Regions of Interest (RoI) from a proposal method (~2k)

Input image



- Region Proposal
 - Selective Search
- Fixed-Size Pooling
 - SPPNet
 - RoI Pooling
- End-to-End Training
 - SVM $\rightarrow$ Softmax Classifier
- Multi-Tasks
 - Classification
 - Regression
- Reduce Overheads
 - High Quality Proposal
 - Single Pass Feature Extraction
 - Region Proposal Network
- Position Sensitive Map
- Extend to Segmentation
 - Mask FCN



SSD : Single Shot MultiBox Detector

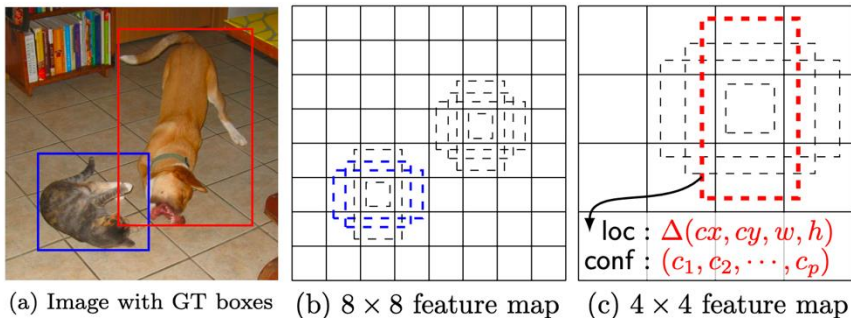
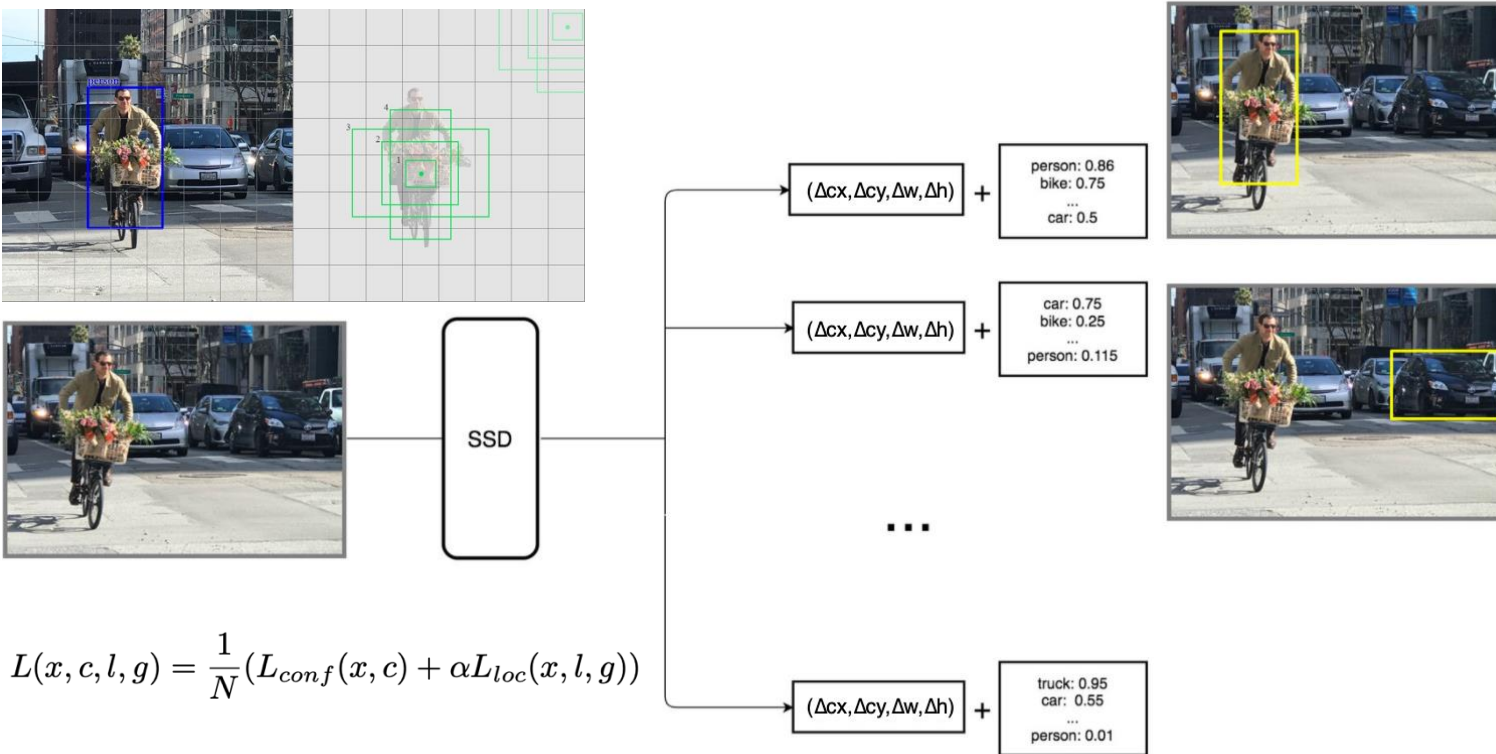
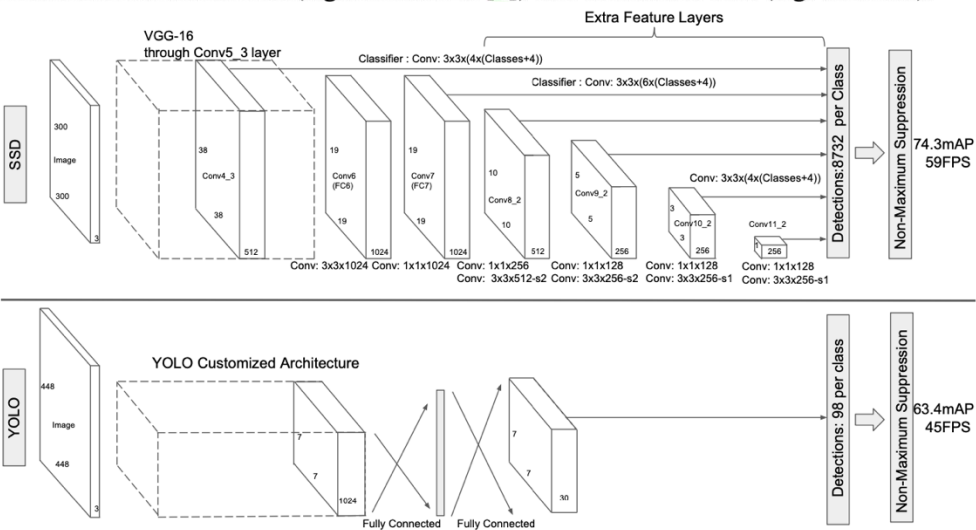


Fig. 1: **SSD framework.** (a) SSD only needs an input image and ground truth boxes for each object during training. In a convolutional fashion, we evaluate a small set (e.g. 4) of default boxes of different aspect ratios at each location in several feature maps with different scales (e.g. 8×8 and 4×4 in (b) and (c)). For each default box, we predict both the shape offsets and the confidences for all object categories ($(c_1, c_2, \dots, c_p)$). At training time, we first match these default boxes to the ground truth boxes. For example, we have matched two default boxes with the cat and one with the dog, which are treated as positives and the rest as negatives. The model loss is a weighted sum between localization loss (e.g. Smooth L1 [6]) and confidence loss (e.g. Softmax).



$$L(x, c, l, g) = \frac{1}{N} (L_{conf}(x, c) + \alpha L_{loc}(x, l, g))$$

Method	mAP	FPS	batch size	# Boxes	Input resolution
Faster R-CNN (VGG16)	73.2	7	1	~ 6000	~ 1000 × 600
Fast YOLO	52.7	155	1	98	448 × 448
YOLO (VGG16)	66.4	21	1	98	448 × 448
SSD300	74.3	46	1	8732	300 × 300
SSD512	76.8	19	1	24564	512 × 512
SSD300	74.3	59	8	8732	300 × 300
SSD512	76.8	22	8	24564	512 × 512

1.Liu, Wei, et al. "Ssd: Single shot multibox detector." Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14. Springer International Publishing, 2016.
2. <https://jonathan-hui.medium.com/ssd-object-detection-single-shot-multi-box-detector-for-real-time-processing-9bd8deac0e06>

YOLO : You Only Look Once

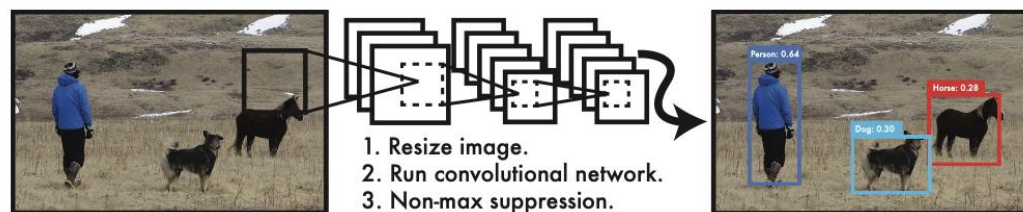
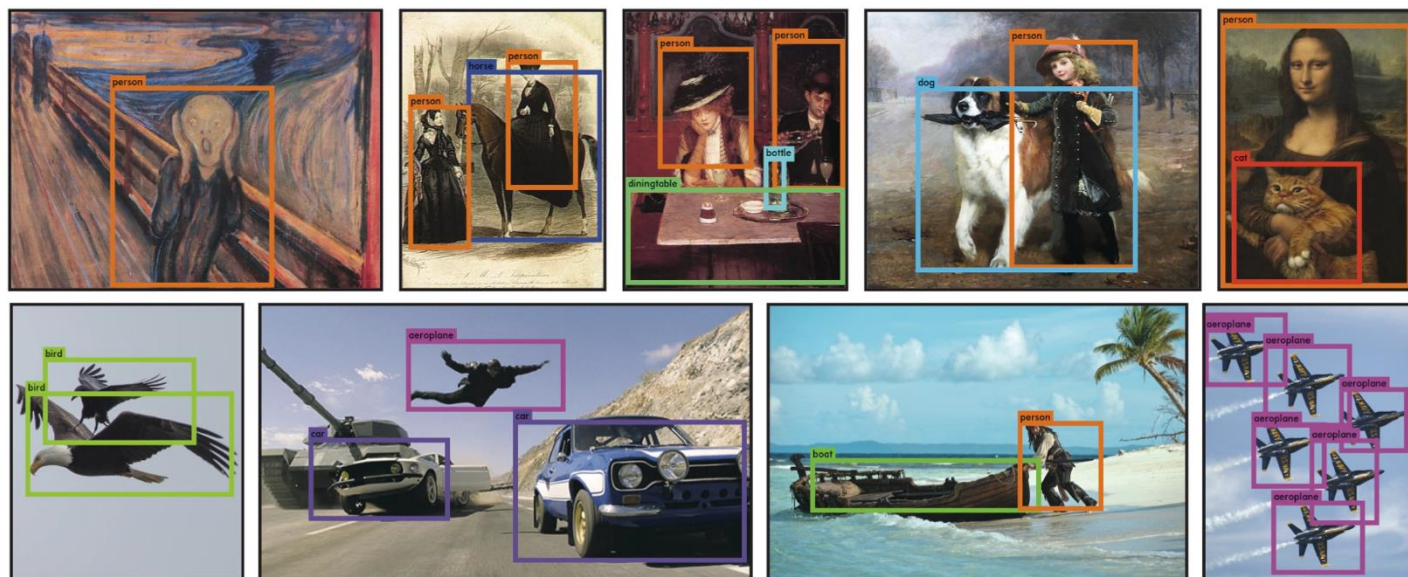
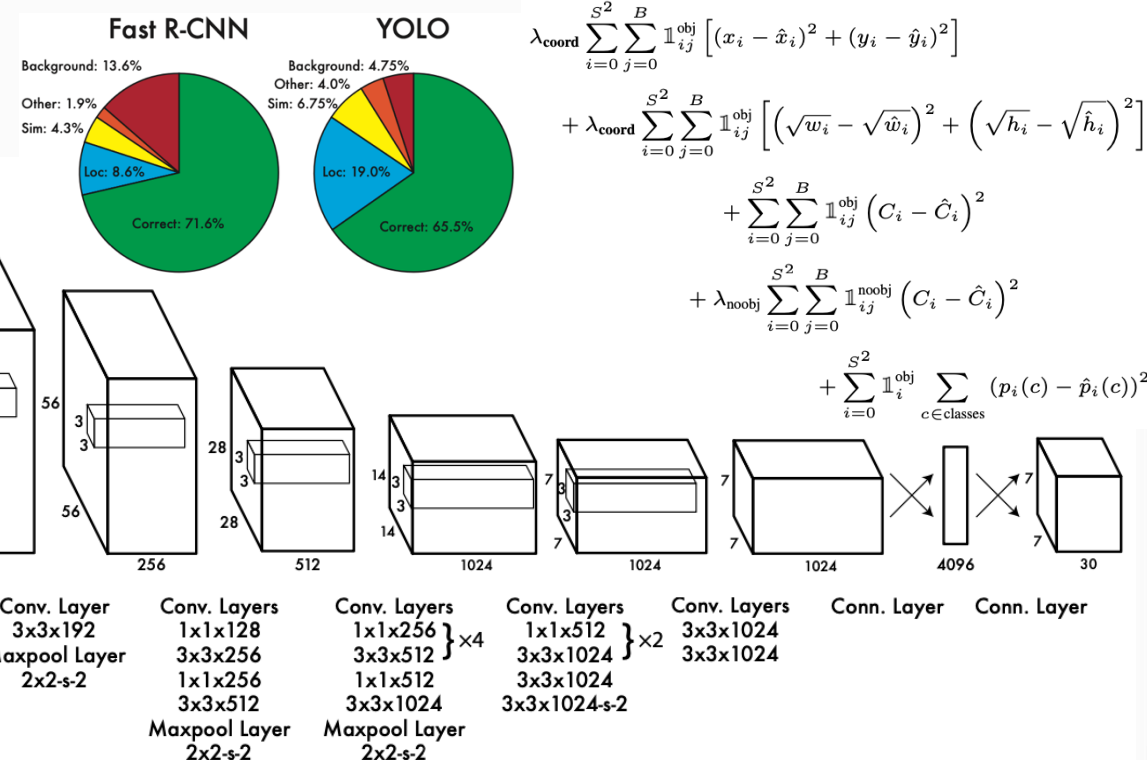
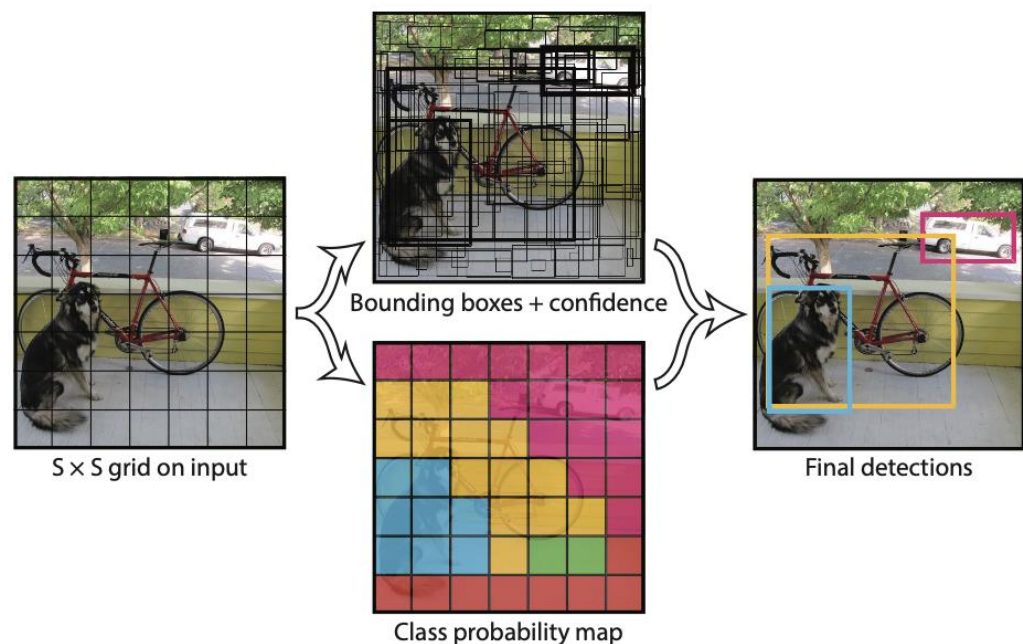


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.



U-Net : Contracting Path & Expanding Path

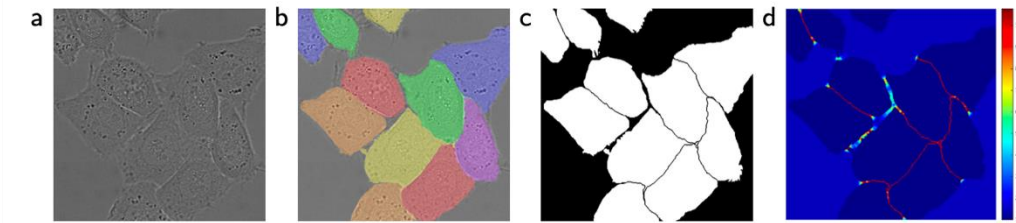
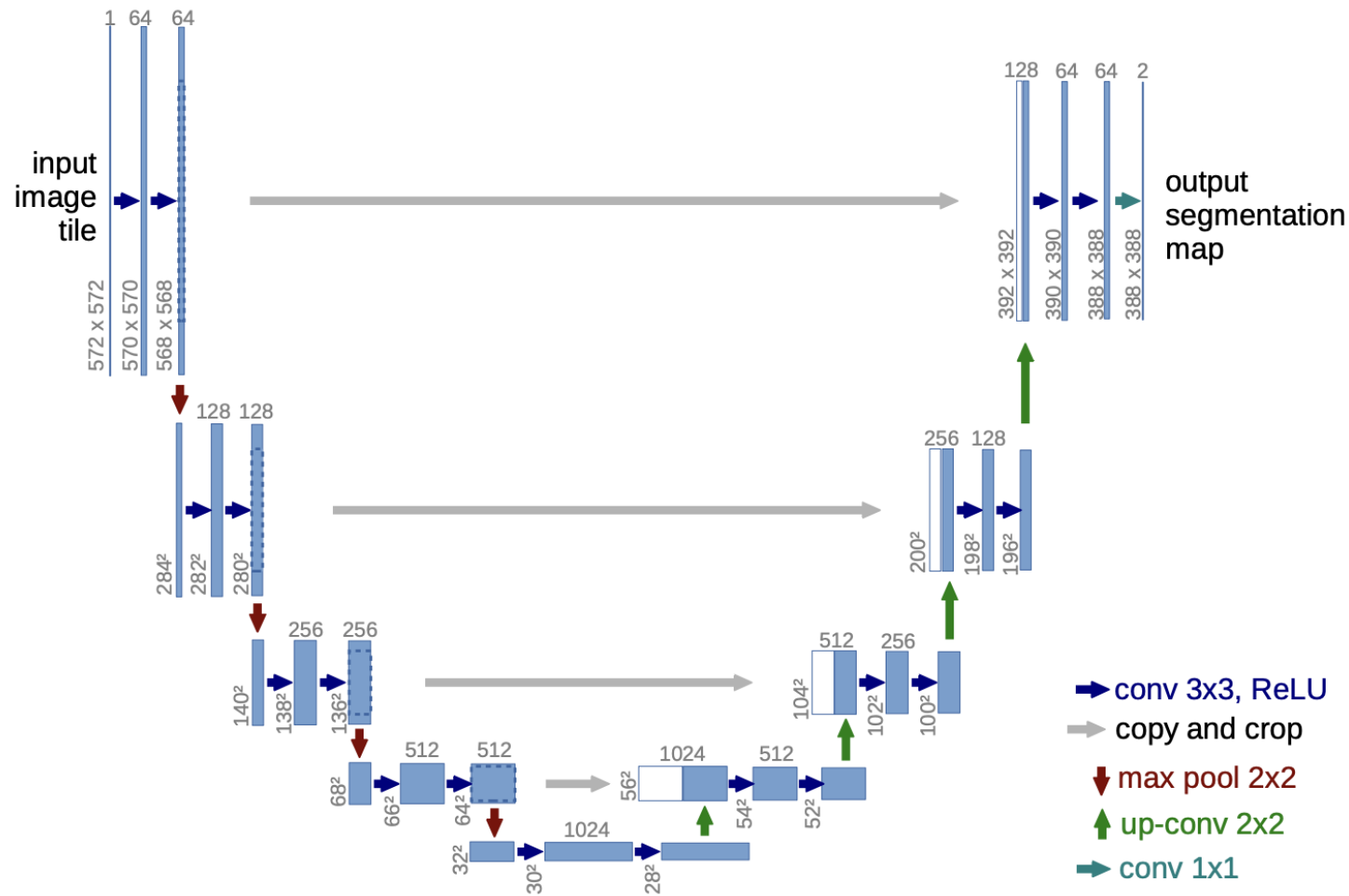


Fig. 3. HeLa cells on glass recorded with DIC (differential interference contrast) microscopy. (a) raw image. (b) overlay with ground truth segmentation. Different colors indicate different instances of the HeLa cells. (c) generated segmentation mask (white: foreground, black: background). (d) map with a pixel-wise loss weight to force the network to learn the border pixels.

$$E = \sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log(p_{\ell(\mathbf{x})}(\mathbf{x}))$$

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$

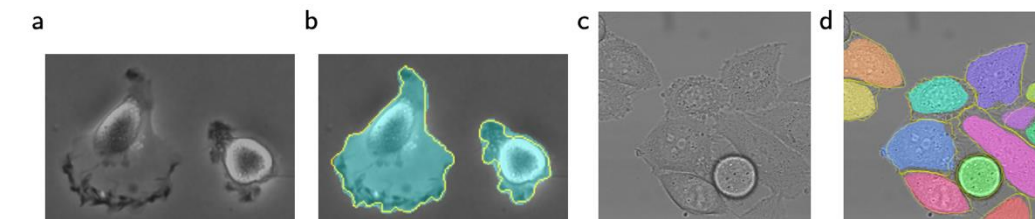
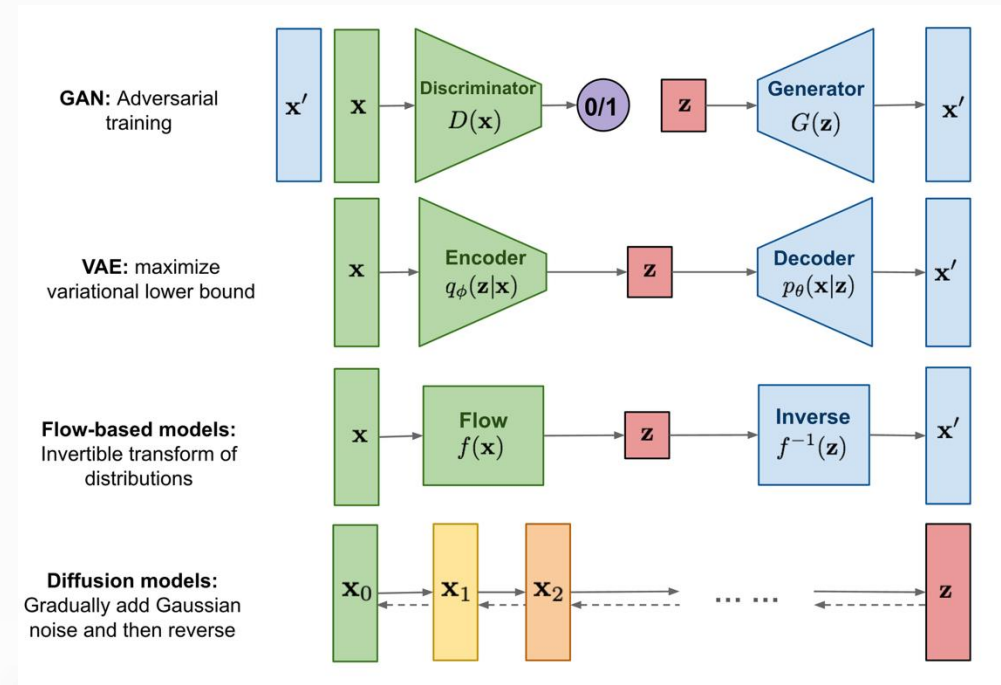


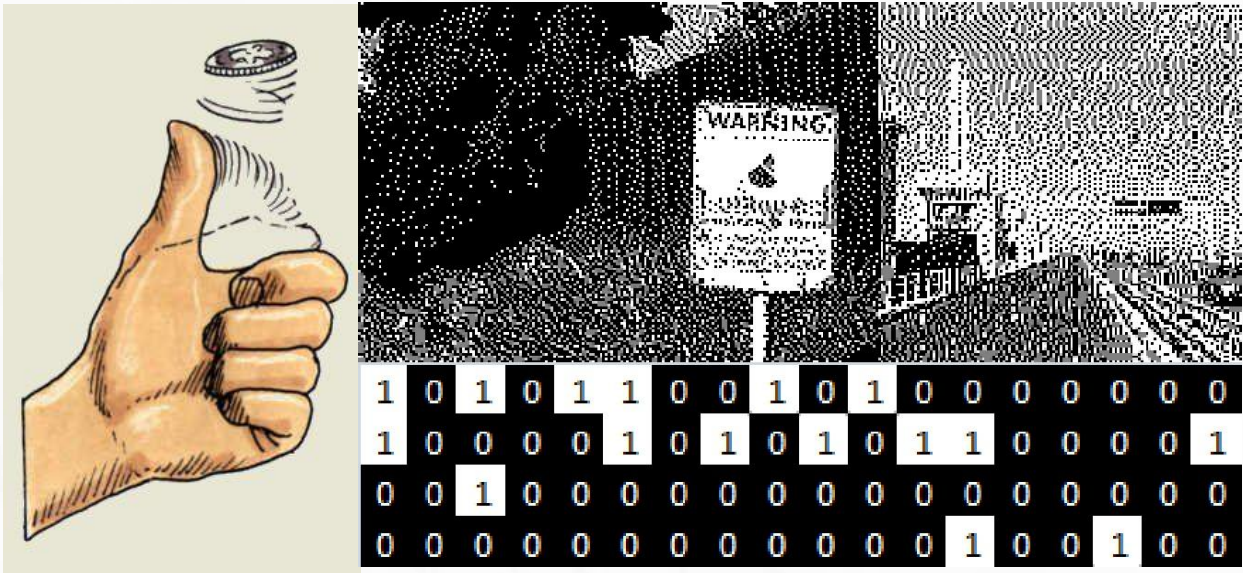
Fig. 4. Result on the ISBI cell tracking challenge. (a) part of an input image of the “PhC-U373” data set. (b) Segmentation result (cyan mask) with manual ground truth (yellow border) (c) input image of the “DIC-HeLa” data set. (d) Segmentation result (random colored masks) with manual ground truth (yellow border).



Generative Models

Photorealistic Image Synthesis

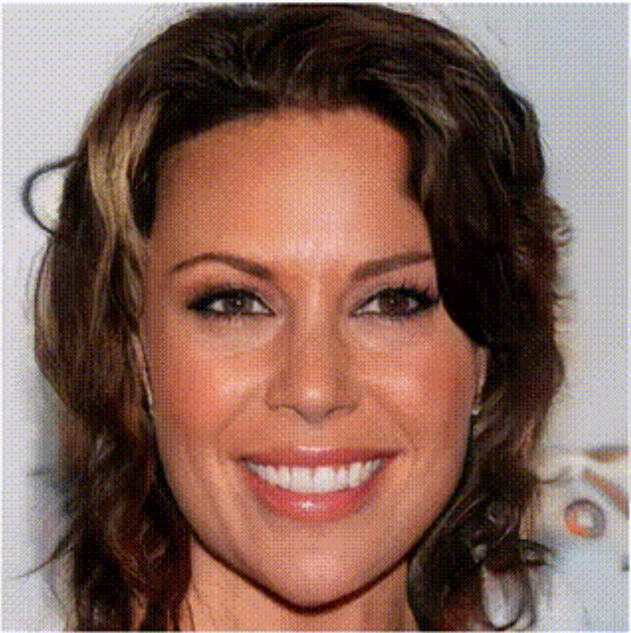
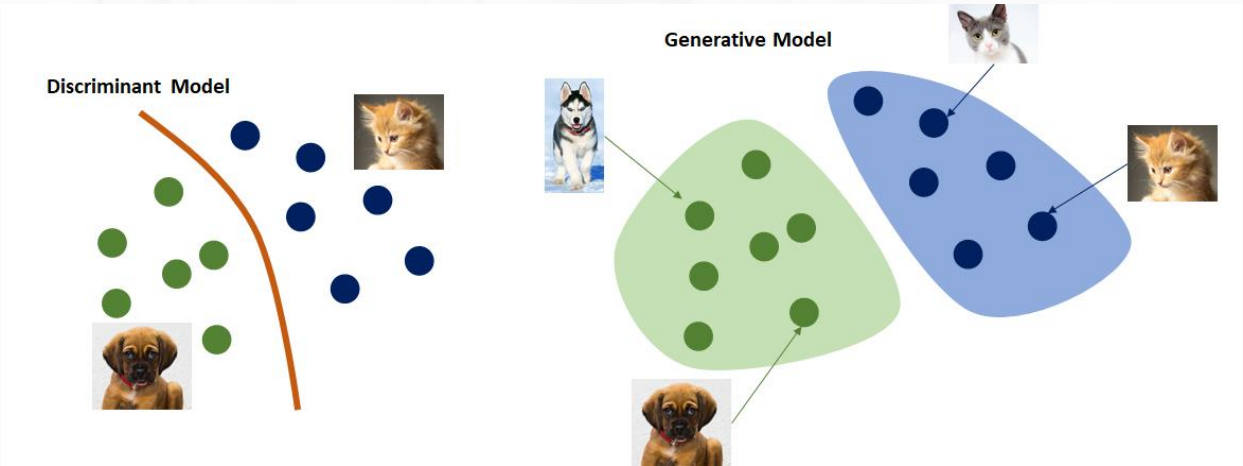
“Shallow” Image Synthesis



- Discriminative models learn the (hard or soft) **boundary** between classes ($p(y | x)$)
- Generative models model the **distribution** of individual classes ($p(x,y)$)



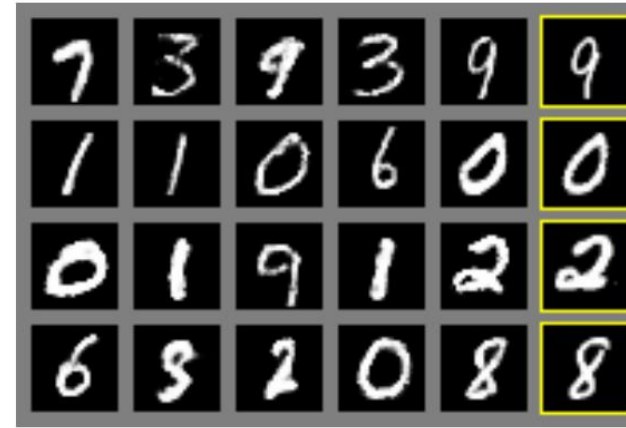
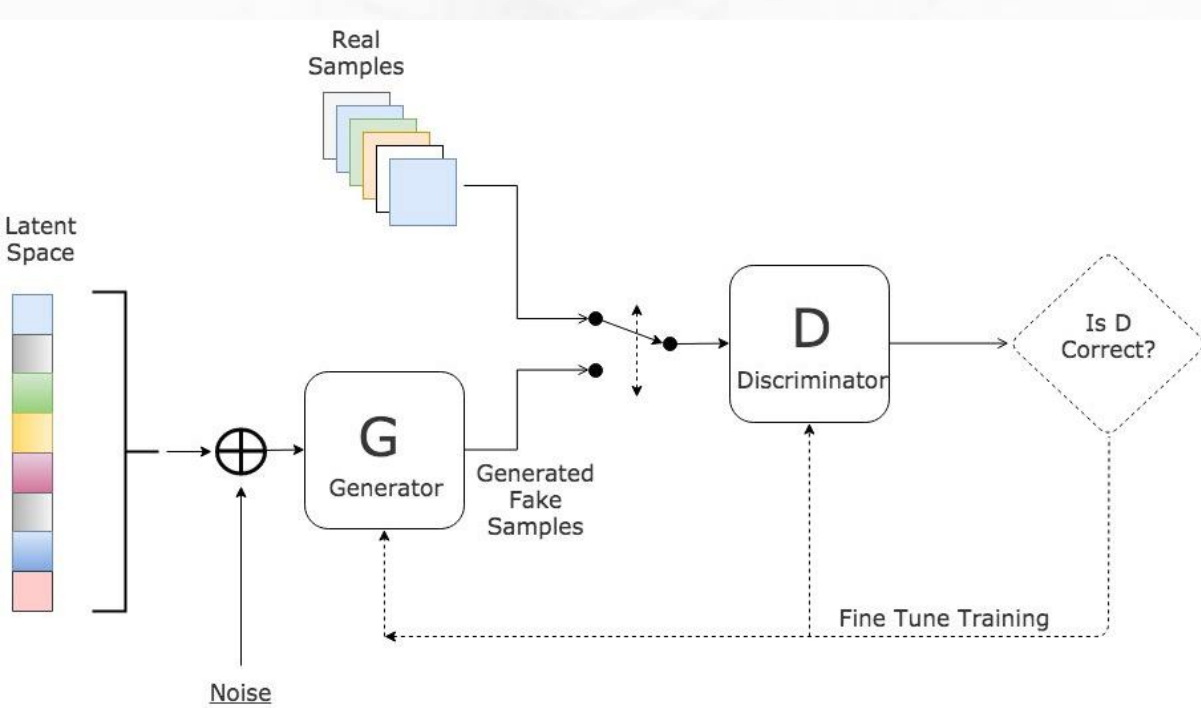
INSTRUCTION: press +/- to adjust feature, toggle feature name to lock the feature



random face		
Male	Age	Skin_Tone
- +	- +	- +
Bangs	Hairline	Bald
- +	- +	- +
Big_Nose	Pointy_Nose	Makeup
- +	- +	- +
Smiling	Mouth_Open	Wavy_Hair
- +	- +	- +
Beard	Goatee	Sideburns
- +	- +	- +
Blond_Hair	Black_Hair	Gray_Hair
- +	- +	- +
Eyeglasses	Earrings	Necktie
- +	- +	- +

1. <https://medium.com/@jordi299/about-generative-and-discriminative-models-d8958b67ad32>

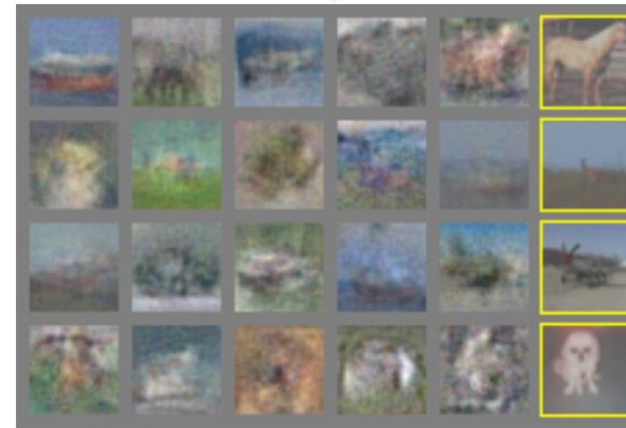
GAN : Generative Adversarial Nets



a)



b)



c)



d)

Figure 2: Visualization of samples from the model. Rightmost column shows the nearest training example of the neighboring sample, in order to demonstrate that the model has not memorized the training set. Samples are fair random draws, not cherry-picked. Unlike most other visualizations of deep generative models, these images show actual samples from the model distributions, not conditional means given samples of hidden units. Moreover, these samples are uncorrelated because the sampling process does not depend on Markov chain mixing. a) MNIST b) TFD c) CIFAR-10 (fully connected model) d) CIFAR-10 (convolutional discriminator and “deconvolutional” generator)

Wasserstein GAN

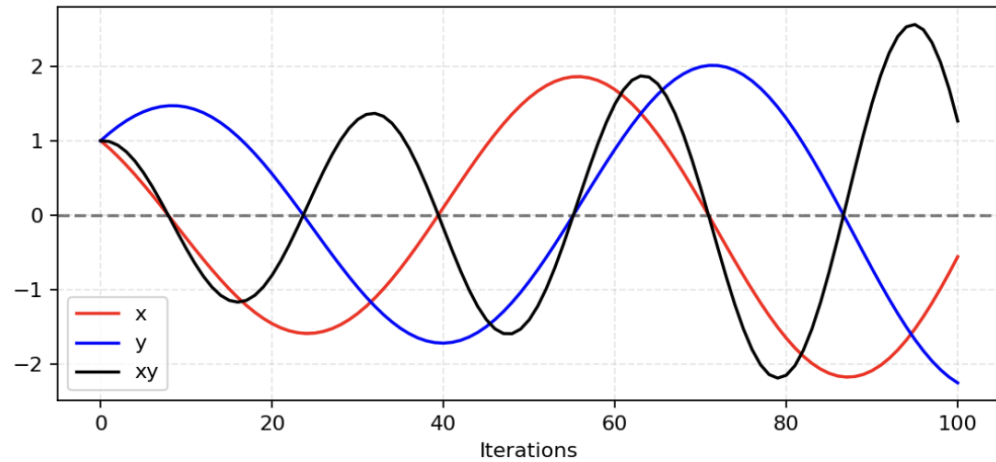


Fig. 3. A simulation of our example for updating x to minimize xy and updating y to minimize $-xy$. The learning rate $\eta = 0.1$. With more iterations, the oscillation grows more and more unstable.

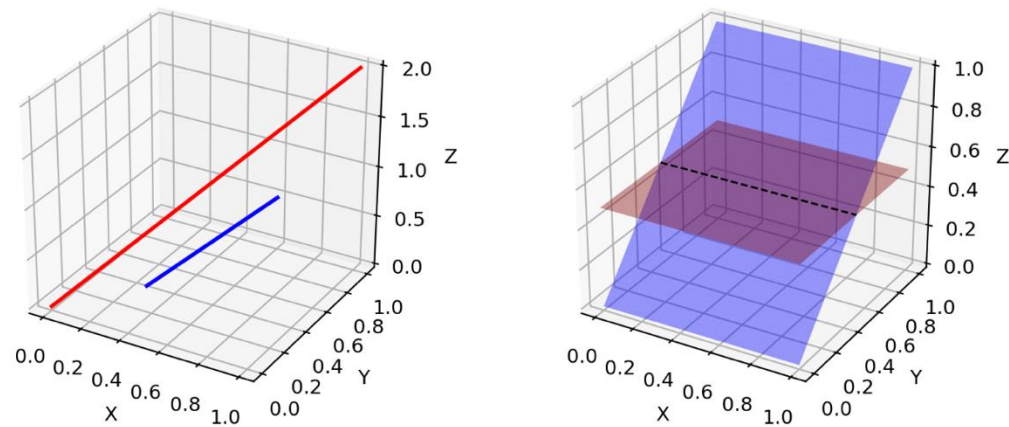
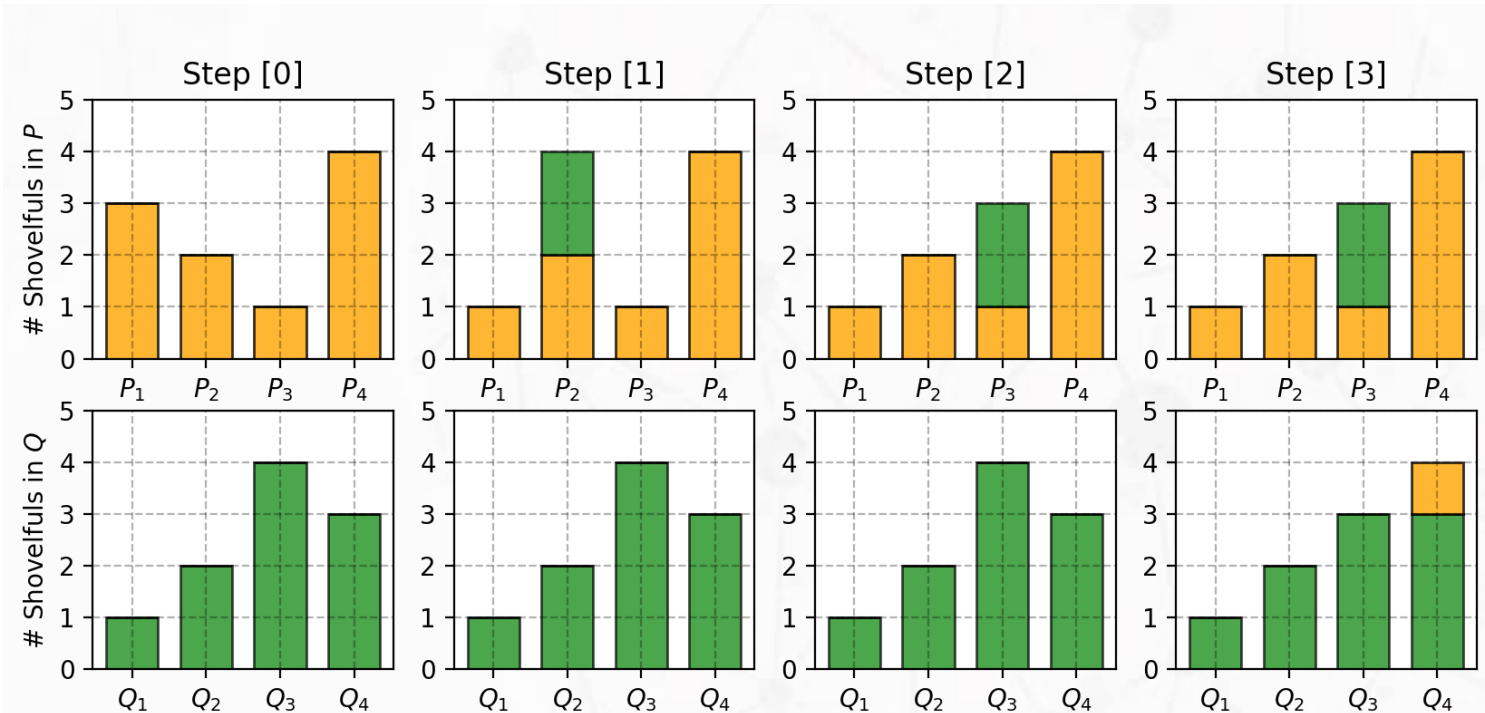


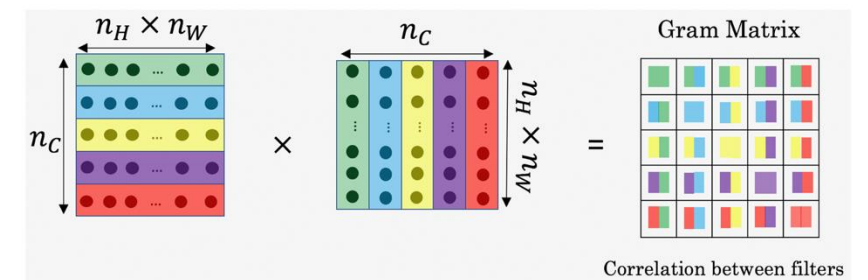
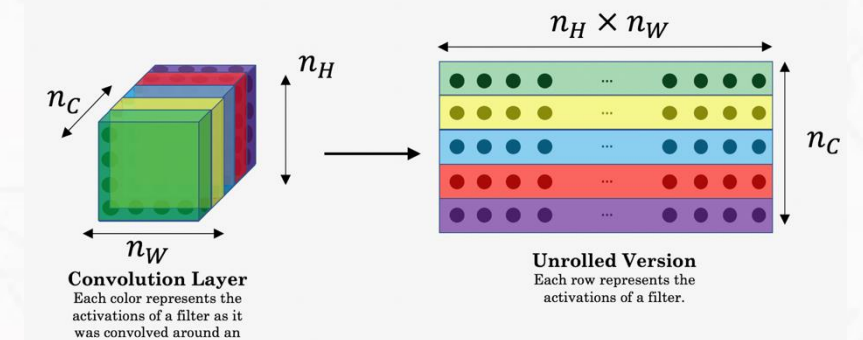
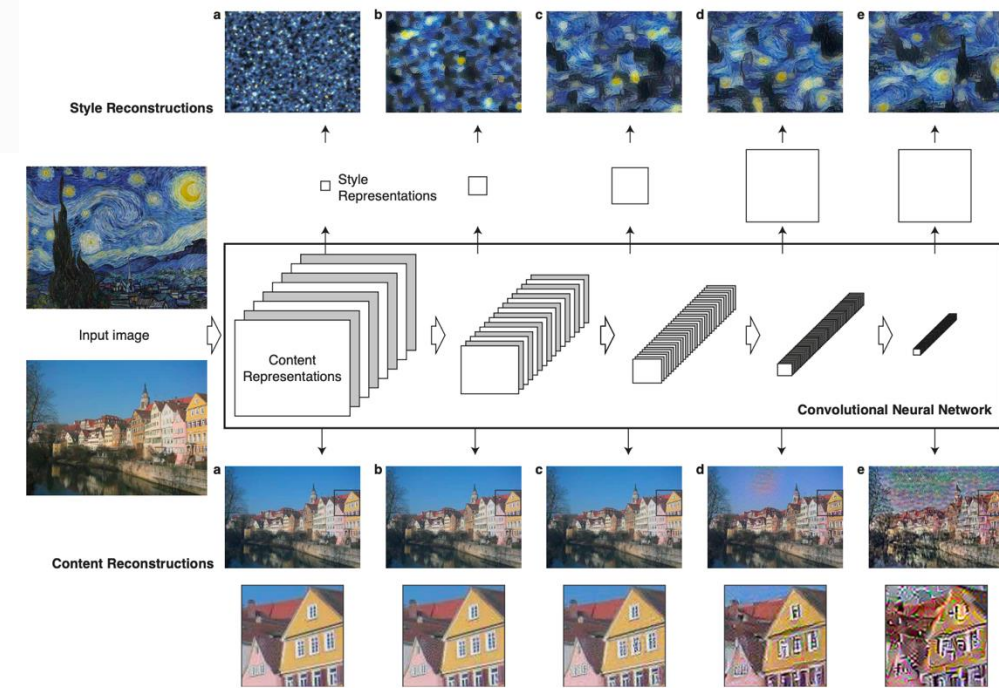
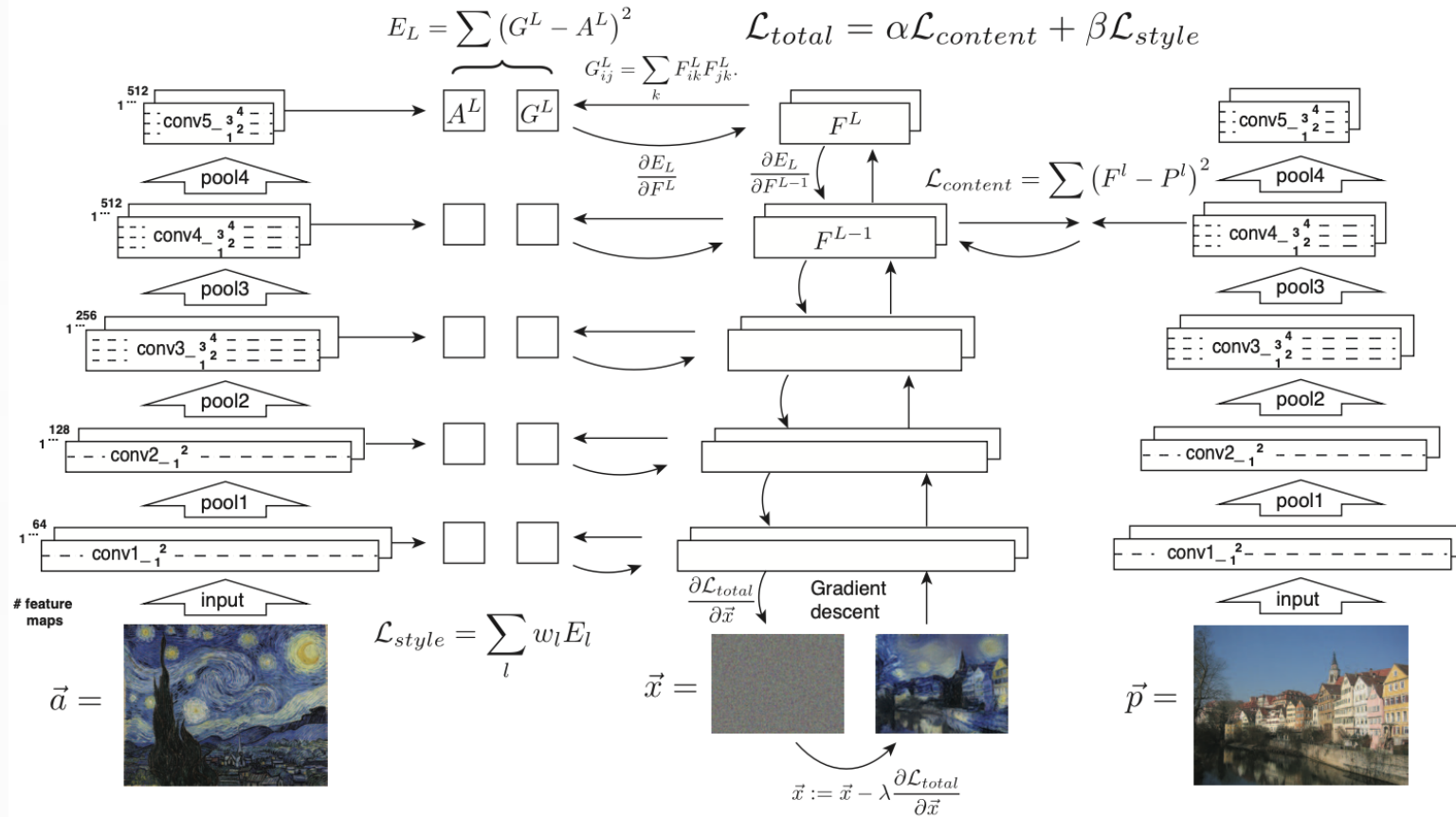
Fig. 4. Low dimensional manifolds in high dimension space can hardly have overlaps. (Left) Two lines in a three-dimension space. (Right) Two surfaces in a three-dimension space.



Fig. 6. A DCGAN model is trained with an MLP network with 4 layers, 512 units and ReLU activation function, configured to lack a strong inductive bias for image generation. The results shows a significant degree of mode collapse. (Image source: [Arjovsky, Chintala, & Bottou, 2017.](#))



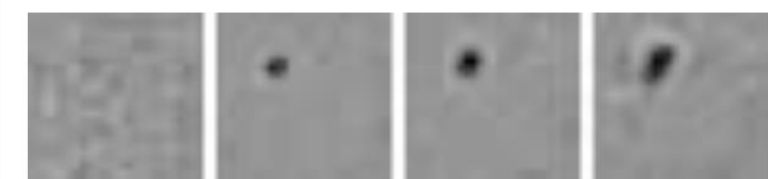
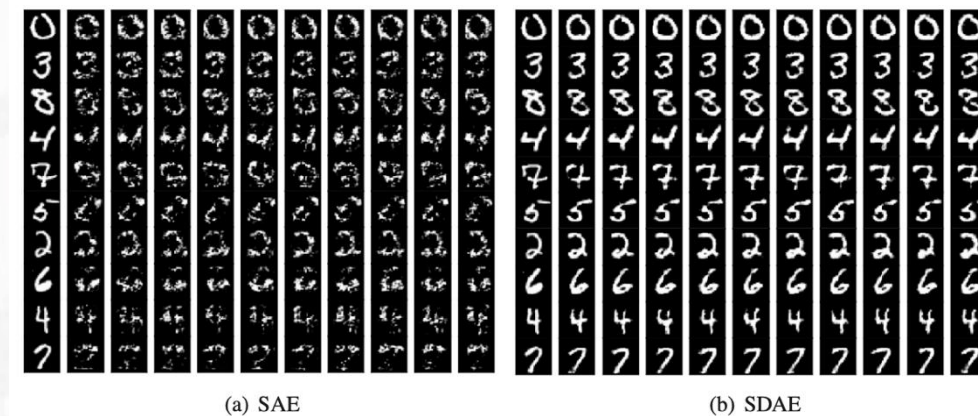
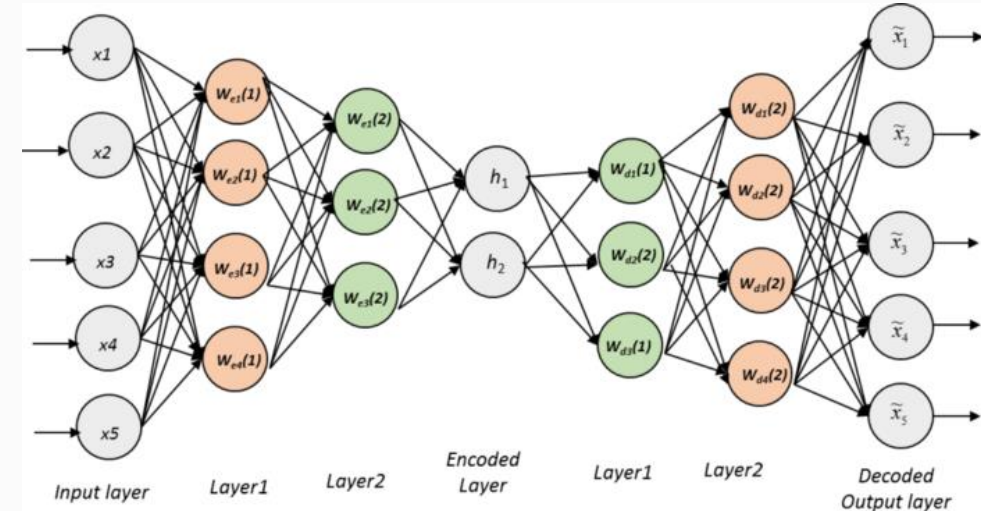
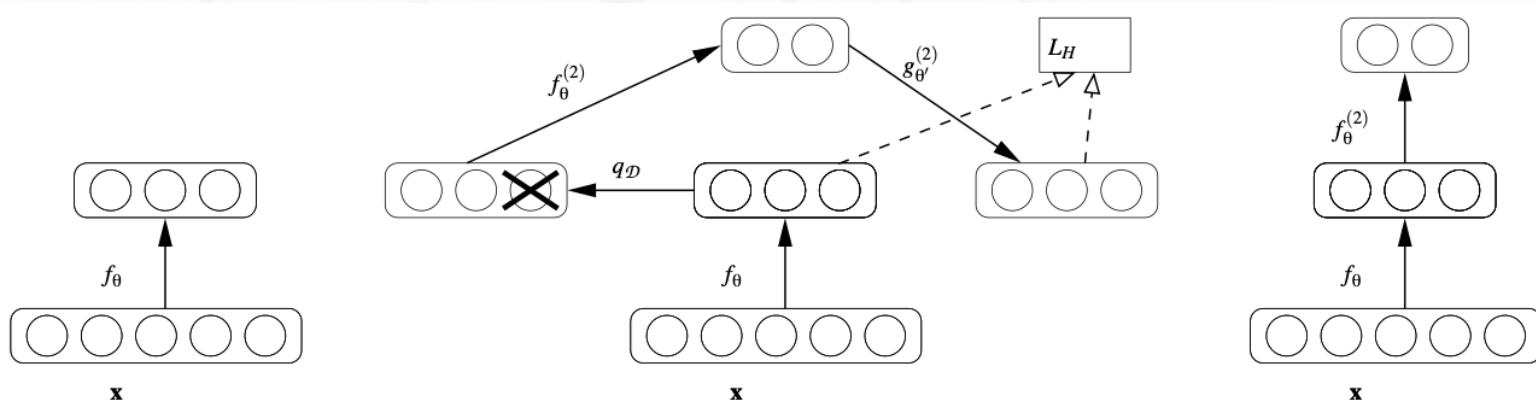
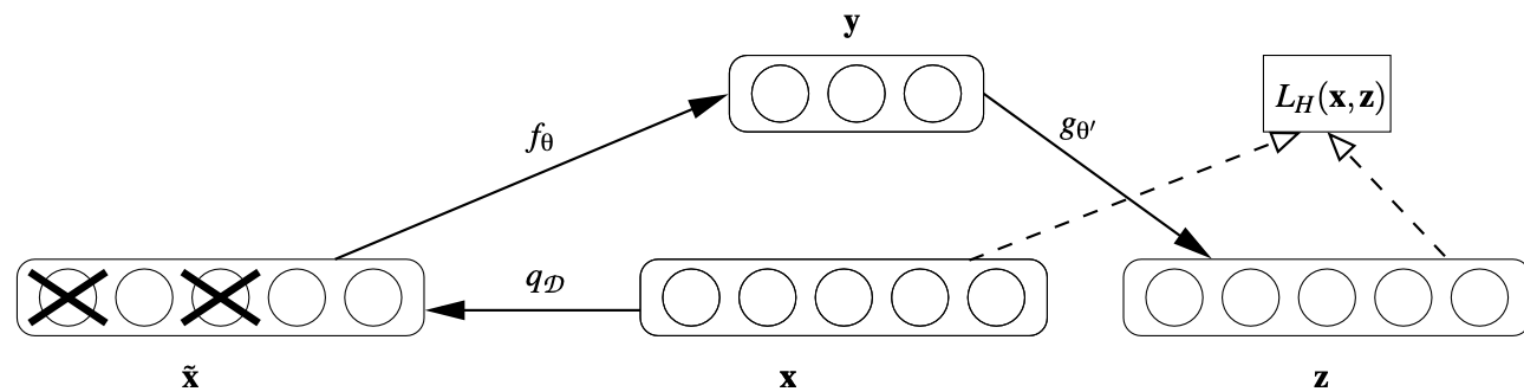
Neural Style-Transfer



$$\mathbf{G}_{gram} = \mathbf{A}_{unrolled} \mathbf{A}_{unrolled}^T$$

Figure 2. Style transfer algorithm. First content and style features are extracted and stored. The style image $\vec{a}$ is passed through the network and its style representation A^l on all layers included are computed and stored (left). The content image $\vec{p}$ is passed through the network and the content representation P^l in one layer is stored (right). Then a random white noise image $\vec{x}$ is passed through the network and its style features G^l and content features F^l are computed. On each layer included in the style representation, the element-wise mean squared difference between G^l and A^l is computed to give the style loss $\mathcal{L}_{style}$ (left). Also the mean squared difference between F^l and P^l is computed to give the content loss $\mathcal{L}_{content}$ (right). The total loss $\mathcal{L}_{total}$ is then a linear combination between the content and the style loss. Its derivative with respect to the pixel values can be computed using error back-propagation (middle). This gradient is used to iteratively update the image $\vec{x}$ until it simultaneously matches the style features of the style image $\vec{a}$ and the content features of the content image $\vec{p}$ (middle, bottom).

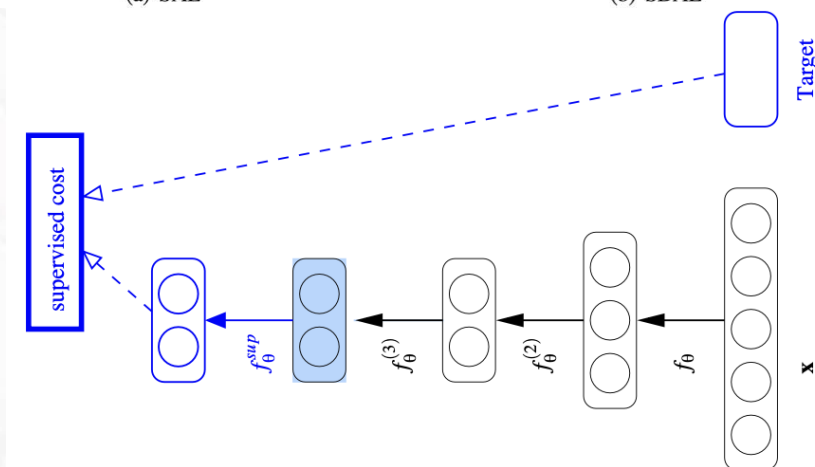
Stacked Denoise Auto-Encoder



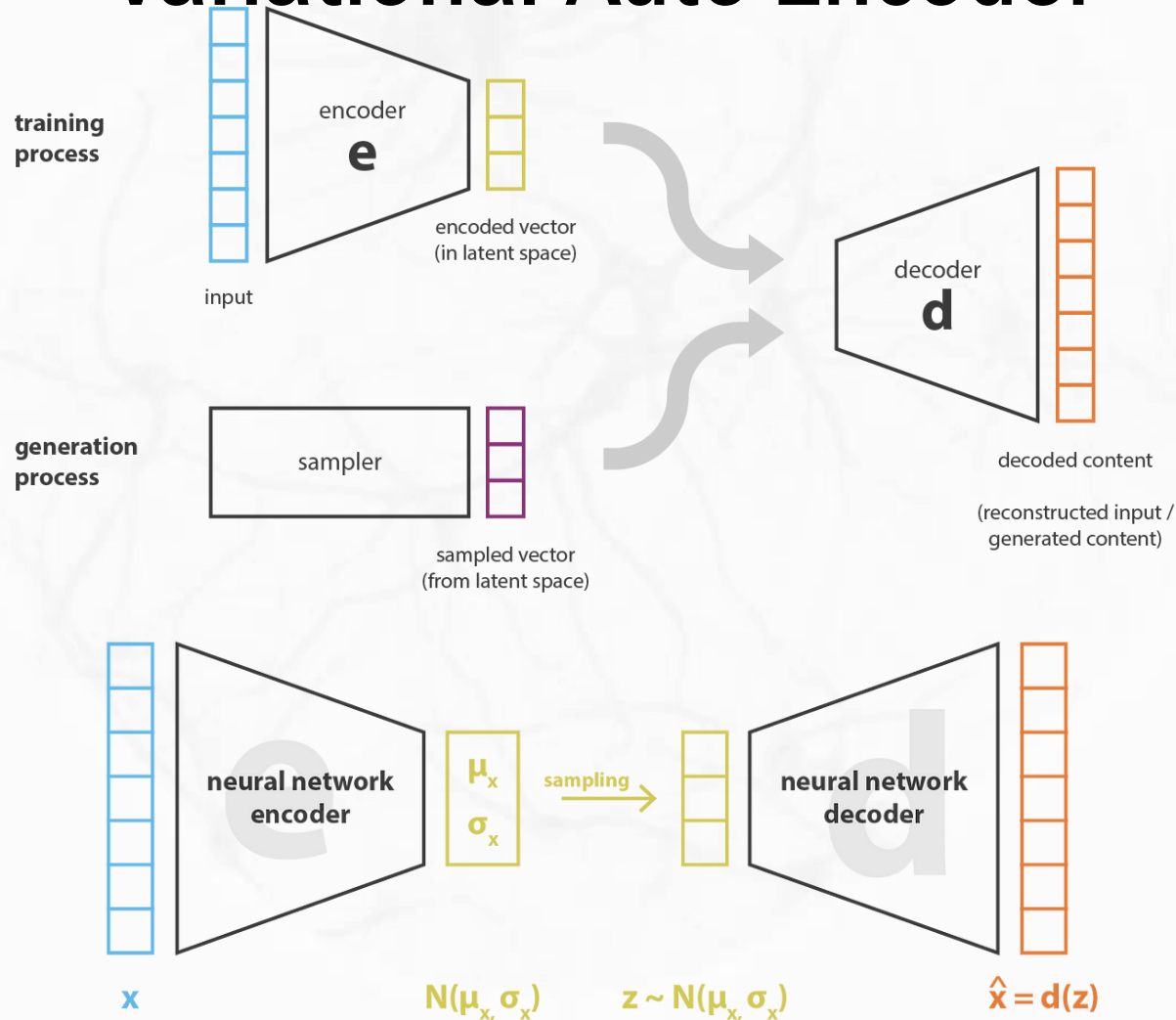
(d) Neuron A (0%, 10%, 20%, 50% corruption)



(e) Neuron B (0%, 10%, 20%, 50% corruption)



Variational Auto-Encoder



$$\text{loss} = ||x - \hat{x}||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = ||x - d(z)||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$

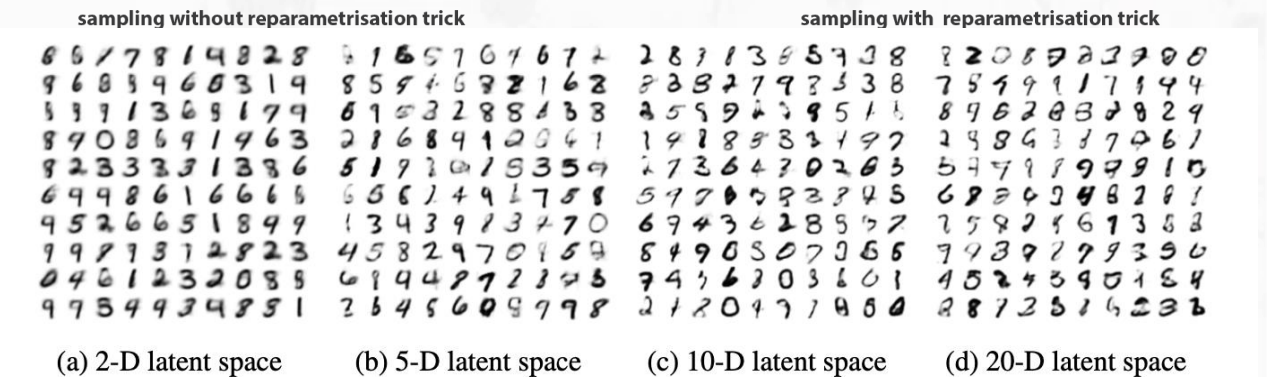
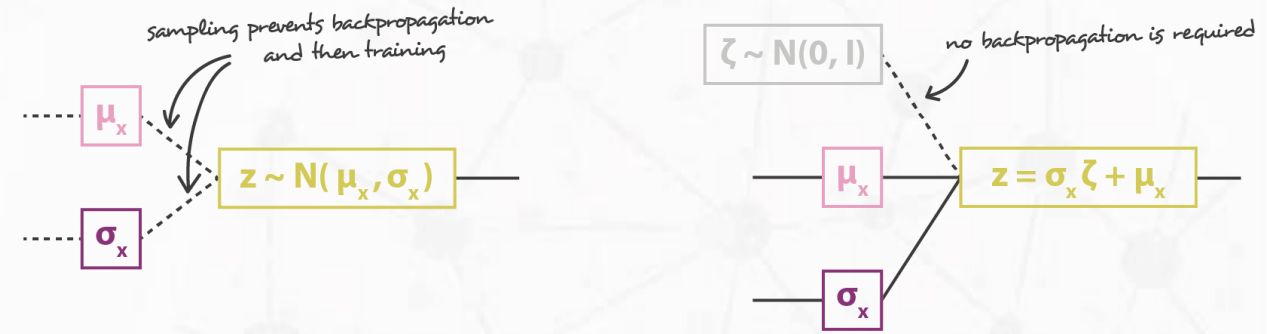
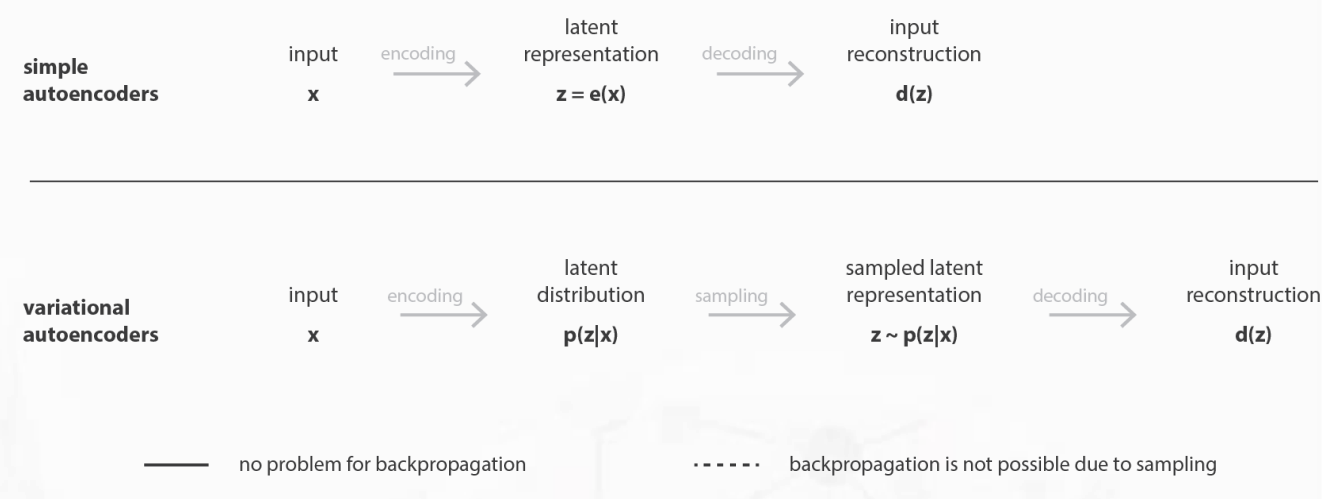


Figure 5: Random samples from learned generative models of MNIST for different dimensionalities of latent space.

Autoregressive Model : PixelRNN + PixelCNN

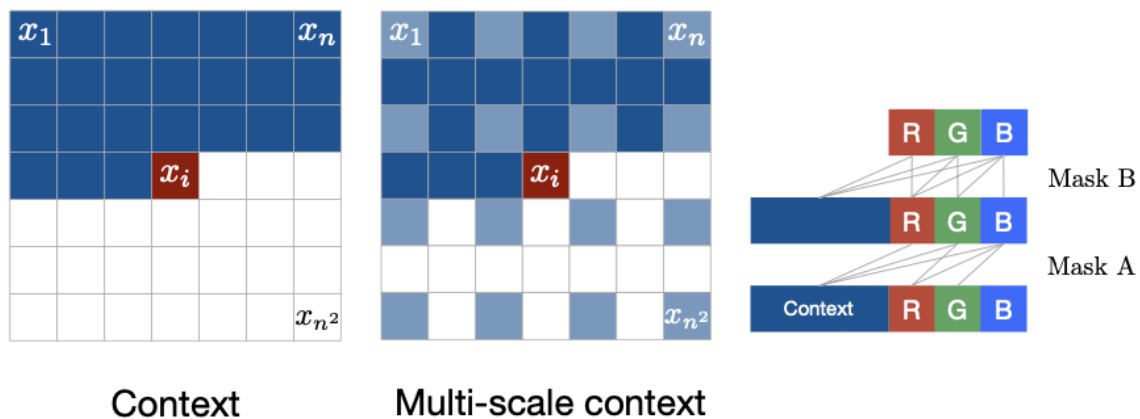


Figure 1. Image completions sampled from a PixelRNN.

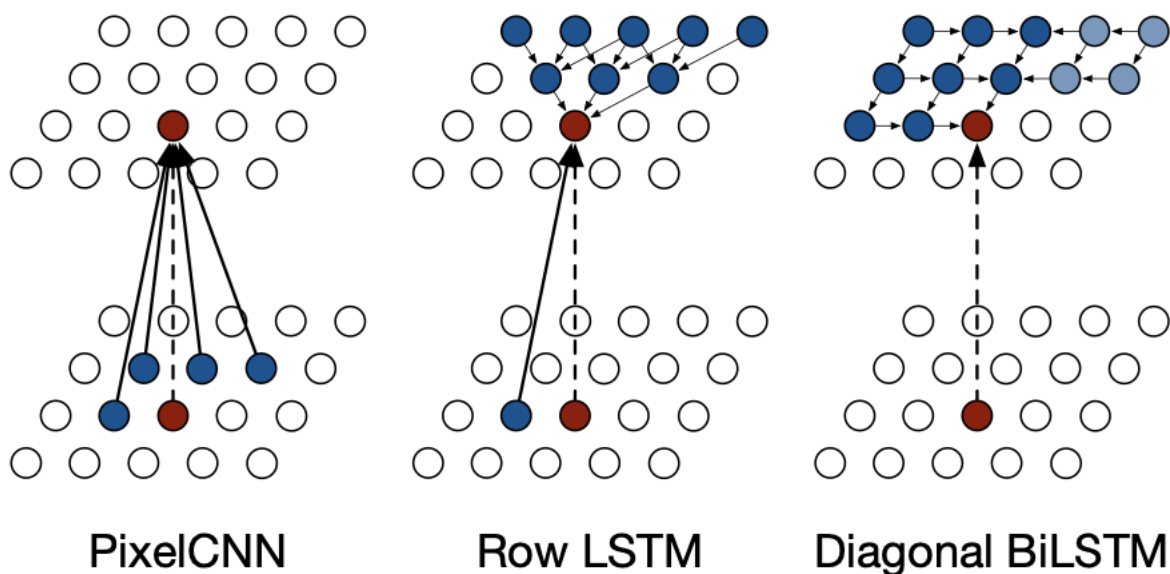
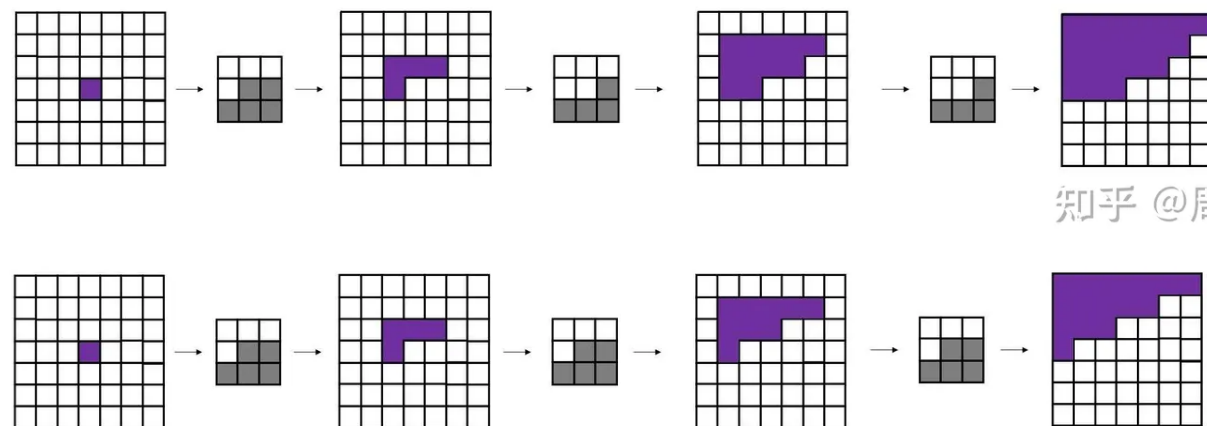


Figure 4. Visualization of the input-to-state and state-to-state mappings for the three proposed architectures.



知乎 @周弈帆

知乎 @周弈帆

Gated PixelCNN

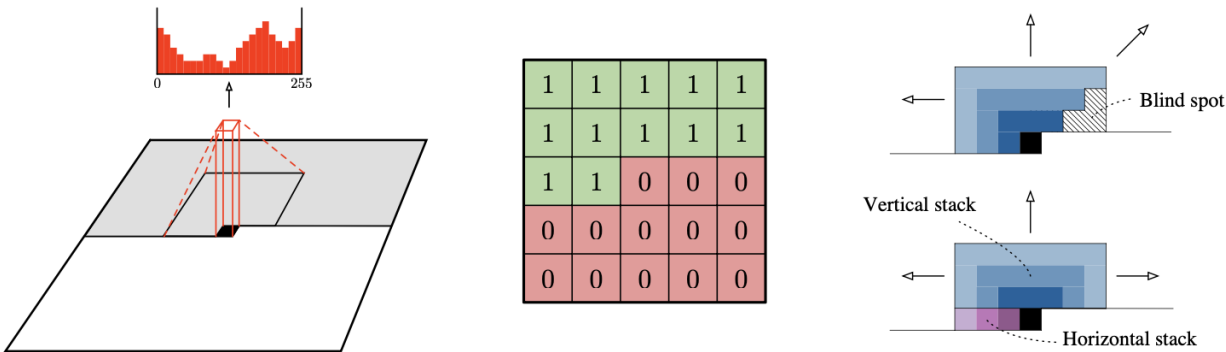
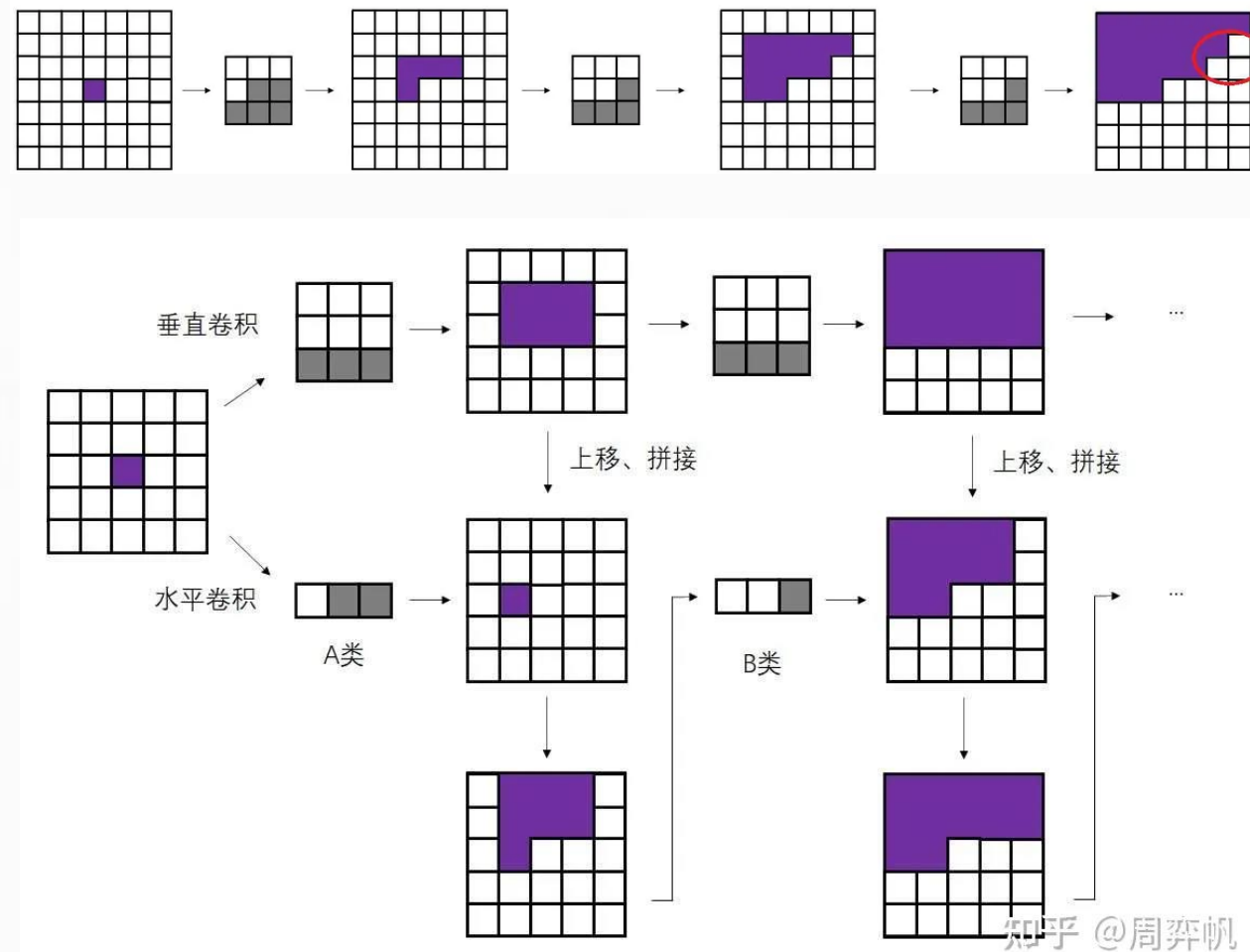
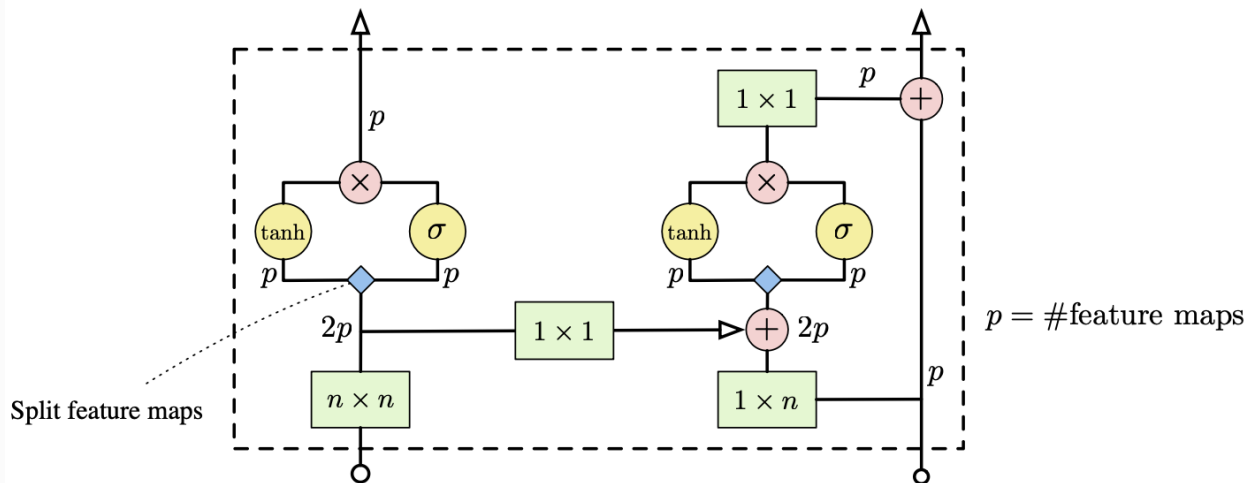
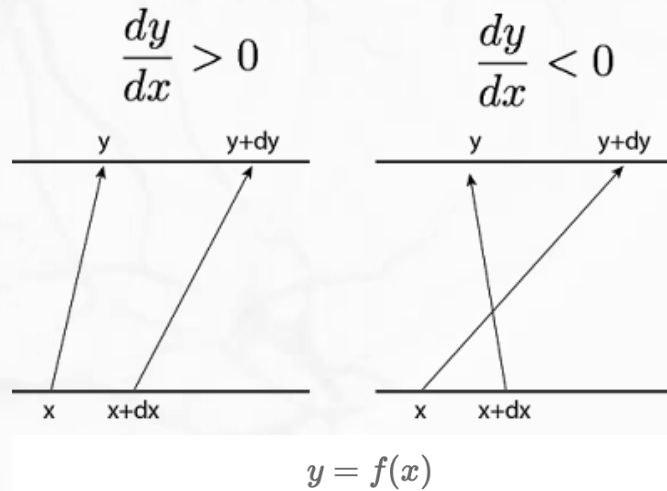
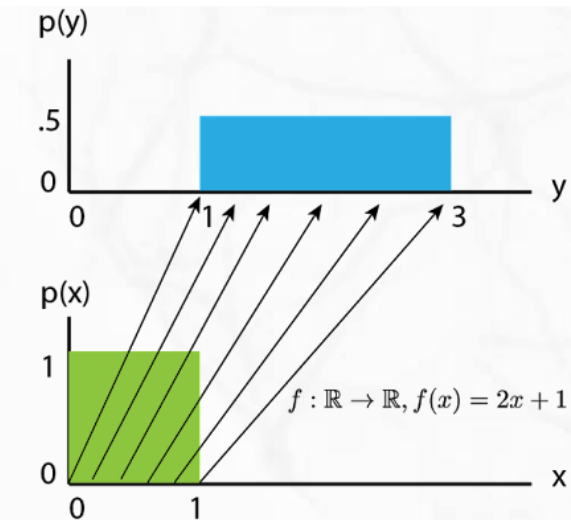
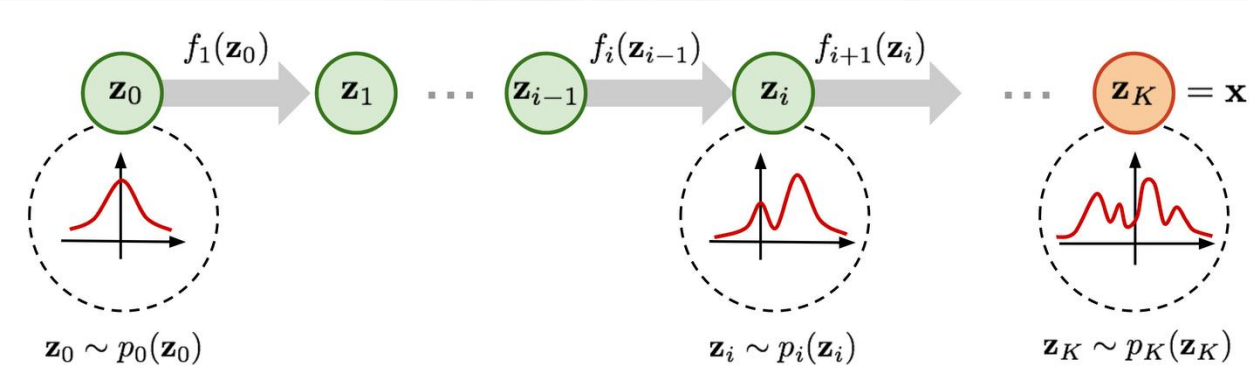


Figure 1: **Left:** A visualization of the PixelCNN that maps a neighborhood of pixels to prediction for the next pixel. To generate pixel x_i the model can only condition on the previously generated pixels $x_1, \dots, x_{i-1}$. **Middle:** an example matrix that is used to mask the 5x5 filters to make sure the model cannot read pixels below (or strictly to the right) of the current pixel to make its predictions. **Right:** Top: PixelCNNs have a *blind spot* in the receptive field that can not be used to make predictions. Bottom: Two convolutional stacks (blue and purple) allow to capture the whole receptive field.



$$y = \tanh(W_{k,f} * \mathbf{x} + V_{k,f}^T \mathbf{h}) \odot \sigma(W_{k,g} * \mathbf{x} + V_{k,g}^T \mathbf{h}),$$

Normalising Flows



$$p(y) = p(f^{-1}(y)) \cdot |\det J(f^{-1}(y))|$$

$$\log p(y) = \log p(f^{-1}(y)) + \log |\det(J(f^{-1}(y)))|$$

• NICE

$$\begin{cases} \mathbf{y}_{1:d} &= \mathbf{x}_{1:d} \\ \mathbf{y}_{d+1:D} &= \mathbf{x}_{d+1:D} + m(\mathbf{x}_{1:d}) \end{cases} \Leftrightarrow \begin{cases} \mathbf{x}_{1:d} &= \mathbf{y}_{1:d} \\ \mathbf{x}_{d+1:D} &= \mathbf{y}_{d+1:D} - m(\mathbf{y}_{1:d}) \end{cases}$$

• RealNVP

$$\begin{cases} \mathbf{y}_{1:d} &= \mathbf{x}_{1:d} \\ \mathbf{y}_{d+1:D} &= \mathbf{x}_{d+1:D} \odot \exp(s(\mathbf{x}_{1:d})) + t(\mathbf{x}_{1:d}) \end{cases} \Leftrightarrow \begin{cases} \mathbf{x}_{1:d} &= \mathbf{y}_{1:d} \\ \mathbf{x}_{d+1:D} &= (\mathbf{y}_{d+1:D} - t(\mathbf{y}_{1:d})) \odot \exp(-s(\mathbf{y}_{1:d})) \end{cases}$$

$$\mathbf{J} = \begin{bmatrix} \mathbb{I}_d & \mathbf{0}_{d \times (D-d)} \\ \frac{\partial \mathbf{y}_{d+1:D}}{\partial \mathbf{x}_{1:d}} & \text{diag}(\exp(s(\mathbf{x}_{1:d}))) \end{bmatrix} \quad \det(\mathbf{J}) = \prod_{j=1}^{D-d} \exp(s(\mathbf{x}_{1:d}))_j = \exp\left(\sum_{j=1}^{D-d} s(\mathbf{x}_{1:d})_j\right)$$

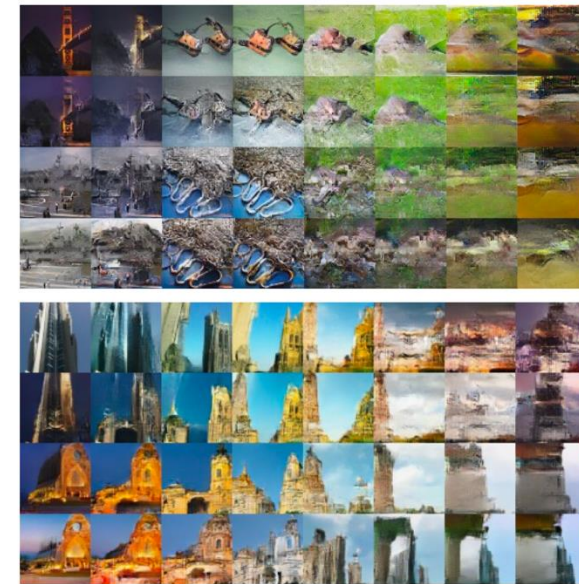
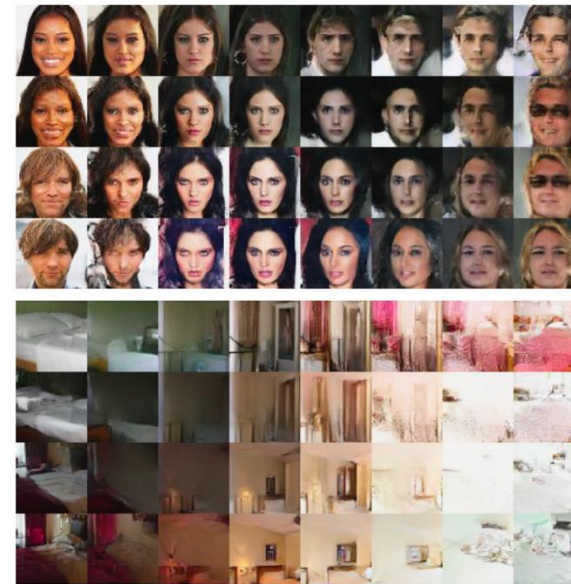


Figure 6: Manifold generated from four examples in the dataset. Clockwise from top left: CelebA, Imagenet (64 × 64), LSUN (tower), LSUN (bedroom).

1. Rezende, Danilo, and Shakir Mohamed. "Variational inference with normalizing flows." International conference on machine learning. PMLR, 2015.

2. <https://lilianweng.github.io/posts/2018-10-13-flow-models/>

3. <https://blog.evjang.com/2018/01/nf1.html> & <https://blog.evjang.com/2018/01/nf2.html>

DDPM : Energy-based Model

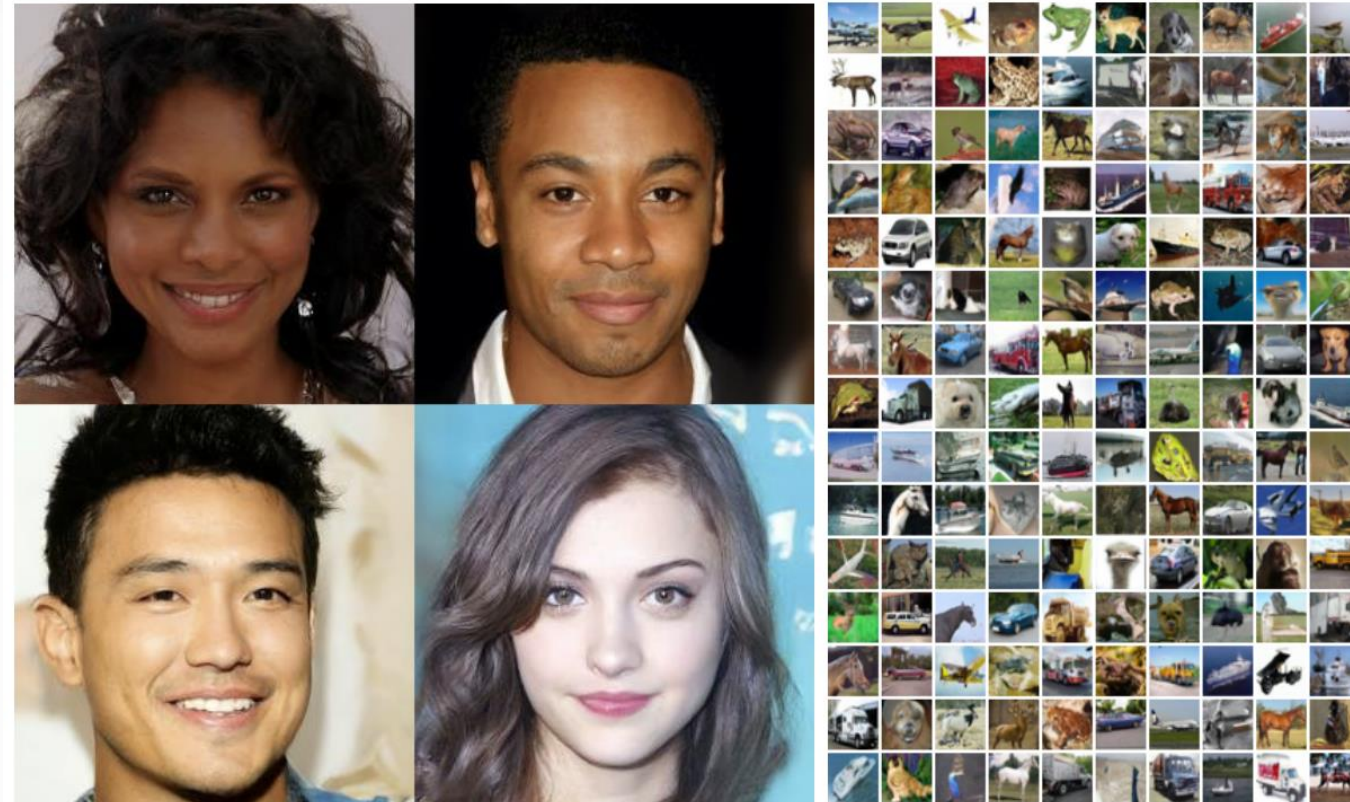
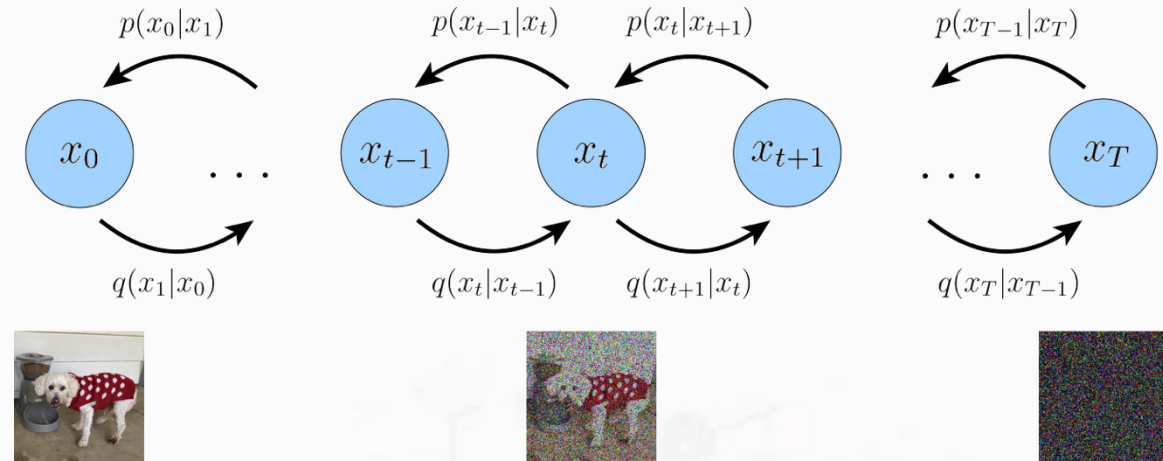
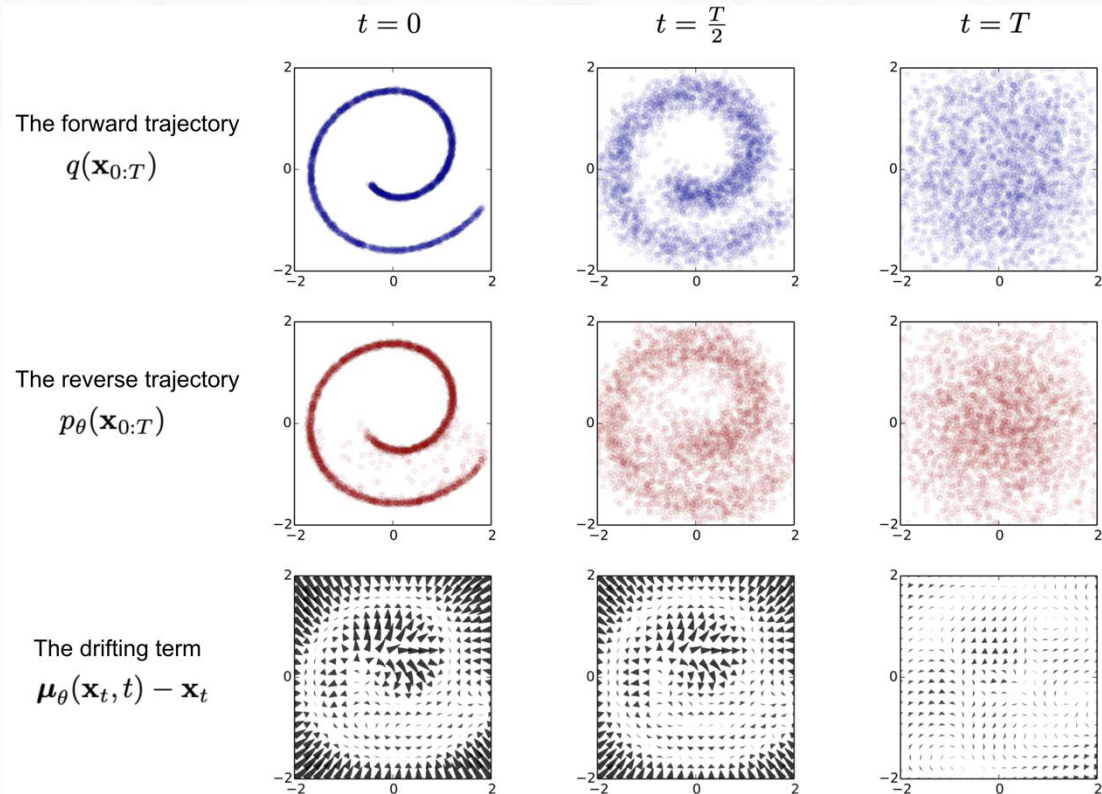
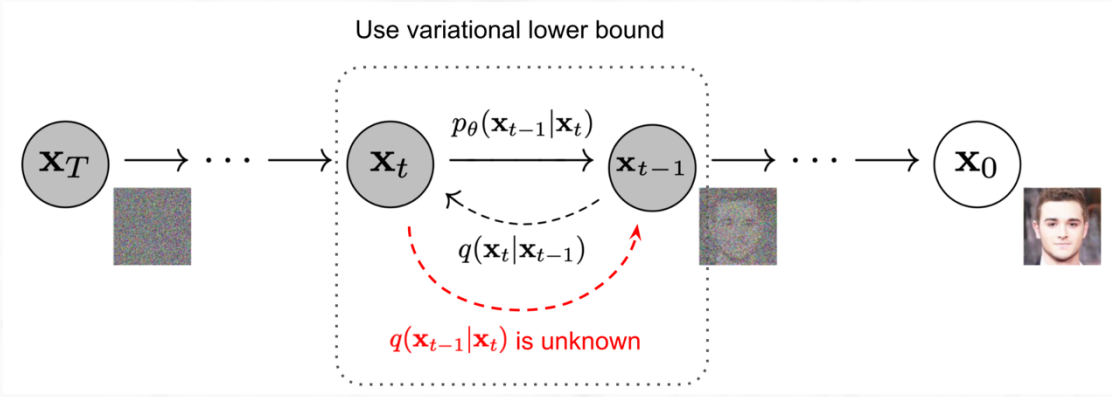


Figure 1: Generated samples on CelebA-HQ 256 x 256 (left) and unconditional CIFAR10 (right)

1. Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." Advances in neural information processing systems 33 (2020): 6840-6851.

2. Luo, Calvin. "Understanding diffusion models: A unified perspective." arXiv preprint arXiv:2208.11970 (2022).

3. <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>

Latent Diffusion

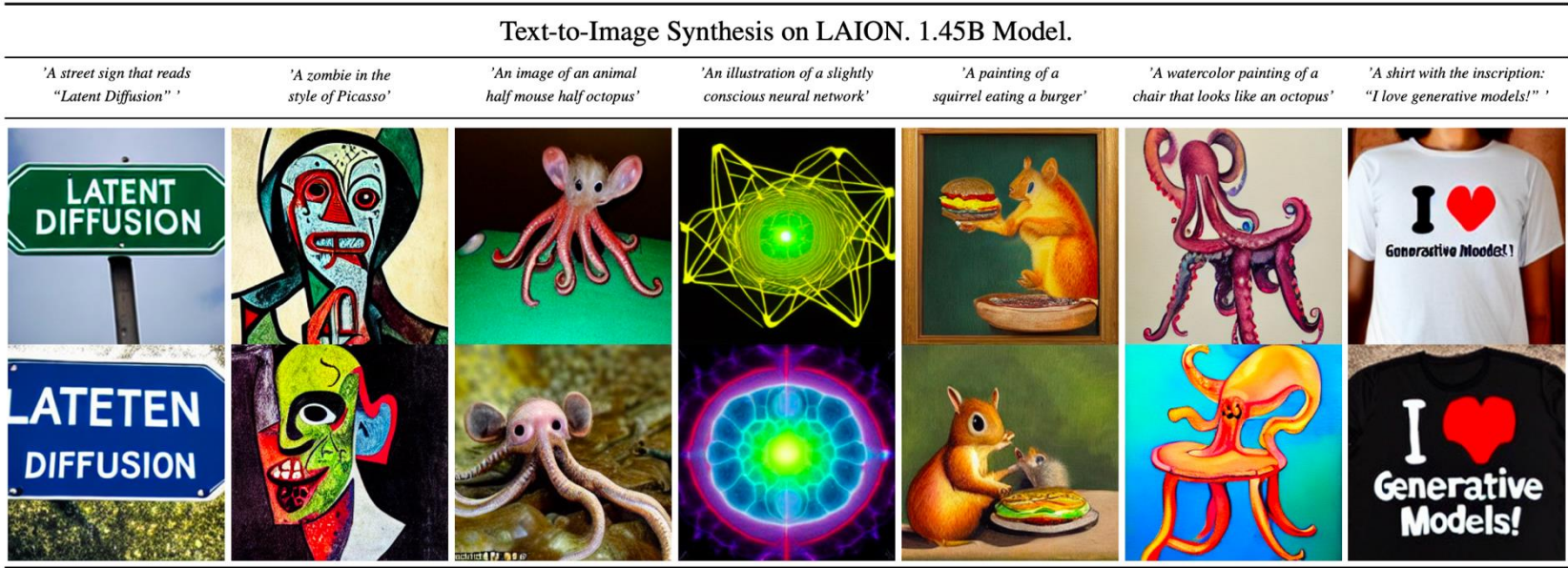
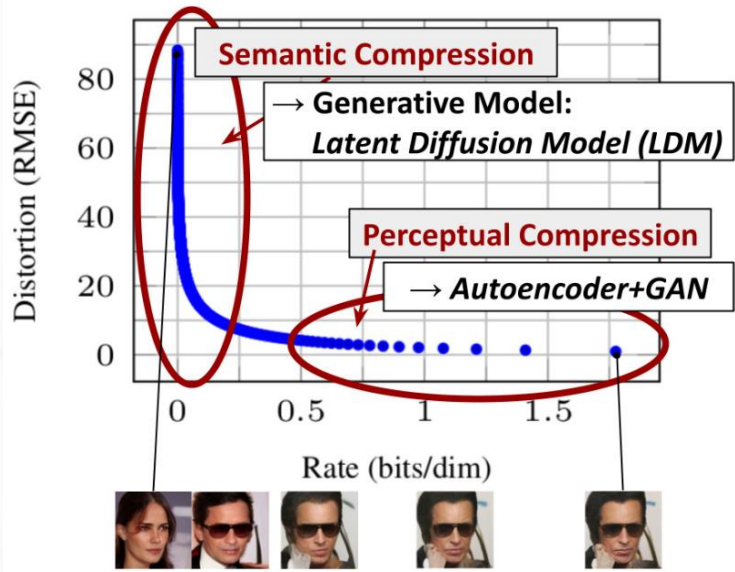


Figure 5. Samples for user-defined text prompts from our model for text-to-image synthesis, *LDM-8 (KL)*, which was trained on the LAION [78] database. Samples generated with 200 DDIM steps and $\eta = 1.0$. We use unconditional guidance [32] with $s = 10.0$.

